SUPPLY MANAGEMENT SYSTEM — Functional Specification Document

FSD Addendum 44 — Financial Posting Gateway & QuickBooks Integration

Property	Value
Document ID	SMS-FSD-ADD-044
Version	1.0
Date	2026-10-16
Status	Draft — Pending Approval
Author	Muhammad Ilyas / Claude (AI-Assisted)
Classification	Internal — Development
Supersedes	None — new middleware layer under `finance` schema
Architecture	.NET 8 / EF Core 8 / SQL Server 2022 / React 18
Change Items	18 new features
Impact	16 migrations, 14 new entities, 8 new services, 9 new UI panels, 2 Hangfire jobs
Depends On	Addendum 42 (Chart of Accounts — CoA hierarchy, ExternalSystemMappings), Addendum 43 (Multi-Tenant — OrgId isolation, module licensing), Addendum 37 (Module Registry — Finance module activation)
Deferred Items	General Ledger Journal Engine (full trial balance, ledger reports, period closing), AR/AP Sub-Ledgers, Bank Reconciliation, Multi-Currency Engine, Tax Configuration & Reporting, Financial Statements Generator, Xero Integration Adapter, Tally Integration Adapter, ERPNext Integration Adapter, Inbound Bank Transaction Sync, Inbound Entity Pull from QB (scheduled full import) — see §29
Naming Convention	Finance entities use standard accounting terminology. Gateway contracts use `FinancialTransaction`, `FinancialPostingGateway`. All entity names are singular. Field names use snake_case in database, PascalCase in C#.

Table of Contents

1. [Purpose & Scope](#1-purpose--scope)
2. [Background & Context](#2-background--context)
3. [Definitions](#3-definitions)
4. [Change Items Overview](#4-change-items-overview)
5. [C1 — Financial Posting Gateway Contract](#5-c1--financial-posting-gateway-

contract)
6. [C2 — Finance DbContext (Separation-Ready)](#6-c2--finance-dbcontext-separation-ready)
7. [C3 — Tenant Finance Configuration & External System Connections](#7-c3--tenant-finance-configuration--external-system-connections)
8. [C4 — Financial Posting Gateway Implementation](#8-c4--financial-posting-gateway-implementation)
9. [C5 — External Account & Entity Mapping Tables](#9-c5--external-account--entity-mapping-tables)
10. [C6 — Provider-Specific Defaults & Enrichment Layer](#10-c6--provider-specific-defaults--enrichment-layer)
11. [C7 — Financial Posting Outbox & Background Processor](#11-c7--financial-posting-outbox--background-processor)
12. [C8 — QuickBooks Online Integration Target](#12-c8--quickbooks-online-integration-target)
13. [C9 — Sync Log & Reconciliation Dashboard](#13-c9--sync-log--reconciliation-dashboard)
14. [C10 — Core Module Integration Points](#14-c10--core-module-integration-points)
15. [C11 — Entity Sync Gateway Contract](#15-c11--entity-sync-gateway-contract)
16. [C12 — Entity Sync Gateway Implementation](#16-c12--entity-sync-gateway-implementation)
17. [C13 — QuickBooks Master Data Sync Adapter](#17-c13--quickbooks-master-data-sync-adapter)
18. [C14 — Entity Sync Outbox & Background Jobs](#18-c14--entity-sync-outbox--background-jobs)
19. [C15 — Tax Code Mapping](#19-c15--tax-code-mapping)
20. [C16 — Pre-Sync Validation & Reference Checks](#20-c16--pre-sync-validation--reference-checks)
21. [C17 — Document Lifecycle Gateway (Void / Cancel / Reversal)](#21-c17--document-lifecycle-gateway-void--cancel--reversal)
22. [C18 — Sync Control Configuration & POS Hold/Release](#22-c18--sync-control-configuration--pos-holdrelease)
23. [Database Migrations](#23-database-migrations)
24. [API Endpoints](#24-api-endpoints)
25. [UI Wireframes & Specifications](#25-ui-wireframes--specifications)
26. [Business Rules](#26-business-rules)
27. [Test Scenarios](#27-test-scenarios)
28. [Development Phases](#28-development-phases)
29. [Deferred Items](#29-deferred-items)

30. [Feature Coverage Matrix](#30-feature-coverage-matrix)

---

1. Purpose & Scope

This addendum introduces the Financial Posting Gateway — a middleware layer inside SMS that sits between core business modules (Sales, Purchase, POS, Inventory, Manufacturing) and all financial targets (internal GL, QuickBooks Online, and future external systems). It establishes the single entry point for every financial transaction in SMS and delivers the first external integration: QuickBooks Online.

1.1 What This Addendum Delivers

1. **Financial Posting Gateway** — A middleware service (`IFinancialPostingGateway`) that every SMS module calls when a business transaction has financial impact. The gateway validates the entry, checks tenant configuration, and routes to the appropriate target(s): internal GL, QuickBooks, or both.

2. **Canonical Financial Transaction Model** — A universal DTO (`FinancialTransaction`) that represents any financial posting in a standardized format. Core modules build this model using SMS-only fields. They have zero knowledge of QuickBooks, Xero, or any external system.

3. **Provider Enrichment Layer** — The gateway enriches the canonical transaction with provider-specific fields (QB Class, QB Department, QB Terms, Xero Tracking Categories) from configuration tables before dispatching to external targets. These fields never pollute the core SMS models.

4. **QuickBooks Online Integration** — The first external accounting adapter. Pushes SMS invoices, bills, payments, and credit notes to QuickBooks Online as native QB documents. Uses OAuth 2.0 authentication, the outbox pattern for guaranteed delivery, and mapping tables for account/entity translation.

5. **Entity Sync Gateway** — A second middleware service (`IEntitySyncGateway`) that handles master data synchronization between SMS and external systems. When a Customer, Vendor, or Product is created or updated in SMS, the gateway pushes the change to all configured external providers (QuickBooks Customer, QB

Vendor, QB Item). Core modules call one interface — they never call QuickBooks, Xero, or any external API directly for entity sync.

6. **QuickBooks Master Data Sync** — Bidirectional sync of Customers ↔ QB Customers, Vendors ↔ QB Vendors, Products ↔ QB Items, and CoA Accounts ↔ QB Accounts. Outbound push on create/update, with initial import support for pulling existing QB entities into the mapping table. Uses the same outbox pattern, same OAuth connection, same sync log as financial postings.

7. **Separation-Ready Architecture** — The finance module uses its own `FinanceDbContext` scoped to the `finance` schema. No cross-schema foreign keys. No navigation properties to sales/purchase entities. Today it runs in the same process and database. Tomorrow the DI registration can swap to an HTTP client pointing at an extracted Finance microservice — zero changes in any core module.

1.2 What This Addendum Does NOT Deliver

Excluded Item	Reason	Planned Addendum
General Ledger Journal Engine (full trial balance, period closing, ledger reports)	Requires this gateway foundation first	Addendum 46
AR/AP Sub-Ledgers	Depends on GL Journal Engine	Addendum 47
Bank Reconciliation	Depends on GL + inbound sync	Addendum 48
Multi-Currency Engine	Cross-cutting — requires all financial entities	Addendum 49
Tax Configuration & Reporting	Requires GL + AR/AP	Addendum 50
Financial Statements Generator	Requires all sub-ledgers	Addendum 51
Xero / Tally / ERPNext Adapters	Gateway supports them; adapters built on demand	Future addendum
Inbound Bank Transaction Sync	Outbound push first, inbound later	Addendum 48
Scheduled Full Entity Import from QB	Initial import covered; scheduled recurring pull is future scope	Future addendum

1.3 Design Principles

- **Single Gateway, Single Call** — Every module makes one call: `_gateway.PostAsync(transaction)`. The gateway decides who gets the posting. Modules never call GL, QuickBooks, or any financial service directly.

- **Core Modules Stay Clean** — `FinancialTransaction` lives in the shared kernel (`SMS.Core`). It contains only SMS-domain fields (AccountCode, PartnerId, Lines with Debit/Credit). No QB Class, no Xero Tracking, no provider-specific fields.

- **Enrichment at the Boundary** — Provider-specific fields are loaded from configuration tables inside the finance module, just before dispatching to external targets. Core modules never populate, reference, or know about these fields.

- **Internal = Synchronous, External = Async** — Internal GL posting adds entities to the same DbContext (same transaction as the source document). External API calls go through the outbox table and are processed by a Hangfire background job. This ensures the source document and internal GL are always atomically consistent, while external sync is eventually consistent with retry logic.

- **Idempotency by Default** — Every posting carries an idempotency key (`{module}:{docType}:{docId}`). Duplicate postings are detected and skipped. This is defensive today (monolith) and essential tomorrow (if extracted to a separate service with network between them).

- **Module License Gate** — If the Finance module is not licensed for a tenant, the gateway returns `NoTargetConfigured` and the source document still posts successfully. Core SMS operates independently of whether finance is enabled.

- **Entity Sync Through Middleware** — Customer, Vendor, and Product synchronization with external systems goes through `IEntitySyncGateway`, not through direct API calls from core modules. When `CustomerService.CreateAsync()` saves a new customer, it calls `_entitySync.PushAsync()` — it never imports `QuickBooksCustomerService` or knows about QB's `POST /customer` endpoint. This decouples master data management from external integrations the same way the Financial Posting Gateway decouples transactions.

- **Outbound-First, Import-Assisted** — Entity sync is primarily outbound: SMS is the master for Customers, Vendors, and Products. An initial import feature pulls existing QB entities to create mapping entries so users don't have to manually map hundreds of records. But ongoing sync is SMS → QB (create/update

push), not continuous bidirectional polling.

- **Separation-Ready from Day One** — The finance module uses its own `FinanceDbContext` with no cross-schema joins or foreign keys. `JournalEntry.SourceDocumentId` is a plain `int`, not a navigation property to `SaleInvoice`. When extraction happens, swap the DI registration from the in-process gateway to an HTTP client — all core modules continue working without code changes.

---

2. Background & Context

2.1 Current Architecture (Pre-Addendum 44)

SMS currently has **tightly coupled** financial calls scattered across core modules. When a sale invoice is posted, the code directly calls accounting functions from within the sales service. This creates hard dependencies between modules that should be independent.

```
CURRENT STATE (Tightly Coupled)
─────────────────────────────────

SaleInvoiceService.PostAsync()
{
    invoice.Status = Posted;
    _db.SaveChanges();

    // DIRECT CALL — Sales module depends on Finance internals
    _accountingService.CreateJournalEntry(...);    // ← tight coupling
    _accountingService.UpdateAR(...);              // ← tight coupling
    _quickBooksService.PushInvoice(...);           // ← tight coupling
}
```

Problems with this approach:
- Sales module has a hard dependency on accounting service classes
- Cannot deploy sales changes without recompiling finance
- Cannot license finance separately — it's wired into every module

- Adding QuickBooks means modifying every module that creates financial transactions
- No central validation — each module builds GL entries differently
- No consistent error handling for external API failures

2.2 Target Architecture (Post-Addendum 44)

```
TARGET STATE (Loosely Coupled via Gateway)
──────────────────────────────────────────

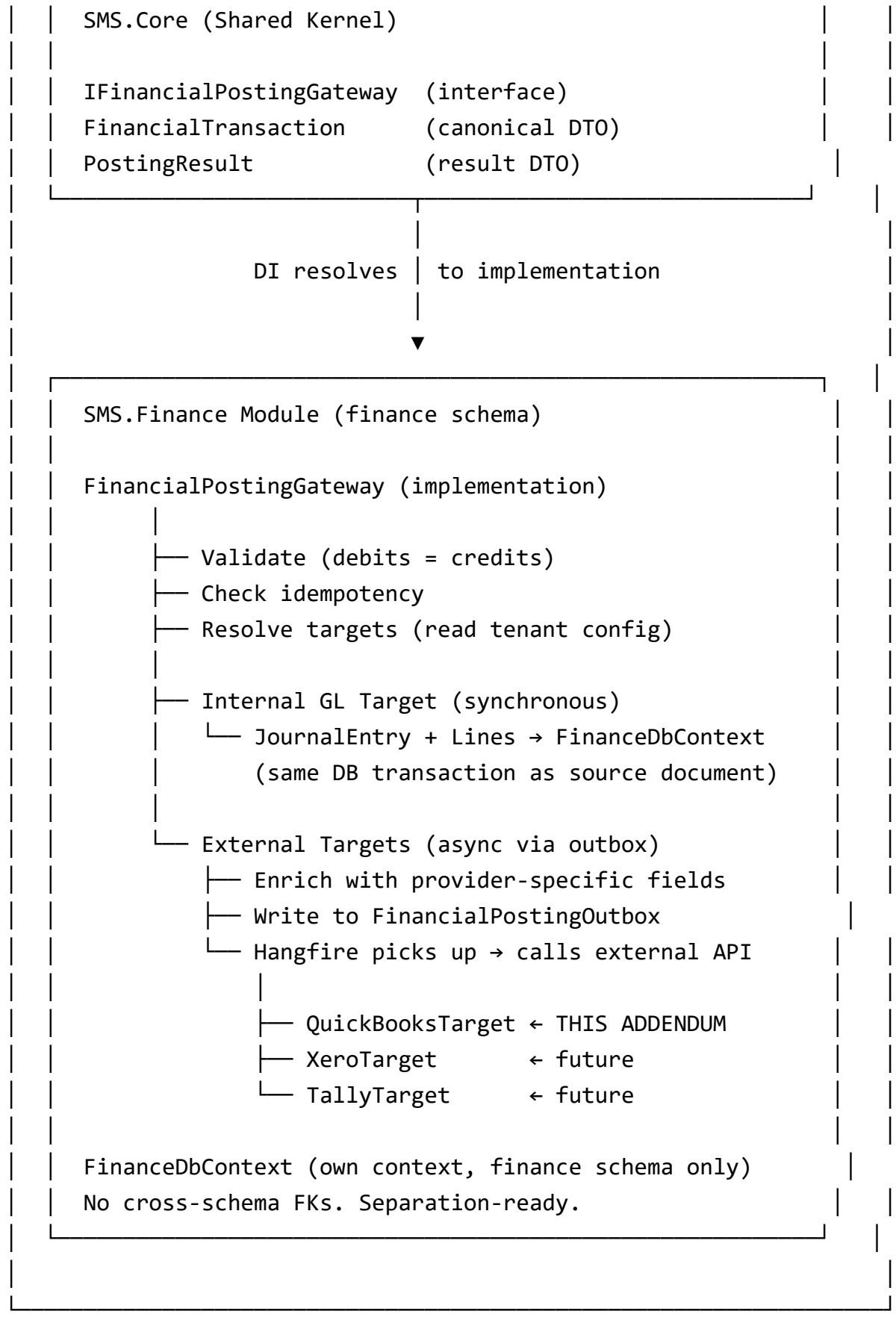

SaleInvoiceService.PostAsync()
{
    invoice.Status = Posted;

    // ONE call to gateway — Sales knows nothing about GL or QuickBooks
    await _gateway.PostAsync(new FinancialTransaction { ... });

    _db.SaveChanges();   // Commits invoice + GL + outbox atomically
}
```

2.3 Architectural Position of the Gateway

```
┌──────────────────────────────────────────────────────────────────────┐
│  SMS APPLICATION (Modular Monolith — Single Deployment)              │
│                                                                      │
│   ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐            │
│   │  Sales   │  │ Purchase │  │   POS    │  │Inventory │            │
│   │  Module  │  │  Module  │  │  Module  │  │  Module  │            │
│   │          │  │          │  │          │  │          │            │
│   │ Ref:     │  │ Ref:     │  │ Ref:     │  │ Ref:     │            │
│   │ SMS.Core │  │ SMS.Core │  │ SMS.Core │  │ SMS.Core │            │
│   └────┬─────┘  └────┬─────┘  └────┬─────┘  └────┬─────┘            │
│        │             │             │             │                  │
│        └─────────────┴─────────────┴─────────────┘                  │
│                      │             │                                │
│                      ▼             ▼                                │
│   ┌──────────────────────────────────────────────────────┐           │
```




```

### 2.4 Dependency Direction
```

```
SMS.Sales   ──references──►  SMS.Core (IFinancialPostingGateway)
SMS.Purchase ──references──► SMS.Core
SMS.POS     ──references──►  SMS.Core
SMS.Finance ──references──►  SMS.Core
SMS.Finance ──implements──►  IFinancialPostingGateway

Sales does NOT reference Finance.  ✓
Finance does NOT reference Sales.  ✓
Both reference Core.               ✓

At runtime, DI resolves:
  IFinancialPostingGateway → FinancialPostingGateway (in-process, today)
  IFinancialPostingGateway → HttpFinanceClient (HTTP, future extraction)
```

---

3. Definitions

Term	Meaning
Financial Posting Gateway	The middleware service (`FinancialPostingGateway`) that routes financial transactions from core modules to internal GL and/or external accounting systems. Single entry point for all financial postings.
Canonical Transaction	`FinancialTransaction` — the standardized DTO carrying a balanced set of debit/credit lines. Uses SMS-only fields. Lives in `SMS.Core`.
Financial Target	A destination that receives financial postings. Targets are either internal (GL journal engine, same DB) or external (QuickBooks, Xero — via API).
Target Resolver	Service that reads tenant configuration and returns the list of active financial targets for that tenant.
Provider Enrichment	The process of loading provider-specific fields (QB Class, Xero Tracking) from configuration tables and attaching them to the posting payload before dispatching to an external target.
Enriched Payload	`EnrichedFinancialPayload` — carries the canonical `FinancialTransaction` plus mapped account IDs, mapped entity IDs, and

provider-specific default fields. Only exists inside the finance module. |
Financial Posting Outbox	Database table (`finance.FinancialPostingOutbox`) that stores external postings for async processing. Hangfire reads pending entries and calls external APIs. Provides guaranteed delivery with retry logic.
Internal GL Target	Writes journal entries directly to `finance.JournalEntries` in the same database transaction as the source document. Synchronous.
QuickBooks Target	Pushes transactions to QuickBooks Online via the QBO REST API. Asynchronous via outbox. Maps SMS documents to native QB entities (Invoice, Bill, Payment, SalesReceipt, JournalEntry).
Account Mapping	Translation of SMS Chart of Account codes to external system account identifiers. Stored per tenant, per provider.
Entity Mapping	Translation of SMS business entities (Customers, Vendors, Products) to external system entity IDs. Stored per tenant, per provider.
Idempotency Key	Unique string (`{module}:{docType}:{docId}`) attached to every posting. Prevents duplicate financial entries when the same source document is processed twice.
Sync Log	Record of each document pushed to an external system, tracking SMS document ID ⇔ external ID, sync timestamp, and status. Used for reconciliation.
FinanceDbContext	Separate EF Core `DbContext` scoped to the `finance` schema. Uses the same connection string as `ApplicationDbContext` today. Swappable to a separate database for future extraction.
Entity Sync Gateway	The middleware service (`EntitySyncGateway`) that routes master data create/update operations from core modules to external systems. Handles Customers, Vendors, Products, and CoA Accounts. Single entry point for all entity synchronization.
Entity Sync Request	`EntitySyncRequest` — the standardized DTO carrying entity type, action (Create/Update), and the serialized entity payload. Lives in `SMS.Core`. Core modules build this DTO; they never know which external system(s) will receive it.
Entity Sync Outbox	Database table (`finance.EntitySyncOutbox`) that stores outbound entity sync operations for async processing. Same pattern as `FinancialPostingOutbox` — Hangfire reads pending entries and calls external APIs with retry logic.
Entity Sync Adapter	A provider-specific class that translates SMS entity data to external system format and pushes via API. `QuickBooksMasterDataAdapter` is the first adapter delivered in this addendum.

|
Initial Import	One-time bulk pull of existing entities from an external system (e.g., QuickBooks) to create mapping entries in `ExternalEntityMappings`. The user triggers this from the UI after connecting a provider. It does NOT overwrite SMS entities — it only creates mapping links.
Entity Mapping Coverage	The percentage of SMS entities (Customers, Vendors, Products) that have corresponding mappings in `ExternalEntityMappings` for a given provider. 100% coverage means every SMS entity has a known external counterpart — sync will not fail due to missing mappings.
Tax Code Mapping	Translation of SMS tax codes/rates to external system tax identifiers. For QuickBooks: SMS tax code → QB TaxCode.Id. Stored per tenant, per provider in `finance.ExternalTaxMappings`.
Pre-Sync Validation	A validation pass that runs inside the outbox processor before calling the external API. Checks that all referenced entities (customers, vendors, products, accounts, tax codes) have valid mappings. Prevents API calls that would fail due to missing references.
Document Lifecycle Operation	A non-posting action on a previously synced document: void (cancel in QB), credit (issue credit memo), or deactivate. Goes through the gateway like postings but calls different QB endpoints (e.g., `POST /void` instead of `POST /invoice`).
Sync Control Configuration	Per-document-type toggle that controls whether a specific transaction type (POS Sales, Supplier Bills, etc.) is synced to an external provider. Allows phased rollout — enable Sales Invoice sync first, enable POS sync later.
POS Hold/Release	When POS sync is disabled, completed POS transactions are held in the outbox with status `Held`. When POS sync is enabled, held entries are released to `Pending` for processing. No transactions are lost during the disabled period.
Reconciliation Comparison	Automated comparison of SMS document totals vs external system document totals for a period. Identifies mismatches (amount differences, missing documents, status conflicts) and generates a reconciliation report.

---

4. Change Items Overview

ID	Change Item	Type	New Entities	Schema
C1	Financial Posting Gateway Contract	Interface + DTOs	— (in SMS.Core)

| – |
C2	Finance DbContext (Separation-Ready)	Infrastructure	– (DbContext class)	`finance`
C3	Tenant Finance Configuration & External System Connections	New Entities	`TenantFinanceConfigs`, `ExternalSystemConnections`	`finance`
C4	Financial Posting Gateway Implementation	Service Class	– (service + resolver)	`finance`
C5	External Account & Entity Mapping Tables	New Entities	`ExternalAccountMappings`, `ExternalEntityMappings`	`finance`
C6	Provider-Specific Defaults & Enrichment Layer	New Entities + Service	`ExternalProviderDefaults`, `ExternalDocumentOverrides`	`finance`
C7	Financial Posting Outbox & Background Processor	New Entity + Hangfire Job	`FinancialPostingOutbox`	`finance`
C8	QuickBooks Online Integration Target	Service + OAuth Flow	– (adapter class + API client)	–
C9	Sync Log & Reconciliation Dashboard	New Entity + UI	`SyncLogs`	`finance`
C10	Core Module Integration Points	Code Changes	– (modify existing services)	`sales`, `purchase`, `pos`, `inventory`
C11	Entity Sync Gateway Contract	Interface + DTOs	– (in SMS.Core)	–
C12	Entity Sync Gateway Implementation	Service Class	– (service class)	`finance`
C13	QuickBooks Master Data Sync Adapter	Service + Mapping Logic	– (adapter class)	`finance`
C14	Entity Sync Outbox & Background Jobs	New Entity + Hangfire Job	`EntitySyncOutbox`	`finance`
C15	Tax Code Mapping	New Entity + Service	`ExternalTaxMappings`	`finance`
C16	Pre-Sync Validation & Reference Checks	Service Class	– (validation service)	`finance`
C17	Document Lifecycle Gateway (Void / Cancel / Reversal)	Interface Extension + Service	– (gateway extension)	`finance`
C18	Sync Control Configuration & POS Hold/Release	New Entity + Service	`SyncControlConfigs`	`finance`

---

5. C1 – Financial Posting Gateway Contract

Location: `SMS.Core/Contracts/Finance/`

These types live in the shared kernel so every module can reference them without depending on the finance module.

5.1 Interface: `IFinancialPostingGateway`

```csharp
namespace SMS.Core.Contracts.Finance;

/// <summary>
/// The SINGLE interface that all SMS modules use for financial postings.
/// Implementation lives in SMS.Finance; resolved via DI.
/// Core modules reference this interface only — never the implementation.
/// </summary>
public interface IFinancialPostingGateway
{
    /// <summary>
    /// Posts a financial transaction to all configured targets
    /// (internal GL, QuickBooks, etc.) based on tenant configuration.
    ///
    /// Adds entities to the DbContext but does NOT call SaveChangesAsync().
    /// The caller's SaveChangesAsync() commits everything atomically:
    /// source document + GL entries + outbox rows.
    /// </summary>
    Task<PostingResult> PostAsync(FinancialTransaction transaction);

    /// <summary>
    /// Reverses a previously posted transaction across all targets.
    /// Creates reversing journal entries and queues reversal requests
    /// for external systems.
    /// </summary>
    Task<PostingResult> ReverseAsync(
        string sourceModule,
        FinDocType documentType,
        int sourceDocumentId,
        int orgId,
        string reason);
}
```

5.2 DTO: `FinancialTransaction`

```csharp
namespace SMS.Core.Contracts.Finance;

/// <summary>
/// Canonical financial transaction model. Every SMS module builds this DTO
/// when a business transaction has financial impact.
///
/// Contains ONLY SMS-domain fields. No QuickBooks Class, no Xero Tracking,
/// no provider-specific fields. Provider enrichment happens inside the
/// gateway (SMS.Finance module) before dispatching to external targets.
/// </summary>
public class FinancialTransaction
{
    // ── Source Document Identity ──
    public FinDocType DocumentType { get; set; }
    public int SourceDocumentId { get; set; }
    public string SourceDocumentNumber { get; set; } = string.Empty;
    public string SourceModule { get; set; } = string.Empty;

    // ── When ──
    public DateTime TransactionDate { get; set; }

    // ── Who (Business Partner) ──
    public int? PartnerId { get; set; }
    public PartnerType? PartnerType { get; set; }

    // ── The GL Entry (Debit/Credit Lines) ──
    public List<FinancialTransactionLine> Lines { get; set; } = new();

    // ── Currency ──
    public string CurrencyCode { get; set; } = "USD";
    public decimal? ExchangeRate { get; set; }

    // ── Description ──
    public string? Description { get; set; }

    // ── Document-Level Metadata (for external systems that create
    //    native documents, not just journal entries) ──
```

```
    public FinancialDocumentMetadata? Metadata { get; set; }

    // ── Tenant Context ──
    public int OrgId { get; set; }

    // ── Idempotency ──
    public string IdempotencyKey =>
        $"{SourceModule}:{DocumentType}:{SourceDocumentId}";
}
```

5.3 DTO: `FinancialTransactionLine`

```csharp
namespace SMS.Core.Contracts.Finance;

public class FinancialTransactionLine
{
    /// <summary>
    /// SMS Chart of Account code (Level 4 leaf account).
    /// Example: "1200" for Accounts Receivable.
    /// The gateway maps this to external account IDs using
ExternalAccountMappings.
    /// </summary>
    public string AccountCode { get; set; } = string.Empty;

    public decimal Debit { get; set; }
    public decimal Credit { get; set; }

    /// <summary>Human-readable label. "Accounts Receivable", "Revenue", etc.
</summary>
    public string? Label { get; set; }

    /// <summary>For AR/AP lines — links to the customer or vendor ID.
</summary>
    public int? PartnerId { get; set; }

    /// <summary>Optional cost center code for management reporting.</summary>
    public string? CostCenter { get; set; }
```

```
    /// <summary>Optional project code for project accounting.</summary>
    public string? Project { get; set; }
}
```

5.4 DTO: `FinancialDocumentMetadata`

```csharp
namespace SMS.Core.Contracts.Finance;

/// <summary>
/// Rich line-item detail that external systems need to create proper
/// native documents (invoices, bills) rather than just journal entries.
/// Optional — only populated when the source is a document with line items.
/// </summary>
public class FinancialDocumentMetadata
{
    public List<DocumentLineItem>? Items { get; set; }
    public decimal? SubTotal { get; set; }
    public decimal? TaxTotal { get; set; }
    public decimal? GrandTotal { get; set; }
    public DateTime? DueDate { get; set; }
    public string? PaymentTerms { get; set; }
    public string? Reference { get; set; }
    public int? LinkedDocumentId { get; set; }
    public string? BankAccountCode { get; set; }
}

public class DocumentLineItem
{
    public int ProductId { get; set; }
    public string? ProductName { get; set; }
    public string? ProductSku { get; set; }
    public decimal Quantity { get; set; }
    public decimal UnitPrice { get; set; }
    public decimal? DiscountAmount { get; set; }
    public decimal? TaxAmount { get; set; }
    public string? TaxCode { get; set; }
    public decimal LineTotal { get; set; }
    public string? Description { get; set; }
```

```
}
```

5.5 Enums

```csharp
namespace SMS.Core.Contracts.Finance;

public enum FinDocType
{
    SalesInvoice,
    SalesReturn,
    CreditNote,
    Payment,
    PurchaseBill,
    PurchaseReturn,
    BillPayment,
    POSSale,
    POSReturn,
    InventoryAdjustment,
    ProductionCost,
    ProductionCompletion,
    BankFee,
    ManualJournal,
    OpeningBalance
}

public enum PartnerType
{
    Customer,
    Vendor
}

public enum PostingStatus
{
    Success,
    PartialFailure,
    NoTargetConfigured,
    DuplicateSkipped,
    ValidationFailed,
```

```
    ModuleNotLicensed
}

public enum TargetPostingStatus
{
    Posted,
    Queued,
    Failed,
    Skipped
}
```

5.6 Result DTOs

```csharp
namespace SMS.Core.Contracts.Finance;

public class PostingResult
{
    public PostingStatus Status { get; set; }
    public string? Message { get; set; }
    public int SourceDocumentId { get; set; }
    public FinDocType DocumentType { get; set; }
    public List<TargetResult> TargetResults { get; set; } = new();
}

public class TargetResult
{
    public string TargetCode { get; set; } = string.Empty;
    public TargetPostingStatus Status { get; set; }
    public string? InternalId { get; set; }
    public string? ExternalId { get; set; }
    public string? Error { get; set; }
}
```

---

6. C2 — Finance DbContext (Separation-Ready)

6.1 Rationale

The finance module uses its own `FinanceDbContext` even though it connects to the same SQL Server database. This enforces a clean boundary:

- Finance entities are only accessible through `FinanceDbContext`
- No `DbSet<SaleInvoice>` in `FinanceDbContext` — no cross-schema navigation
- `JournalEntry.SourceDocumentId` is a plain `int`, not a foreign key to any sales/purchase table
- Today: same connection string as `ApplicationDbContext`
- Future: change connection string to point to a separate Finance database

6.2 DbContext Definition

```csharp
namespace SMS.Finance.Data;

public class FinanceDbContext : DbContext
{
    // ── Finance-Owned Entities ──
    public DbSet<TenantFinanceConfig> TenantFinanceConfigs =>
Set<TenantFinanceConfig>();
    public DbSet<ExternalSystemConnection> ExternalSystemConnections =>
Set<ExternalSystemConnection>();
    public DbSet<ExternalAccountMapping> ExternalAccountMappings =>
Set<ExternalAccountMapping>();
    public DbSet<ExternalEntityMapping> ExternalEntityMappings =>
Set<ExternalEntityMapping>();
    public DbSet<ExternalProviderDefault> ExternalProviderDefaults =>
Set<ExternalProviderDefault>();
    public DbSet<ExternalDocumentOverride> ExternalDocumentOverrides =>
Set<ExternalDocumentOverride>();
    public DbSet<FinancialPostingOutbox> PostingOutbox =>
Set<FinancialPostingOutbox>();
    public DbSet<SyncLog> SyncLogs => Set<SyncLog>();
    public DbSet<JournalEntry> JournalEntries => Set<JournalEntry>();
    public DbSet<JournalEntryLine> JournalEntryLines => Set<JournalEntryLine>
();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
```

```
    {
        modelBuilder.HasDefaultSchema("finance");

        // Multi-tenant global query filter on all entities
        // OrgId resolved from ITenantContext (same pattern as
ApplicationDbContext)

        // CRITICAL: No navigation properties to sales, purchase, or other
schemas.
        // JournalEntry.SourceDocumentId is a plain int — NOT an FK.
    }
}
```

6.3 Registration

```csharp
// Program.cs
// TODAY: Same connection string — same database
builder.Services.AddDbContext<FinanceDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")));

// FUTURE EXTRACTION: Change to separate connection string
// builder.Services.AddDbContext<FinanceDbContext>(options =>
//     options.UseSqlServer(
//         builder.Configuration.GetConnectionString("FinanceDatabase")));
```

6.4 Transaction Coordination

While both contexts point to the same database, `TransactionScope` wraps them so that the source document and financial entries commit atomically:

```csharp
// In the calling service (e.g., SaleInvoiceService)
using var scope = new
TransactionScope(TransactionScopeAsyncFlowOption.Enabled);

invoice.Status = InvoiceStatus.Posted;
```

```
await _gateway.PostAsync(BuildTransaction(invoice));

await _appDb.SaveChangesAsync();         // Invoice update
// Gateway internally added entities to FinanceDbContext
// FinanceDbContext.SaveChangesAsync() called inside gateway

scope.Complete();  // Both commits succeed or both roll back
```

6.5 Cross-Schema Rules

Rule	Enforcement
No FK from `finance.*` to `sales.*`, `purchase.*`, `pos.*`, `inventory.*`	Code review + EF model validation
No `Include()` / navigation property across schemas	`FinanceDbContext` has no DbSet for non-finance entities
Source document reference stored as plain `int` + `string`	`SourceDocumentId` (int), `SourceDocumentType` (string), `SourceModule` (string)
If finance needs document detail, it came via `FinancialTransaction` DTO	Gateway receives all needed data in the DTO; never queries source tables
All finance queries go through `FinanceDbContext`	No `_appDb.Set<JournalEntry>()` calls anywhere

---

7. C3 — Tenant Finance Configuration & External System Connections

7.1 Entity: `finance.TenantFinanceConfigs`

Stores per-tenant finance module settings. Determines which targets the gateway routes to.

Column	Type	Constraints	Description
`id`	`int`	PK, Identity	Auto-increment primary key
`org_id`	`int`	FK → Organizations.Id, Unique, Not Null	One config per tenant
`internal_accounting_enabled`	`bit`	Not Null, Default `0`	Whether

internal GL journal entries are created |
`auto_post_to_gl`	`bit`	Not Null, Default `1`	If true, GL entries are posted immediately. If false, they are created in Draft status for manual review.
`default_currency_code`	`nvarchar(3)`	Not Null, Default `'USD'`	Tenant's base reporting currency
`fiscal_year_start_month`	`int`	Not Null, Default `1`	Month (1–12) when fiscal year begins
`created_at`	`datetimeoffset`	Not Null, Default `SYSDATETIMEOFFSET()`	Record creation
`created_by`	`nvarchar(450)`	Not Null	UserId of creator
`updated_at`	`datetimeoffset`	Nullable	Last update
`updated_by`	`nvarchar(450)`	Nullable	UserId of updater

Indexes:

Index Name	Columns	Type
`IX_TenantFinanceConfigs_OrgId`	`org_id`	Unique

7.2 Entity: `finance.ExternalSystemConnections`

Stores OAuth credentials and connection details for each external accounting system a tenant has connected.

Column	Type	Constraints	Description
`id`	`int`	PK, Identity	Auto-increment primary key
`org_id`	`int`	FK → Organizations.Id, Not Null	Owning tenant
`provider_code`	`nvarchar(50)`	Not Null	`"quickbooks"`, `"xero"`, `"tally"`, `"erpnext"`
`display_name`	`nvarchar(200)`	Not Null	User-facing label, e.g., "Our QuickBooks"
`is_active`	`bit`	Not Null, Default `1`	Whether this connection is currently active
`realm_id`	`nvarchar(100)`	Nullable	QuickBooks: Company ID (Realm ID)
`access_token_encrypted`	`nvarchar(max)`	Nullable	Encrypted OAuth 2.0 access token
`refresh_token_encrypted`	`nvarchar(max)`	Nullable	Encrypted OAuth 2.0 refresh token

`token_expires_at`	`datetimeoffset`	Nullable	When current access token expires
`environment`	`nvarchar(20)`	Not Null, Default `'production'`	`"sandbox"` or `"production"`
`sync_mode`	`nvarchar(20)`	Not Null, Default `'document_push'`	`"document_push"` (native docs) or `"journal_entry_push"` (JE only)
`last_sync_at`	`datetimeoffset`	Nullable	Timestamp of last successful sync
`last_error`	`nvarchar(max)`	Nullable	Last error message (for diagnostics)
`created_at`	`datetimeoffset`	Not Null, Default `SYSDATETIMEOFFSET()`	Record creation
`created_by`	`nvarchar(450)`	Not Null	UserId
`updated_at`	`datetimeoffset`	Nullable	Last update
`updated_by`	`nvarchar(450)`	Nullable	UserId

Indexes:

Index Name	Columns	Type
`IX_ExternalSystemConnections_OrgId_Provider`	`org_id`, `provider_code`	Unique
`IX_ExternalSystemConnections_IsActive`	`org_id`, `is_active`	Non-unique (filtered)

Security Note: `access_token_encrypted` and `refresh_token_encrypted` are encrypted at rest using ASP.NET Core Data Protection API. They are never logged, never returned in API responses, and never serialized to the outbox payload.

7.3 Tenant Scenarios

Scenario	TenantFinanceConfig	ExternalSystemConnections	Gateway Behavior
No finance license	No row (or module not in ModuleActivations)	—	Returns `ModuleNotLicensed`. Invoice still posts.
Internal GL only	`internal_accounting_enabled = true`	No rows	Creates JournalEntry. No outbox.
QuickBooks only	`internal_accounting_enabled = false`	1 row:

`quickbooks`, active | No JournalEntry. Writes to outbox for QB push. |
| **Internal GL + QuickBooks** | `internal_accounting_enabled = true` | 1 row: `quickbooks`, active | Creates JournalEntry AND writes to outbox. |
| **Internal GL + QuickBooks + Xero** | `internal_accounting_enabled = true` | 2 rows: `quickbooks` + `xero`, both active | JournalEntry + two outbox entries (one per provider). |

---

8. C4 — Financial Posting Gateway Implementation

8.1 Location

`SMS.Finance/Gateway/FinancialPostingGateway.cs`

8.2 Gateway Implementation

```csharp
namespace SMS.Finance.Gateway;

public class FinancialPostingGateway : IFinancialPostingGateway
{
    private readonly FinanceDbContext _financeDb;
    private readonly ITargetResolver _targetResolver;
    private readonly IJournalEntryBuilder _journalBuilder;
    private readonly IProviderEnrichmentService _enrichment;
    private readonly ILogger<FinancialPostingGateway> _logger;

    public async Task<PostingResult> PostAsync(FinancialTransaction
transaction)
    {
        var result = new PostingResult
        {
            SourceDocumentId = transaction.SourceDocumentId,
            DocumentType = transaction.DocumentType
        };

        // ── STEP 1: Module License Check ──
        var financeEnabled = await _financeDb.Set<ModuleActivation>()
            .AnyAsync(m => m.OrgId == transaction.OrgId
```

```
                    && m.ModuleCode == "FINANCE"
                    && m.IsActive);

        if (!financeEnabled)
        {
            result.Status = PostingStatus.ModuleNotLicensed;
            return result;
        }

        // ── STEP 2: Validate — Debits Must Equal Credits ──
        var totalDebits = transaction.Lines.Sum(l => l.Debit);
        var totalCredits = transaction.Lines.Sum(l => l.Credit);

        if (Math.Abs(totalDebits - totalCredits) > 0.001m)
        {
            result.Status = PostingStatus.ValidationFailed;
            result.Message = $"Unbalanced entry: Debits ({totalDebits}) !=
Credits ({totalCredits})";
            return result;
        }

        // ── STEP 3: Idempotency Check ──
        var duplicateExists = await _financeDb.JournalEntries
            .AnyAsync(j => j.IdempotencyKey == transaction.IdempotencyKey
                    && j.OrgId == transaction.OrgId);

        if (duplicateExists)
        {
            result.Status = PostingStatus.DuplicateSkipped;
            result.Message = $"Duplicate posting:
{transaction.IdempotencyKey}";
            _logger.LogWarning("Duplicate posting detected: {Key}",
transaction.IdempotencyKey);
            return result;
        }

        // ── STEP 4: Resolve Targets ──
        var config = await _targetResolver.ResolveAsync(transaction.OrgId);

        if (!config.HasAnyTarget)
```

```
{
    result.Status = PostingStatus.NoTargetConfigured;
    return result;
}

// ── STEP 5: Internal GL (Synchronous — Same Transaction) ──
if (config.InternalAccountingEnabled)
{
    try
    {
        var je = await _journalBuilder.BuildAsync(transaction);
        _financeDb.JournalEntries.Add(je);

        result.TargetResults.Add(new TargetResult
        {
            TargetCode = "internal_gl",
            Status = TargetPostingStatus.Posted,
            InternalId = je.EntryNumber
        });
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Internal GL posting failed for {Key}",
            transaction.IdempotencyKey);

        result.TargetResults.Add(new TargetResult
        {
            TargetCode = "internal_gl",
            Status = TargetPostingStatus.Failed,
            Error = ex.Message
        });
    }
}

// ── STEP 6: External Targets (Async via Outbox) ──
foreach (var connection in config.ExternalConnections)
{
    try
    {
        var enrichedPayload = await _enrichment.EnrichAsync(
```

```
            transaction, connection.ProviderCode);

        _financeDb.PostingOutbox.Add(new FinancialPostingOutbox
        {
            ProviderCode = connection.ProviderCode,
            DocumentType = transaction.DocumentType.ToString(),
            SourceDocumentId = transaction.SourceDocumentId,
            SourceDocumentNumber = transaction.SourceDocumentNumber,
            SourceModule = transaction.SourceModule,
            Payload = JsonSerializer.Serialize(enrichedPayload),
            IdempotencyKey = transaction.IdempotencyKey,
            Status = "Pending",
            RetryCount = 0,
            CreatedAt = DateTime.UtcNow,
            OrgId = transaction.OrgId
        });

        result.TargetResults.Add(new TargetResult
        {
            TargetCode = connection.ProviderCode,
            Status = TargetPostingStatus.Queued
        });
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Failed to enqueue for {Provider}: {Key}",
            connection.ProviderCode, transaction.IdempotencyKey);

        result.TargetResults.Add(new TargetResult
        {
            TargetCode = connection.ProviderCode,
            Status = TargetPostingStatus.Failed,
            Error = ex.Message
        });
    }
}

// ── STEP 7: Save finance entities ──
await _financeDb.SaveChangesAsync();
```

```
        result.Status = result.TargetResults.All(r =>
            r.Status is TargetPostingStatus.Posted or
TargetPostingStatus.Queued)
            ? PostingStatus.Success
            : PostingStatus.PartialFailure;

        return result;
    }
}
```

8.3 Target Resolver

```csharp
namespace SMS.Finance.Gateway;

public class TargetResolverResult
{
    public bool InternalAccountingEnabled { get; set; }
    public List<ExternalSystemConnection> ExternalConnections { get; set; } =
new();
    public bool HasAnyTarget => InternalAccountingEnabled ||
ExternalConnections.Any();
}

public class TargetResolver : ITargetResolver
{
    private readonly FinanceDbContext _db;

    public async Task<TargetResolverResult> ResolveAsync(int orgId)
    {
        var result = new TargetResolverResult();

        var config = await _db.TenantFinanceConfigs
            .AsNoTracking()
            .FirstOrDefaultAsync(c => c.OrgId == orgId);

        if (config != null)
        {
            result.InternalAccountingEnabled =
```

```
config.InternalAccountingEnabled;
        }

        result.ExternalConnections = await _db.ExternalSystemConnections
            .AsNoTracking()
            .Where(c => c.OrgId == orgId && c.IsActive)
            .ToListAsync();

        return result;
    }
}
```

8.4 DI Registration

```csharp
// Program.cs or FinanceModuleExtensions.cs
builder.Services.AddScoped<IFinancialPostingGateway, FinancialPostingGateway>
();
builder.Services.AddScoped<ITargetResolver, TargetResolver>();
builder.Services.AddScoped<IJournalEntryBuilder, JournalEntryBuilder>();
builder.Services.AddScoped<IProviderEnrichmentService,
ProviderEnrichmentService>();
builder.Services.AddScoped<QuickBooksTarget>();

// FUTURE EXTRACTION: Swap to HTTP client
// builder.Services.AddScoped<IFinancialPostingGateway, HttpFinanceClient>();
```

---

9. C5 — External Account & Entity Mapping Tables

9.1 Entity: `finance.ExternalAccountMappings`

Maps SMS Chart of Account codes to external system account identifiers. Each tenant configures this per provider.

Column	Type	Constraints	Description

`id`	`int`	PK, Identity	Auto-increment primary key
`org_id`	`int`	FK → Organizations.Id, Not Null	Owning tenant
`provider_code`	`nvarchar(50)`	Not Null	`"quickbooks"`, `"xero"`, etc.
`sms_account_code`	`nvarchar(20)`	Not Null	SMS CoA Level 4 code, e.g., `"1200"`
`external_account_id`	`nvarchar(100)`	Not Null	Account ID in external system, e.g., QB `"84"`
`external_account_name`	`nvarchar(200)`	Nullable	Display name from external system, e.g., `"Accounts Receivable"`
`sync_direction`	`nvarchar(20)`	Not Null, Default `'outbound'`	`"outbound"`, `"inbound"`, `"bidirectional"`
`is_active`	`bit`	Not Null, Default `1`	Active mapping
`created_at`	`datetimeoffset`	Not Null, Default `SYSDATETIMEOFFSET()`	Created
`updated_at`	`datetimeoffset`	Nullable	Updated

Indexes:

Index Name	Columns	Type
`IX_ExternalAccountMappings_Lookup`	`org_id`, `provider_code`, `sms_account_code`	Unique

Relationship to Addendum 42: This table extends the integration-ready design established in C10 of Addendum 42, where `ExternalSystemMappings` was introduced. This addendum provides the detailed mapping at the account and entity level.

9.2 Entity: `finance.ExternalEntityMappings`

Maps SMS business entities (Customers, Vendors, Products) to their counterparts in external systems.

Column	Type	Constraints	Description
`id`	`int`	PK, Identity	Auto-increment primary key
`org_id`	`int`	FK → Organizations.Id, Not Null	Owning tenant
`provider_code`	`nvarchar(50)`	Not Null	`"quickbooks"`, `"xero"`, etc.

`sms_entity_type`	`nvarchar(50)`	Not Null	`"Customer"`, `"Vendor"`, `"Product"`
`sms_entity_id`	`int`	Not Null	PK of the entity in SMS
`external_entity_id`	`nvarchar(100)`	Not Null	ID in external system
`external_entity_name`	`nvarchar(200)`	Nullable	Display name in external system
`sync_direction`	`nvarchar(20)`	Not Null, Default `'outbound'`	`"outbound"`, `"inbound"`, `"bidirectional"`
`is_active`	`bit`	Not Null, Default `1`	Active mapping
`last_synced_at`	`datetimeoffset`	Nullable	Last sync timestamp
`created_at`	`datetimeoffset`	Not Null, Default `SYSDATETIMEOFFSET()`	Created
`updated_at`	`datetimeoffset`	Nullable	Updated

Indexes:

Index Name	Columns	Type
`IX_ExternalEntityMappings_Lookup`	`org_id`, `provider_code`, `sms_entity_type`, `sms_entity_id`	Unique
`IX_ExternalEntityMappings_ReverseLookup`	`org_id`, `provider_code`, `external_entity_id`	Non-unique

---

10. C6 — Provider-Specific Defaults & Enrichment Layer

10.1 Rationale

External accounting systems require fields that do not exist in SMS's domain model. QuickBooks has Class, Department, Terms, Location. Xero has Tracking Categories and Branding Themes. Tally has Voucher Types and Cost Centres. These fields must never pollute the canonical `FinancialTransaction` or any core SMS module.

The solution: store provider-specific defaults in configuration tables. The enrichment service loads them and attaches them to the payload just before dispatching to the external target.

10.2 Entity: `finance.ExternalProviderDefaults`

Column	Type	Constraints	Description
`id`	`int`	PK, Identity	Auto-increment primary key
`org_id`	`int`	FK → Organizations.Id, Not Null	Owning tenant
`provider_code`	`nvarchar(50)`	Not Null	`"quickbooks"`, `"xero"`, etc.
`setting_key`	`nvarchar(100)`	Not Null	Field name, e.g., `"DefaultClassRef"`, `"DefaultDepartmentRef"`
`setting_value`	`nvarchar(500)`	Not Null	Value, e.g., `"300000012"`, `"1"`
`scope`	`nvarchar(20)`	Not Null, Default `'Document'`	`"Document"` (per-doc), `"Line"` (per-line), `"Payment"` (payment-specific)
`description`	`nvarchar(500)`	Nullable	Admin-facing description of this setting
`created_at`	`datetimeoffset`	Not Null, Default `SYSDATETIMEOFFSET()`	Created
`updated_at`	`datetimeoffset`	Nullable	Updated

Indexes:

Index Name	Columns	Type
`IX_ExternalProviderDefaults_Lookup`	`org_id`, `provider_code`, `setting_key`	Unique

10.3 Entity: `finance.ExternalDocumentOverrides`

Per-document-type overrides. For example, POS sales might use a different QB Class than regular invoices.

Column	Type	Constraints	Description
`id`	`int`	PK, Identity	Auto-increment primary key
`org_id`	`int`	FK → Organizations.Id, Not Null	Owning tenant
`provider_code`	`nvarchar(50)`	Not Null	Provider
`sms_document_type`	`nvarchar(50)`	Not Null	`"SalesInvoice"`, `"POSSale"`, `"PurchaseBill"`, etc.
`setting_key`	`nvarchar(100)`	Not Null	Same keys as ExternalProviderDefaults

`setting_value`	`nvarchar(500)`	Not Null	Override value for this document type
`created_at`	`datetimeoffset`	Not Null, Default `SYSDATETIMEOFFSET()`	Created
`updated_at`	`datetimeoffset`	Nullable	Updated

Indexes:

Index Name	Columns	Type
`IX_ExternalDocumentOverrides_Lookup`	`org_id`, `provider_code`, `sms_document_type`, `setting_key`	Unique

10.4 QuickBooks Provider Default Keys

Setting Key	Scope	Example Value	QB API Field
`DefaultClassRef`	Document / Line	`"300000012"`	`ClassRef.value`
`DefaultDepartmentRef`	Document	`"1"`	`DepartmentRef.value`
`DefaultTermsRef`	Document	`"3"`	`SalesTermRef.value` (Net 30)
`DefaultPaymentMethodRef`	Payment	`"1"`	`PaymentMethodRef.value`
`DefaultDepositAccount`	Payment	`"35"`	`DepositToAccountRef.value`
`CustomField1Name`	Document	`"PO Number"`	`CustomField[0].Name`
`ShipAddrFormat`	Document	`"combined"`	How to format shipping address

10.5 Enrichment Service

```csharp
namespace SMS.Finance.Gateway;

public class ProviderEnrichmentService : IProviderEnrichmentService
{
    private readonly FinanceDbContext _db;

    public async Task<EnrichedFinancialPayload> EnrichAsync(
        FinancialTransaction transaction, string providerCode)
    {
        return new EnrichedFinancialPayload
        {
            Transaction = transaction,
```

```
            AccountMappings = await _db.ExternalAccountMappings
                .AsNoTracking()
                .Where(m => m.OrgId == transaction.OrgId
                         && m.ProviderCode == providerCode
                         && m.IsActive)
                .ToDictionaryAsync(m => m.SmsAccountCode, m =>
m.ExternalAccountId),

            EntityMappings = await _db.ExternalEntityMappings
                .AsNoTracking()
                .Where(m => m.OrgId == transaction.OrgId
                         && m.ProviderCode == providerCode
                         && m.IsActive)
                .ToDictionaryAsync(
                    m => $"{m.SmsEntityType}:{m.SmsEntityId}",
                    m => m.ExternalEntityId),

            ProviderFields = await LoadProviderFieldsAsync(
                transaction.OrgId, providerCode, transaction.DocumentType)
        };
    }

    private async Task<Dictionary<string, string>> LoadProviderFieldsAsync(
        int orgId, string providerCode, FinDocType docType)
    {
        var defaults = await _db.ExternalProviderDefaults
            .AsNoTracking()
            .Where(d => d.OrgId == orgId && d.ProviderCode == providerCode)
            .ToDictionaryAsync(d => d.SettingKey, d => d.SettingValue);

        var overrides = await _db.ExternalDocumentOverrides
            .AsNoTracking()
            .Where(d => d.OrgId == orgId
                     && d.ProviderCode == providerCode
                     && d.SmsDocumentType == docType.ToString())
            .ToDictionaryAsync(d => d.SettingKey, d => d.SettingValue);

        foreach (var kvp in overrides)
            defaults[kvp.Key] = kvp.Value;
```

```
        return defaults;
    }
}
```

10.6 Enriched Payload (Internal to Finance Module)

```csharp
namespace SMS.Finance.Gateway;

/// <summary>
/// Carries the canonical transaction plus provider-specific data.
/// This class NEVER leaves the finance module. Core modules never see it.
/// It is serialized into the outbox Payload column as JSON.
/// </summary>
public class EnrichedFinancialPayload
{
    public FinancialTransaction Transaction { get; set; } = null!;
    public Dictionary<string, string> AccountMappings { get; set; } = new();
    public Dictionary<string, string> EntityMappings { get; set; } = new();
    public Dictionary<string, string> ProviderFields { get; set; } = new();
}
```

---

11. C7 — Financial Posting Outbox & Background Processor

11.1 Entity: `finance.FinancialPostingOutbox`

Column	Type	Constraints	Description
`id`	`int`	PK, Identity	Auto-increment primary key
`org_id`	`int`	FK → Organizations.Id, Not Null	Owning tenant
`provider_code`	`nvarchar(50)`	Not Null	Target provider
`document_type`	`nvarchar(50)`	Not Null	`"SalesInvoice"`, `"Payment"`, etc.
`source_document_id`	`int`	Not Null	SMS document PK
`source_document_number`	`nvarchar(50)`	Nullable	Human-readable doc

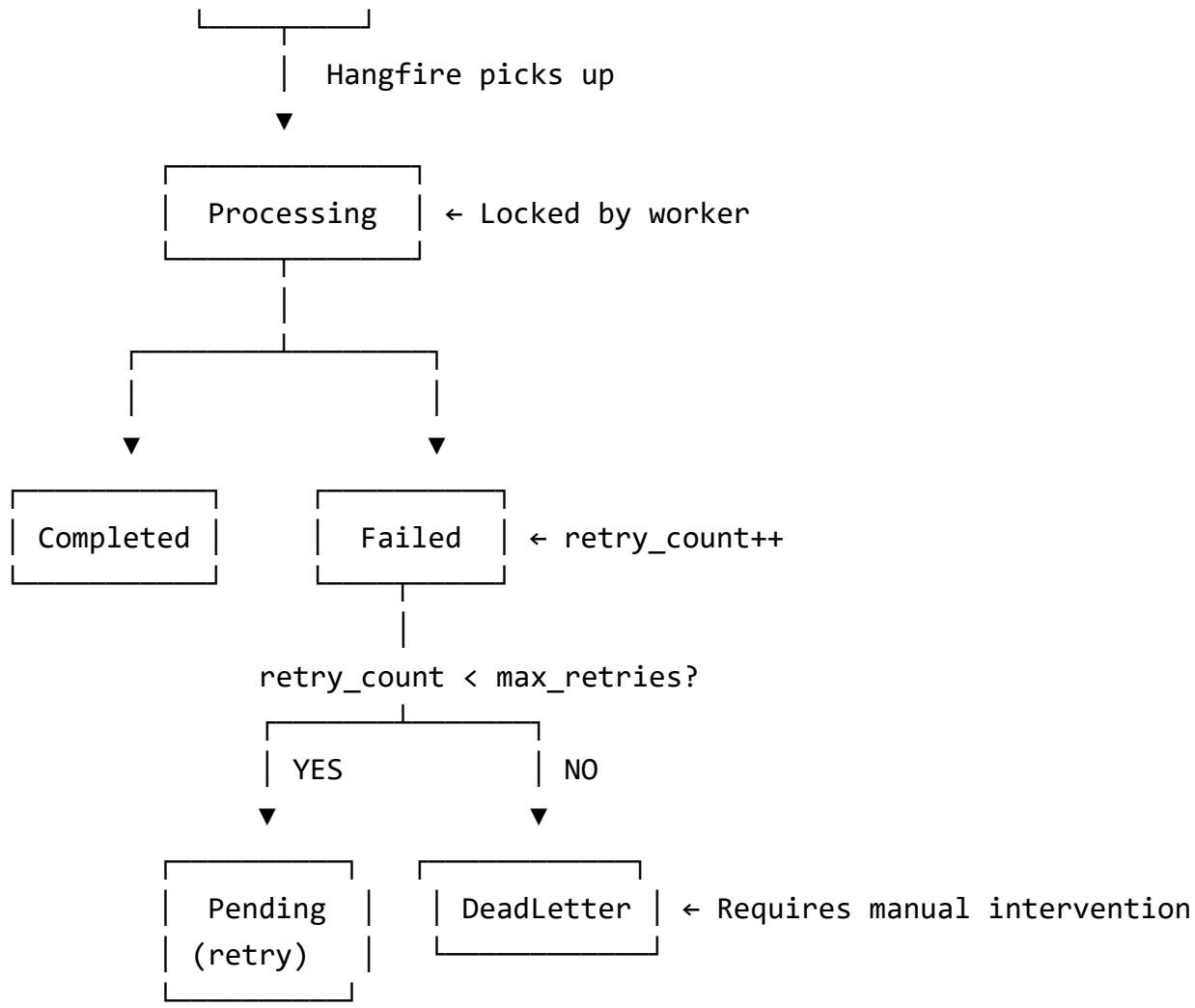
number |
`source_module`	`nvarchar(20)`	Not Null	`"sales"`, `"purchase"`, `"pos"`
`idempotency_key`	`nvarchar(200)`	Not Null	Unique key for duplicate detection
`payload`	`nvarchar(max)`	Not Null	Serialized `EnrichedFinancialPayload` JSON
`status`	`nvarchar(20)`	Not Null, Default `'Pending'`	`"Pending"`, `"Processing"`, `"Completed"`, `"Failed"`, `"DeadLetter"`
`retry_count`	`int`	Not Null, Default `0`	Number of processing attempts
`max_retries`	`int`	Not Null, Default `5`	Maximum retry attempts before dead-letter
`next_retry_at`	`datetimeoffset`	Nullable	Scheduled next retry (exponential backoff)
`external_id`	`nvarchar(100)`	Nullable	ID returned by external system on success
`error_message`	`nvarchar(max)`	Nullable	Last error details
`created_at`	`datetimeoffset`	Not Null, Default `SYSDATETIMEOFFSET()`	When enqueued
`processing_started_at`	`datetimeoffset`	Nullable	When processing began
`completed_at`	`datetimeoffset`	Nullable	When completed successfully

Indexes:

Index Name	Columns	Type
`IX_FinancialPostingOutbox_Pending`	`status`, `next_retry_at`	Non-unique (filtered: status IN ('Pending', 'Failed'))
`IX_FinancialPostingOutbox_Idempotency`	`org_id`, `idempotency_key`, `provider_code`	Unique
`IX_FinancialPostingOutbox_Source`	`org_id`, `source_module`, `source_document_id`	Non-unique

11.2 Outbox Status State Machine

```
                ┌─────────┐
                │ Pending │ ← Created by gateway
```




```

### 11.3 Hangfire Background Processor

```csharp
namespace SMS.Finance.Jobs;

/// <summary>
/// Hangfire recurring job that processes pending outbox entries.
/// Runs every 30 seconds. Picks up entries in FIFO order with row locking.
/// </summary>
public class OutboxProcessor
{
    private readonly FinanceDbContext _db;
    private readonly IServiceProvider _serviceProvider;
    private readonly ILogger<OutboxProcessor> _logger;

    [AutomaticRetry(Attempts = 0)]  // Outbox handles its own retries
```
```

```
    public async Task ProcessPendingAsync()
    {
        var pendingEntries = await _db.PostingOutbox
            .Where(o => o.Status == "Pending"
                    && (o.NextRetryAt == null || o.NextRetryAt <=
DateTime.UtcNow))
            .OrderBy(o => o.CreatedAt)
            .Take(10)  // Process 10 at a time
            .ToListAsync();

        foreach (var entry in pendingEntries)
        {
            entry.Status = "Processing";
            entry.ProcessingStartedAt = DateTime.UtcNow;
            await _db.SaveChangesAsync();

            try
            {
                var payload =
JsonSerializer.Deserialize<EnrichedFinancialPayload>(entry.Payload)!;
                var target = ResolveTarget(entry.ProviderCode);

                var result = await target.PushAsync(payload, entry.OrgId);

                if (result.Success)
                {
                    entry.Status = "Completed";
                    entry.ExternalId = result.ExternalId;
                    entry.CompletedAt = DateTime.UtcNow;
                }
                else
                {
                    HandleFailure(entry, result.ErrorMessage);
                }
            }
            catch (Exception ex)
            {
                _logger.LogError(ex, "Outbox processing failed for entry {Id}",
entry.Id);
                HandleFailure(entry, ex.Message);
```

```
            }

            await _db.SaveChangesAsync();
        }
    }

    private void HandleFailure(FinancialPostingOutbox entry, string?
errorMessage)
    {
        entry.RetryCount++;
        entry.ErrorMessage = errorMessage;

        if (entry.RetryCount >= entry.MaxRetries)
        {
            entry.Status = "DeadLetter";
            _logger.LogError(
                "Outbox entry {Id} moved to dead letter after {Count} retries.
" +
                "Provider: {Provider}, Doc: {DocType}:{DocId}",
                entry.Id, entry.RetryCount, entry.ProviderCode,
                entry.DocumentType, entry.SourceDocumentId);
        }
        else
        {
            entry.Status = "Pending";
            // Exponential backoff: 30s, 2m, 8m, 32m, 2h
            var delaySeconds = Math.Pow(4, entry.RetryCount) * 30;
            entry.NextRetryAt =
DateTime.UtcNow.AddSeconds(Math.Min(delaySeconds, 7200));
        }
    }
}
```

11.4 Hangfire Registration

```csharp
// Program.cs
RecurringJob.AddOrUpdate<OutboxProcessor>(
    "financial-outbox-processor",
```

```
    x => x.ProcessPendingAsync(),
    "*/30 * * * * *");  // Every 30 seconds
```

---

12. C8 — QuickBooks Online Integration Target

12.1 Overview

The QuickBooks target pushes SMS financial transactions to QuickBooks Online via the QBO REST API v3. It supports two sync modes:

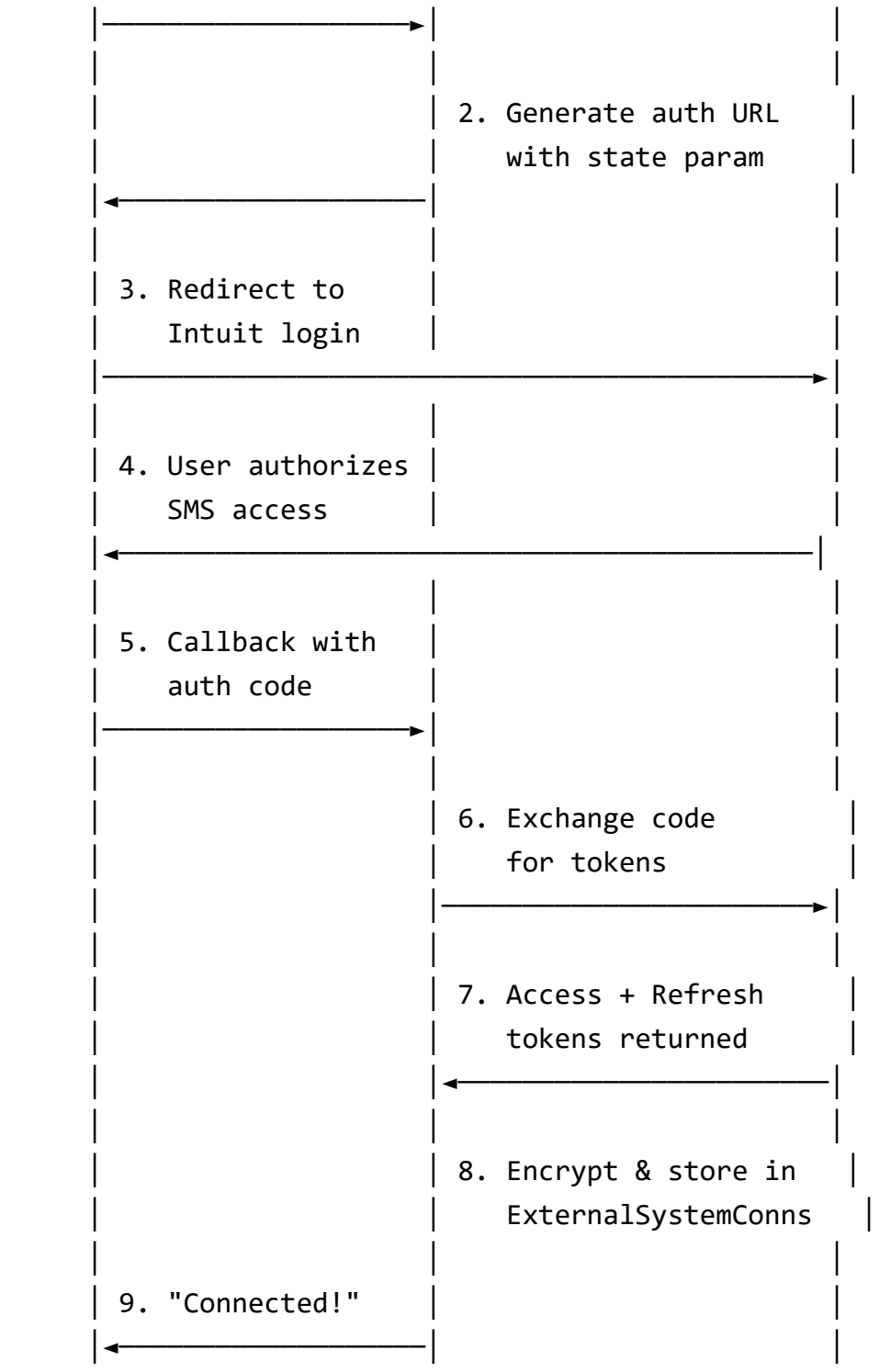
- **Document Push** (default): Creates native QuickBooks documents (Invoice, Bill, Payment, SalesReceipt, BillPayment). QuickBooks generates its own journal entries from these documents.
- **Journal Entry Push**: Sends everything as QBO Journal Entries. Simpler but loses document-level detail in QuickBooks.

12.2 QuickBooks Online API Reference

Property	Value
Base URL	`https://quickbooks.api.intuit.com` (production)
Sandbox URL	`https://sandbox-quickbooks.api.intuit.com`
Authentication	OAuth 2.0 (Authorization Code flow)
Rate Limit	500 requests/minute per realm, 10 concurrent requests
API Version	v3
Data Format	JSON

12.3 OAuth 2.0 Connection Flow

```
┌──────────┐       ┌──────────────┐       ┌──────────────────┐
│  SMS UI  │       │  SMS Backend │       │  Intuit OAuth    │
│  (React) │       │  (API)       │       │  Server          │
└────┬─────┘       └──────┬───────┘       └────────┬─────────┘
     │                    │                        │
     │ 1. Click           │                        │
     │ "Connect QB"       │                        │
```



12.4 Document Type Mapping (SMS → QuickBooks)

SMS FinDocType	QB Sync Mode: Document Push	QB Sync Mode: JE Push
`SalesInvoice`	`POST /invoice`	`POST /journalentry`
`Payment`	`POST /payment`	`POST /journalentry`
`CreditNote`	`POST /creditmemo`	`POST /journalentry`

`PurchaseBill`	`POST /bill`	`POST /journalentry`
`BillPayment`	`POST /billpayment`	`POST /journalentry`
`POSSale`	`POST /salesreceipt`	`POST /journalentry`
`POSReturn`	`POST /refundreceipt`	`POST /journalentry`
`InventoryAdjustment`	`POST /journalentry`	`POST /journalentry`
`ManualJournal`	`POST /journalentry`	`POST /journalentry`

12.5 QuickBooks Target Implementation

```csharp
namespace SMS.Finance.External;

public class QuickBooksTarget : IExternalAccountingTarget
{
    public string ProviderCode => "quickbooks";

    private readonly IQuickBooksApiClient _qbClient;
    private readonly ISyncLogService _syncLog;
    private readonly ILogger<QuickBooksTarget> _logger;

    public async Task<ExternalPushResult> PushAsync(
        EnrichedFinancialPayload payload, int orgId)
    {
        var txn = payload.Transaction;

        // Determine sync mode from connection config
        var connection = await _db.ExternalSystemConnections
            .FirstAsync(c => c.OrgId == orgId && c.ProviderCode ==
"quickbooks");

        object qbDocument;
        string qbEndpoint;

        if (connection.SyncMode == "document_push")
        {
            (qbDocument, qbEndpoint) = txn.DocumentType switch
            {
                FinDocType.SalesInvoice => (BuildQBInvoice(payload),
"invoice"),
                FinDocType.Payment     => (BuildQBPayment(payload), "payment"),
```

```
                FinDocType.CreditNote  => (BuildQBCreditMemo(payload),
"creditmemo"),
                FinDocType.PurchaseBill => (BuildQBBill(payload), "bill"),
                FinDocType.BillPayment => (BuildQBBillPayment(payload),
"billpayment"),
                FinDocType.POSSale     => (BuildQBSalesReceipt(payload),
"salesreceipt"),
                _                      => (BuildQBJournalEntry(payload),
"journalentry")
            };
        }
        else
        {
            qbDocument = BuildQBJournalEntry(payload);
            qbEndpoint = "journalentry";
        }

        var response = await _qbClient.PostAsync(
            connection.RealmId!, qbEndpoint, qbDocument, orgId);

        // Record in sync log
        await _syncLog.RecordAsync(new SyncLog
        {
            OrgId = orgId,
            ProviderCode = "quickbooks",
            SmsDocumentType = txn.DocumentType.ToString(),
            SmsDocumentId = txn.SourceDocumentId,
            SmsDocumentNumber = txn.SourceDocumentNumber,
            ExternalEntityType = qbEndpoint,
            ExternalId = response.Id,
            ExternalNumber = response.DocNumber,
            SyncDirection = "outbound",
            Status = response.IsSuccess ? "Synced" : "Failed",
            ErrorMessage = response.ErrorMessage,
            SyncedAt = DateTime.UtcNow
        });

        return new ExternalPushResult
        {
            Success = response.IsSuccess,
```

```
            ExternalId = response.Id,
            ErrorMessage = response.ErrorMessage
        };
    }

    private object BuildQBInvoice(EnrichedFinancialPayload payload)
    {
        var txn = payload.Transaction;
        var invoice = new Dictionary<string, object>
        {
            ["CustomerRef"] = new { value = payload.EntityMappings
                .GetValueOrDefault($"Customer:{txn.PartnerId}") },
            ["DocNumber"] = txn.SourceDocumentNumber,
            ["TxnDate"] = txn.TransactionDate.ToString("yyyy-MM-dd"),
            ["Line"] = txn.Metadata!.Items!.Select(item => new
Dictionary<string, object>
            {
                ["Amount"] = item.LineTotal,
                ["DetailType"] = "SalesItemLineDetail",
                ["SalesItemLineDetail"] = new
                {
                    ItemRef = new { value = payload.EntityMappings
                        .GetValueOrDefault($"Product:{item.ProductId}") },
                    Qty = item.Quantity,
                    UnitPrice = item.UnitPrice
                }
            }).ToList()
        };

        // ── Provider-Specific Fields (from ExternalProviderDefaults) ──
        if (payload.ProviderFields.TryGetValue("DefaultClassRef", out var cls))
            invoice["ClassRef"] = new { value = cls };

        if (payload.ProviderFields.TryGetValue("DefaultDepartmentRef", out var
dept))
            invoice["DepartmentRef"] = new { value = dept };

        if (payload.ProviderFields.TryGetValue("DefaultTermsRef", out var
terms))
            invoice["SalesTermRef"] = new { value = terms };
```

```
        if (txn.Metadata.DueDate.HasValue)
            invoice["DueDate"] = txn.Metadata.DueDate.Value.ToString("yyyy-MM-
dd");

        return invoice;
    }
}
```

12.6 QuickBooks API Client

```csharp
namespace SMS.Finance.External;

public interface IQuickBooksApiClient
{
    Task<QBApiResponse> PostAsync(string realmId, string endpoint, object
payload, int orgId);
    Task<QBApiResponse> GetAsync(string realmId, string endpoint, int orgId);
}

/// <summary>
/// Handles HTTP calls to QuickBooks Online API with automatic token refresh.
/// </summary>
public class QuickBooksApiClient : IQuickBooksApiClient
{
    private readonly HttpClient _httpClient;
    private readonly FinanceDbContext _db;

    public async Task<QBApiResponse> PostAsync(
        string realmId, string endpoint, object payload, int orgId)
    {
        var connection = await GetOrRefreshTokenAsync(orgId);
        var baseUrl = connection.Environment == "sandbox"
            ? "https://sandbox-quickbooks.api.intuit.com"
            : "https://quickbooks.api.intuit.com";

        var url = $"{baseUrl}/v3/company/{realmId}/{endpoint}?minorversion=65";
```

```
        var request = new HttpRequestMessage(HttpMethod.Post, url)
        {
            Content = JsonContent.Create(payload)
        };
        request.Headers.Authorization =
            new AuthenticationHeaderValue("Bearer",
DecryptToken(connection.AccessTokenEncrypted!));
        request.Headers.Accept.Add(new
MediaTypeWithQualityHeaderValue("application/json"));

        var response = await _httpClient.SendAsync(request);

        if (response.StatusCode == HttpStatusCode.Unauthorized)
        {
            // Token expired — refresh and retry once
            await RefreshTokenAsync(connection);
            request.Headers.Authorization =
                new AuthenticationHeaderValue("Bearer",
DecryptToken(connection.AccessTokenEncrypted!));
            response = await _httpClient.SendAsync(request);
        }

        var body = await response.Content.ReadAsStringAsync();

        return new QBApiResponse
        {
            IsSuccess = response.IsSuccessStatusCode,
            StatusCode = (int)response.StatusCode,
            Id = ExtractId(body),
            DocNumber = ExtractDocNumber(body),
            ErrorMessage = response.IsSuccessStatusCode ? null : body
        };
    }
}
```

12.7 Token Refresh Logic

QuickBooks access tokens expire after 1 hour. Refresh tokens expire after 100 days. The API client automatically refreshes expired access tokens.

```csharp
private async Task RefreshTokenAsync(ExternalSystemConnection connection)
{
    var tokenEndpoint =
"https://oauth.platform.intuit.com/oauth2/v1/tokens/bearer";
    var refreshToken = DecryptToken(connection.RefreshTokenEncrypted!);

    var request = new HttpRequestMessage(HttpMethod.Post, tokenEndpoint)
    {
        Content = new FormUrlEncodedContent(new[]
        {
            new KeyValuePair<string, string>("grant_type", "refresh_token"),
            new KeyValuePair<string, string>("refresh_token", refreshToken)
        })
    };
    request.Headers.Authorization = new AuthenticationHeaderValue("Basic",
        Convert.ToBase64String(Encoding.UTF8.GetBytes($"{_clientId}:
{_clientSecret}")));

    var response = await _httpClient.SendAsync(request);
    var tokens = await response.Content.ReadFromJsonAsync<OAuthTokenResponse>
();

    connection.AccessTokenEncrypted = EncryptToken(tokens!.AccessToken);
    connection.RefreshTokenEncrypted = EncryptToken(tokens.RefreshToken);
    connection.TokenExpiresAt = DateTime.UtcNow.AddSeconds(tokens.ExpiresIn);
    connection.UpdatedAt = DateTime.UtcNow;

    await _db.SaveChangesAsync();
}
```

---

13. C9 — Sync Log & Reconciliation Dashboard

13.1 Entity: `finance.SyncLogs`

| Column | Type | Constraints | Description |

|--------|------|-------------|-------------|
`id`	`bigint`	PK, Identity	Auto-increment (bigint for high volume)
`org_id`	`int`	FK → Organizations.Id, Not Null	Owning tenant
`provider_code`	`nvarchar(50)`	Not Null	Provider
`sms_document_type`	`nvarchar(50)`	Not Null	SMS document type
`sms_document_id`	`int`	Not Null	SMS document PK
`sms_document_number`	`nvarchar(50)`	Nullable	Human-readable number
`external_entity_type`	`nvarchar(50)`	Not Null	QB entity type (`"invoice"`, `"bill"`, etc.)
`external_id`	`nvarchar(100)`	Nullable	ID in external system
`external_number`	`nvarchar(100)`	Nullable	Doc number in external system
`sync_direction`	`nvarchar(20)`	Not Null	`"outbound"`, `"inbound"`
`status`	`nvarchar(20)`	Not Null	`"Synced"`, `"Failed"`, `"Pending"`
`error_message`	`nvarchar(max)`	Nullable	Error details
`synced_at`	`datetimeoffset`	Not Null	Timestamp
`created_at`	`datetimeoffset`	Not Null, Default `SYSDATETIMEOFFSET()`	Record creation

Indexes:

Index Name	Columns	Type
`IX_SyncLogs_SMSDoc`	`org_id`, `sms_document_type`, `sms_document_id`	Non-unique
`IX_SyncLogs_ExternalId`	`org_id`, `provider_code`, `external_id`	Non-unique
`IX_SyncLogs_Status`	`org_id`, `status`, `synced_at`	Non-unique

13.2 Reconciliation Dashboard UI

Location: Finance → Settings → External Integrations → Sync Status

Panel	Content
Connection Status	List of connected providers with status (Connected/Disconnected/Token Expired), last sync time, environment (Sandbox/Production)
Sync Summary	Cards showing: Total Synced (last 24h), Pending, Failed, Dead Letter — per provider

Sync Log Table	Filterable table: SMS Doc Number, Type, Provider, External ID, Status, Timestamp. Click row to see error details.
Dead Letter Queue	Failed entries that exceeded max retries. Admin can: Retry, Skip, View Error.
Mapping Coverage	Shows which SMS accounts/entities are mapped to external system and which are unmapped. Highlights missing mappings that would cause sync failures.

---

14. C10 — Core Module Integration Points

14.1 Integration Pattern

Every core module that creates financial transactions adds a dependency on `IFinancialPostingGateway` and calls `PostAsync()` after the business transaction is committed.

14.2 Sales Module Integration

File: `SMS.Sales/Services/SaleInvoiceService.cs`

Change: Add `IFinancialPostingGateway` dependency. Call gateway after setting invoice status to Posted.

```csharp
public class SaleInvoiceService
{
    private readonly ApplicationDbContext _appDb;
    private readonly IFinancialPostingGateway _finance;

    public async Task PostInvoiceAsync(int invoiceId)
    {
        using var scope = new TransactionScope(TransactionScopeAsyncFlowOption.Enabled);

        var invoice = await _appDb.SaleInvoices
            .Include(i => i.Lines)
            .FirstAsync(i => i.Id == invoiceId);
```

```
        invoice.Status = InvoiceStatus.Posted;

        await _finance.PostAsync(new FinancialTransaction
        {
            DocumentType = FinDocType.SalesInvoice,
            SourceDocumentId = invoice.Id,
            SourceDocumentNumber = invoice.InvoiceNumber,
            SourceModule = "sales",
            TransactionDate = invoice.InvoiceDate,
            PartnerId = invoice.CustomerId,
            PartnerType = PartnerType.Customer,
            CurrencyCode = invoice.CurrencyCode,
            OrgId = invoice.OrgId,
            Lines = new()
            {
                new() { AccountCode = "1200", Debit = invoice.GrandTotal,
                        PartnerId = invoice.CustomerId, Label = "Accounts
Receivable" },
                new() { AccountCode = "4100", Credit = invoice.SubTotal,
                        Label = "Sales Revenue" },
                new() { AccountCode = "2300", Credit = invoice.TaxTotal,
                        Label = "Tax Payable" }
            },
            Metadata = new()
            {
                Items = invoice.Lines.Select(l => new DocumentLineItem
                {
                    ProductId = l.ProductId,
                    ProductName = l.ProductName,
                    Quantity = l.Quantity,
                    UnitPrice = l.UnitPrice,
                    TaxAmount = l.TaxAmount,
                    LineTotal = l.LineTotal
                }).ToList(),
                SubTotal = invoice.SubTotal,
                TaxTotal = invoice.TaxTotal,
                GrandTotal = invoice.GrandTotal,
                DueDate = invoice.DueDate
            }
        });
```

```
        await _appDb.SaveChangesAsync();
        scope.Complete();
    }
}
```

14.3 Full Domain Event Catalog

Every business transaction with financial impact calls the gateway. The account codes shown below are defaults from standard templates; tenants may have different codes via their CoA configuration.

Module	Trigger Event	FinDocType	Debit Account(s)	Credit Account(s)
Sales	Invoice Posted	`SalesInvoice`	1200 AR	4100 Revenue + 2300 Tax
Sales	Payment Received	`Payment`	1100 Cash/Bank	1200 AR
Sales	Credit Note Issued	`CreditNote`	4100 Revenue + 2300 Tax	1200 AR
Sales	Sales Return	`SalesReturn`	4100 Revenue	1200 AR + 1131 Inventory
Purchase	Bill Posted	`PurchaseBill`	5100 COGS / 1131 Inventory	2100 AP
Purchase	Bill Payment Made	`BillPayment`	2100 AP	1100 Cash/Bank
Purchase	Purchase Return	`PurchaseReturn`	2100 AP	5100 COGS / 1131 Inventory
POS	Sale Completed	`POSSale`	1100 Cash/Card	4100 Revenue + 2300 Tax
POS	Return Processed	`POSReturn`	4100 Revenue + 2300 Tax	1100 Cash
Inventory	Stock Adjustment	`InventoryAdjustment`	1131 Inventory (if increase)	6100 Adjustment Expense
Manufacturing	Production Cost	`ProductionCost`	1132 WIP	1131 Raw Materials
Manufacturing	Production Complete	`ProductionCompletion`	1133 Finished Goods	1132 WIP
Finance	Manual Journal	`ManualJournal`	(user-specified)	(user-specified)

---

15. C11 — Entity Sync Gateway Contract

Location: `SMS.Core/Contracts/Finance/`

These types live in the shared kernel so every module can call entity sync without depending on the finance module.

15.1 Interface: `IEntitySyncGateway`

```csharp
namespace SMS.Core.Contracts.Finance;

/// <summary>
/// The SINGLE interface that all SMS modules use for entity synchronization
/// with external systems. Implementation lives in SMS.Finance; resolved via DI.
/// Core modules reference this interface only — never the implementation,
/// never any QuickBooks/Xero service directly.
/// </summary>
public interface IEntitySyncGateway
{
    /// <summary>
    /// Pushes an entity create/update to all configured external providers.
    /// Writes to the EntitySyncOutbox — never calls external APIs synchronously.
    /// The caller's SaveChangesAsync() commits the entity + outbox row atomically.
    /// </summary>
    Task<EntitySyncResult> PushAsync(EntitySyncRequest request);

    /// <summary>
    /// Pushes a batch of entity operations (e.g., bulk import from CSV).
    /// Each entity gets its own outbox entry for independent retry.
    /// </summary>
    Task<BatchEntitySyncResult> PushBatchAsync(IReadOnlyList<EntitySyncRequest> requests);

    /// <summary>
    /// Triggers initial import: pulls entities from an external provider
    /// and creates mapping entries in ExternalEntityMappings.
    /// Does NOT create SMS entities — only mapping links.
```

```
    /// Returns the count of entities imported.
    /// </summary>
    Task<ImportResult> ImportMappingsFromProviderAsync(
        int orgId,
        ExternalSystemType provider,
        SyncEntityType entityType);
}
```

15.2 DTO: `EntitySyncRequest`

```csharp
namespace SMS.Core.Contracts.Finance;

/// <summary>
/// Canonical entity sync request. Core modules build this DTO when
/// creating or updating a Customer, Vendor, or Product. They populate
/// SMS-domain fields only — provider-specific translation happens
/// inside the gateway.
/// </summary>
public class EntitySyncRequest
{
    /// <summary>Entity type being synced.</summary>
    public SyncEntityType EntityType { get; init; }

    /// <summary>Create, Update, or Deactivate.</summary>
    public SyncAction Action { get; init; }

    /// <summary>SMS primary key of the entity.</summary>
    public int EntityId { get; init; }

    /// <summary>Tenant identifier.</summary>
    public int OrgId { get; init; }

    /// <summary>
    /// Idempotency key: "{entityType}:{action}:{entityId}:{timestamp}".
    /// Prevents duplicate sync when the same save triggers multiple events.
    /// </summary>
    public string IdempotencyKey { get; init; } = null!;
```

```
    /// <summary>
    /// Serialized entity snapshot at the time of the request.
    /// The gateway reads this — it never queries back into the source module.
    /// </summary>
    public EntitySnapshot Snapshot { get; init; } = null!;

    /// <summary>Who triggered this sync (system or user email).</summary>
    public string InitiatedBy { get; init; } = "system";

    /// <summary>Optional: target only a specific provider (null = all
configured).</summary>
    public ExternalSystemType? TargetProvider { get; init; }
}
```

15.3 DTO: `EntitySnapshot`

```csharp
namespace SMS.Core.Contracts.Finance;

/// <summary>
/// Flattened snapshot of the entity at the moment sync was requested.
/// Contains all fields that any external provider might need.
/// The gateway NEVER queries ApplicationDbContext to get entity data —
/// it relies entirely on this snapshot (separation-ready).
/// </summary>
public class EntitySnapshot
{
    // ── Common Fields ──
    public string DisplayName { get; init; } = null!;
    public string? Email { get; init; }
    public string? Phone { get; init; }
    public bool IsActive { get; init; } = true;

    // ── Customer/Vendor Fields ──
    public string? CompanyName { get; init; }
    public string? FirstName { get; init; }
    public string? LastName { get; init; }
    public string? BillingAddressLine1 { get; init; }
    public string? BillingAddressCity { get; init; }
```

```
    public string? BillingAddressState { get; init; }
    public string? BillingAddressPostalCode { get; init; }
    public string? BillingAddressCountry { get; init; }
    public string? ShippingAddressLine1 { get; init; }
    public string? ShippingAddressCity { get; init; }
    public string? ShippingAddressState { get; init; }
    public string? ShippingAddressPostalCode { get; init; }
    public string? ShippingAddressCountry { get; init; }
    public string? TaxIdentificationNumber { get; init; }
    public string? CurrencyCode { get; init; }
    public int? PaymentTermDays { get; init; }

    // ── Product/Item Fields ──
    public string? Sku { get; init; }
    public string? Description { get; init; }
    public decimal? UnitPrice { get; init; }
    public decimal? PurchaseCost { get; init; }
    public string? IncomeAccountCode { get; init; }
    public string? ExpenseAccountCode { get; init; }
    public string? AssetAccountCode { get; init; }
    public bool? TrackQuantityOnHand { get; init; }
    public decimal? QuantityOnHand { get; init; }
    public string? UnitOfMeasure { get; init; }

    // ── Account Fields (CoA Sync) ──
    public string? AccountCode { get; init; }
    public string? AccountType { get; init; }
    public string? AccountSubType { get; init; }
}
```

15.4 Enums

```csharp
namespace SMS.Core.Contracts.Finance;

/// <summary>Entity types that can be synced to external systems.</summary>
public enum SyncEntityType
{
    Customer,
```

```
    Vendor,
    Product,
    Account   // CoA account
}

/// <summary>What operation triggered the sync.</summary>
public enum SyncAction
{
    Create,
    Update,
    Deactivate  // soft-delete — maps to "Active = false" in QB
}
```

15.5 Result DTOs

```csharp
namespace SMS.Core.Contracts.Finance;

public class EntitySyncResult
{
    public bool Success { get; init; }
    public EntitySyncOutcome Outcome { get; init; }
    public int OutboxEntriesCreated { get; init; }
    public string? ErrorMessage { get; init; }
}

public class BatchEntitySyncResult
{
    public int TotalRequests { get; init; }
    public int OutboxEntriesCreated { get; init; }
    public int DuplicatesSkipped { get; init; }
    public List<string> Errors { get; init; } = new();
}

public class ImportResult
{
    public bool Success { get; init; }
    public int EntitiesFound { get; init; }
    public int MappingsCreated { get; init; }
```

```
    public int MappingsAlreadyExist { get; init; }
    public string? ErrorMessage { get; init; }
}

public enum EntitySyncOutcome
{
    Queued,                // Outbox entries created for async processing
    NoTargetConfigured,    // No external connections configured
    ModuleNotLicensed,     // Finance module not licensed
    DuplicateSkipped,      // Idempotency key already processed
    ValidationFailed       // Entity data validation failed
}
```

15.6 Design Rules for `IEntitySyncGateway`

Rule	Detail
Lives in SMS.Core	Interface + all DTOs + enums in `SMS.Core.Contracts.Finance`.
No provider types	Nothing in this contract references QuickBooks, Xero, or any external system.
Snapshot-based	The gateway receives a snapshot — it never queries `ApplicationDbContext` for entity data. This keeps the contract serializable and separation-ready.
Outbox-only	`PushAsync()` writes to `EntitySyncOutbox` — it NEVER calls an external API synchronously. Unlike the financial posting gateway (which has a synchronous internal GL path), entity sync is always async because there is no "internal entity target."
Atomicity	`PushAsync()` adds outbox rows to `FinanceDbContext`. The caller's `SaveChangesAsync()` commits both the entity and the outbox row in one transaction.
Caller pattern	Core modules call `PushAsync()` after building the entity, before `SaveChangesAsync()`. Same pattern as `IFinancialPostingGateway.PostAsync()`.

---

16. C12 — Entity Sync Gateway Implementation

Location: `SMS.Finance/Services/EntitySyncGateway.cs`

16.1 Service Class

```csharp
namespace SMS.Finance.Services;

/// <summary>
/// Routes entity sync requests to configured external providers via the
outbox.
/// Mirrors the FinancialPostingGateway pattern:
///   1. Check module license
///   2. Check idempotency
///   3. Resolve target providers
///   4. Write one outbox entry per provider
///   5. Return result (caller's SaveChanges commits atomically)
/// </summary>
public class EntitySyncGateway : IEntitySyncGateway
{
    private readonly FinanceDbContext _financeDb;
    private readonly IEntitySyncTargetResolver _targetResolver;
    private readonly ILogger<EntitySyncGateway> _logger;

    public EntitySyncGateway(
        FinanceDbContext financeDb,
        IEntitySyncTargetResolver targetResolver,
        ILogger<EntitySyncGateway> logger)
    {
        _financeDb = financeDb;
        _targetResolver = targetResolver;
        _logger = logger;
    }

    public async Task<EntitySyncResult> PushAsync(EntitySyncRequest request)
    {
        // ── 1. Module License Check ──
        var licensed = await
_targetResolver.IsFinanceLicensedAsync(request.OrgId);
        if (!licensed)
        {
```

```
            _logger.LogDebug("Finance not licensed for OrgId {OrgId}, skipping
entity sync",
                request.OrgId);
            return new EntitySyncResult
            {
                Success = true,
                Outcome = EntitySyncOutcome.ModuleNotLicensed
            };
        }

        // ── 2. Idempotency Check ──
        var exists = await _financeDb.EntitySyncOutbox
            .AnyAsync(o => o.IdempotencyKey == request.IdempotencyKey
                        && o.OrgId == request.OrgId);
        if (exists)
        {
            _logger.LogDebug("Duplicate entity sync: {Key}",
request.IdempotencyKey);
            return new EntitySyncResult
            {
                Success = true,
                Outcome = EntitySyncOutcome.DuplicateSkipped
            };
        }

        // ── 3. Resolve Targets ──
        var targets = await _targetResolver
            .GetActiveProvidersAsync(request.OrgId, request.TargetProvider);

        if (targets.Count == 0)
        {
            return new EntitySyncResult
            {
                Success = true,
                Outcome = EntitySyncOutcome.NoTargetConfigured
            };
        }

        // ── 4. Write Outbox Entry Per Provider ──
        int entriesCreated = 0;
```

```
        foreach (var target in targets)
        {
            var outboxEntry = new EntitySyncOutbox
            {
                OrgId = request.OrgId,
                EntityType = request.EntityType.ToString(),
                EntityId = request.EntityId,
                Action = request.Action.ToString(),
                Provider = target.Provider.ToString(),
                ConnectionId = target.ConnectionId,
                PayloadJson = JsonSerializer.Serialize(request.Snapshot),
                IdempotencyKey = request.IdempotencyKey,
                Status = OutboxStatus.Pending,
                CreatedAtUtc = DateTime.UtcNow,
                RetryCount = 0,
                InitiatedBy = request.InitiatedBy
            };

            _financeDb.EntitySyncOutbox.Add(outboxEntry);
            entriesCreated++;
        }

        _logger.LogInformation(
            "Entity sync queued: {EntityType} {Action} #{EntityId} → {Count}
provider(s)",
            request.EntityType, request.Action, request.EntityId,
entriesCreated);

        // ── 5. Return — caller's SaveChangesAsync() commits everything ──
        return new EntitySyncResult
        {
            Success = true,
            Outcome = EntitySyncOutcome.Queued,
            OutboxEntriesCreated = entriesCreated
        };
    }

    public async Task<BatchEntitySyncResult> PushBatchAsync(
        IReadOnlyList<EntitySyncRequest> requests)
    {
```

```
        var result = new BatchEntitySyncResult { TotalRequests = requests.Count
};

        foreach (var request in requests)
        {
            var single = await PushAsync(request);
            if (single.Outcome == EntitySyncOutcome.DuplicateSkipped)
                result.DuplicatesSkipped++;
            else
                result.OutboxEntriesCreated += single.OutboxEntriesCreated;

            if (!single.Success)
                result.Errors.Add($"{request.EntityType}:{request.EntityId} —
{single.ErrorMessage}");
        }

        return result;
    }

    public async Task<ImportResult> ImportMappingsFromProviderAsync(
        int orgId,
        ExternalSystemType provider,
        SyncEntityType entityType)
    {
        // Delegates to the specific adapter — see C13
        var adapter = _targetResolver.GetAdapter(provider);
        if (adapter == null)
        {
            return new ImportResult
            {
                Success = false,
                ErrorMessage = $"No adapter registered for {provider}"
            };
        }

        return await adapter.ImportEntitiesAsync(orgId, entityType);
    }
}
```

16.2 Target Resolver

```csharp
namespace SMS.Finance.Services;

public interface IEntitySyncTargetResolver
{
    Task<bool> IsFinanceLicensedAsync(int orgId);
    Task<List<SyncTarget>> GetActiveProvidersAsync(int orgId,
ExternalSystemType? filter);
    IEntitySyncAdapter? GetAdapter(ExternalSystemType provider);
}

public class SyncTarget
{
    public ExternalSystemType Provider { get; init; }
    public int ConnectionId { get; init; }
}

/// <summary>
/// Reads ExternalSystemConnections for the tenant and returns providers
/// that are connected AND have entity sync enabled.
/// Reuses the same ExternalSystemConnections table from C3.
/// </summary>
public class EntitySyncTargetResolver : IEntitySyncTargetResolver
{
    private readonly FinanceDbContext _financeDb;
    private readonly ApplicationDbContext _appDb;
    private readonly IEnumerable<IEntitySyncAdapter> _adapters;

    public EntitySyncTargetResolver(
        FinanceDbContext financeDb,
        ApplicationDbContext appDb,
        IEnumerable<IEntitySyncAdapter> adapters)
    {
        _financeDb = financeDb;
        _appDb = appDb;
        _adapters = adapters;
    }
```

```
    public async Task<bool> IsFinanceLicensedAsync(int orgId)
    {
        return await _appDb.ModuleActivations
            .AnyAsync(m => m.OrgId == orgId
                        && m.ModuleCode == "FINANCE"
                        && m.IsActive);
    }

    public async Task<List<SyncTarget>> GetActiveProvidersAsync(
        int orgId, ExternalSystemType? filter)
    {
        var query = _financeDb.ExternalSystemConnections
            .Where(c => c.OrgId == orgId
                      && c.IsActive
                      && c.EntitySyncEnabled);

        if (filter.HasValue)
            query = query.Where(c => c.Provider == filter.Value);

        return await query.Select(c => new SyncTarget
        {
            Provider = c.Provider,
            ConnectionId = c.Id
        }).ToListAsync();
    }

    public IEntitySyncAdapter? GetAdapter(ExternalSystemType provider)
    {
        return _adapters.FirstOrDefault(a => a.Provider == provider);
    }
}
```

16.3 ExternalSystemConnections — New Field

Add `EntitySyncEnabled` column to the existing `ExternalSystemConnections`
entity from C3:

```csharp
// Added to existing entity from C3
```

```
public class ExternalSystemConnection
{
    // ... existing fields from C3 ...

    /// <summary>
    /// When true, entity create/update operations (Customers, Vendors,
Products)
    /// are pushed to this provider. Independent of financial posting — a
tenant
    /// can sync entities to QuickBooks without posting financial transactions.
    /// </summary>
    public bool EntitySyncEnabled { get; set; } = true;
}
```

16.4 DI Registration

```csharp
// In SMS.Finance/DependencyInjection.cs
services.AddScoped<IEntitySyncGateway, EntitySyncGateway>();
services.AddScoped<IEntitySyncTargetResolver, EntitySyncTargetResolver>();
services.AddScoped<IEntitySyncAdapter, QuickBooksMasterDataAdapter>();
// Future: services.AddScoped<IEntitySyncAdapter, XeroMasterDataAdapter>();
```

16.5 Design Rules for EntitySyncGateway

Rule	Detail
Same pattern as FinancialPostingGateway	License check → idempotency → resolve targets → write outbox → return.
Always async	Unlike financial postings (which have a synchronous internal GL path), entity sync is always outbox-based. There is no "internal entity target."
One outbox entry per provider	If tenant has QuickBooks AND Xero connected, one `PushAsync()` creates two outbox rows. Each retries independently.
Snapshot, not query-back	The gateway serializes `EntitySnapshot` into the outbox `PayloadJson`. The Hangfire processor reads the snapshot — it never queries `ApplicationDbContext` for entity data.

| **Reuses C3 connections** | Same `ExternalSystemConnections` table, same OAuth tokens, same `ExternalSystemType` enum. |

---

17. C13 — QuickBooks Master Data Sync Adapter

Location:
`SMS.Finance/Integrations/QuickBooks/QuickBooksMasterDataAdapter.cs`

17.1 Adapter Interface

```csharp
namespace SMS.Finance.Integrations;

/// <summary>
/// Provider-specific adapter for entity synchronization.
/// Each external system implements this interface.
/// The EntitySyncOutboxProcessor calls the adapter for the outbox entry's provider.
/// </summary>
public interface IEntitySyncAdapter
{
    ExternalSystemType Provider { get; }

    /// <summary>Push one entity to the external system.</summary>
    Task<AdapterResult> PushEntityAsync(EntitySyncOutbox outboxEntry);

    /// <summary>Pull existing entities to create mapping entries (initial import).</summary>
    Task<ImportResult> ImportEntitiesAsync(int orgId, SyncEntityType entityType);
}

public class AdapterResult
{
    public bool Success { get; init; }
    public string? ExternalId { get; init; }
    public string? ErrorMessage { get; init; }
    public bool Retryable { get; init; } = true;
```

```
}
```

17.2 QuickBooks Master Data Adapter

```csharp
namespace SMS.Finance.Integrations.QuickBooks;

/// <summary>
/// Handles entity sync between SMS and QuickBooks Online.
/// Maps:
///   SMS Customer  → QB Customer
///   SMS Vendor    → QB Vendor
///   SMS Product   → QB Item (Type = Inventory/NonInventory/Service)
///   SMS Account   → QB Account
///
/// Uses the same QuickBooksApiClient from C8 for OAuth and HTTP calls.
/// </summary>
public class QuickBooksMasterDataAdapter : IEntitySyncAdapter
{
    public ExternalSystemType Provider => ExternalSystemType.QuickBooksOnline;

    private readonly FinanceDbContext _financeDb;
    private readonly IQuickBooksApiClient _qbClient;
    private readonly ILogger<QuickBooksMasterDataAdapter> _logger;

    public QuickBooksMasterDataAdapter(
        FinanceDbContext financeDb,
        IQuickBooksApiClient qbClient,
        ILogger<QuickBooksMasterDataAdapter> logger)
    {
        _financeDb = financeDb;
        _qbClient = qbClient;
        _logger = logger;
    }

    public async Task<AdapterResult> PushEntityAsync(EntitySyncOutbox
outboxEntry)
    {
        var snapshot = JsonSerializer.Deserialize<EntitySnapshot>
```

```
(outboxEntry.PayloadJson)!;
        var action = Enum.Parse<SyncAction>(outboxEntry.Action);
        var entityType = Enum.Parse<SyncEntityType>(outboxEntry.EntityType);

        return entityType switch
        {
            SyncEntityType.Customer => await SyncCustomerAsync(outboxEntry,
snapshot, action),
            SyncEntityType.Vendor   => await SyncVendorAsync(outboxEntry,
snapshot, action),
            SyncEntityType.Product  => await SyncProductAsync(outboxEntry,
snapshot, action),
            SyncEntityType.Account  => await SyncAccountAsync(outboxEntry,
snapshot, action),
            _ => new AdapterResult
            {
                Success = false,
                ErrorMessage = $"Unsupported entity type: {entityType}",
                Retryable = false
            }
        };
    }

    // ── Customer Sync ──
    private async Task<AdapterResult> SyncCustomerAsync(
        EntitySyncOutbox entry, EntitySnapshot snapshot, SyncAction action)
    {
        // Check if mapping exists (update) or not (create)
        var mapping = await _financeDb.ExternalEntityMappings
            .FirstOrDefaultAsync(m => m.OrgId == entry.OrgId
                                   && m.EntityType == "Customer"
                                   && m.InternalEntityId == entry.EntityId
                                   && m.Provider ==
ExternalSystemType.QuickBooksOnline);

        var qbCustomer = new
        {
            DisplayName = snapshot.DisplayName,
            CompanyName = snapshot.CompanyName,
            GivenName = snapshot.FirstName,
```

```
        FamilyName = snapshot.LastName,
        PrimaryEmailAddr = snapshot.Email != null
            ? new { Address = snapshot.Email } : null,
        PrimaryPhone = snapshot.Phone != null
            ? new { FreeFormNumber = snapshot.Phone } : null,
        BillAddr = snapshot.BillingAddressLine1 != null ? new
        {
            Line1 = snapshot.BillingAddressLine1,
            City = snapshot.BillingAddressCity,
            CountrySubDivisionCode = snapshot.BillingAddressState,
            PostalCode = snapshot.BillingAddressPostalCode,
            Country = snapshot.BillingAddressCountry
        } : null,
        ShipAddr = snapshot.ShippingAddressLine1 != null ? new
        {
            Line1 = snapshot.ShippingAddressLine1,
            City = snapshot.ShippingAddressCity,
            CountrySubDivisionCode = snapshot.ShippingAddressState,
            PostalCode = snapshot.ShippingAddressPostalCode,
            Country = snapshot.ShippingAddressCountry
        } : null,
        Active = snapshot.IsActive,
        CurrencyRef = snapshot.CurrencyCode != null
            ? new { value = snapshot.CurrencyCode } : null
    };

    if (mapping != null && action == SyncAction.Update)
    {
        // UPDATE — QB requires SyncToken for optimistic locking
        var current = await _qbClient.GetAsync<dynamic>(
            entry.OrgId, $"customer/{mapping.ExternalEntityId}");
        var syncToken = current?.SyncToken;

        var result = await _qbClient.PostAsync(entry.OrgId, "customer", new
        {
            Id = mapping.ExternalEntityId,
            SyncToken = syncToken,
            sparse = true,
            // spread qbCustomer fields
            qbCustomer.DisplayName,
```

```
            qbCustomer.CompanyName,
            qbCustomer.GivenName,
            qbCustomer.FamilyName,
            qbCustomer.PrimaryEmailAddr,
            qbCustomer.PrimaryPhone,
            qbCustomer.BillAddr,
            qbCustomer.ShipAddr,
            qbCustomer.Active,
            qbCustomer.CurrencyRef
        });

        return new AdapterResult
        {
            Success = true,
            ExternalId = mapping.ExternalEntityId
        };
    }
    else
    {
        // CREATE
        var result = await _qbClient.PostAsync<dynamic>(
            entry.OrgId, "customer", qbCustomer);

        string qbId = result.Customer.Id.ToString();

        // Create mapping entry
        _financeDb.ExternalEntityMappings.Add(new ExternalEntityMapping
        {
            OrgId = entry.OrgId,
            EntityType = "Customer",
            InternalEntityId = entry.EntityId,
            Provider = ExternalSystemType.QuickBooksOnline,
            ExternalEntityId = qbId,
            ExternalEntityName = snapshot.DisplayName,
            LastSyncedAtUtc = DateTime.UtcNow,
            SyncStatus = "Synced"
        });
        await _financeDb.SaveChangesAsync();

        return new AdapterResult { Success = true, ExternalId = qbId };
```

```
        }
    }

    // ── Vendor Sync ──
    private async Task<AdapterResult> SyncVendorAsync(
        EntitySyncOutbox entry, EntitySnapshot snapshot, SyncAction action)
    {
        var mapping = await _financeDb.ExternalEntityMappings
            .FirstOrDefaultAsync(m => m.OrgId == entry.OrgId
                                   && m.EntityType == "Vendor"
                                   && m.InternalEntityId == entry.EntityId
                                   && m.Provider ==
ExternalSystemType.QuickBooksOnline);

        var qbVendor = new
        {
            DisplayName = snapshot.DisplayName,
            CompanyName = snapshot.CompanyName,
            GivenName = snapshot.FirstName,
            FamilyName = snapshot.LastName,
            PrimaryEmailAddr = snapshot.Email != null
                ? new { Address = snapshot.Email } : null,
            PrimaryPhone = snapshot.Phone != null
                ? new { FreeFormNumber = snapshot.Phone } : null,
            BillAddr = snapshot.BillingAddressLine1 != null ? new
            {
                Line1 = snapshot.BillingAddressLine1,
                City = snapshot.BillingAddressCity,
                CountrySubDivisionCode = snapshot.BillingAddressState,
                PostalCode = snapshot.BillingAddressPostalCode,
                Country = snapshot.BillingAddressCountry
            } : null,
            Active = snapshot.IsActive,
            TaxIdentifier = snapshot.TaxIdentificationNumber,
            CurrencyRef = snapshot.CurrencyCode != null
                ? new { value = snapshot.CurrencyCode } : null
        };

        if (mapping != null && action == SyncAction.Update)
        {
```

```
        var current = await _qbClient.GetAsync<dynamic>(
            entry.OrgId, $"vendor/{mapping.ExternalEntityId}");

        await _qbClient.PostAsync(entry.OrgId, "vendor", new
        {
            Id = mapping.ExternalEntityId,
            SyncToken = current?.SyncToken,
            sparse = true,
            qbVendor.DisplayName,
            qbVendor.CompanyName,
            qbVendor.GivenName,
            qbVendor.FamilyName,
            qbVendor.PrimaryEmailAddr,
            qbVendor.PrimaryPhone,
            qbVendor.BillAddr,
            qbVendor.Active,
            qbVendor.TaxIdentifier,
            qbVendor.CurrencyRef
        });

        return new AdapterResult
        {
            Success = true,
            ExternalId = mapping.ExternalEntityId
        };
    }
    else
    {
        var result = await _qbClient.PostAsync<dynamic>(
            entry.OrgId, "vendor", qbVendor);

        string qbId = result.Vendor.Id.ToString();

        _financeDb.ExternalEntityMappings.Add(new ExternalEntityMapping
        {
            OrgId = entry.OrgId,
            EntityType = "Vendor",
            InternalEntityId = entry.EntityId,
            Provider = ExternalSystemType.QuickBooksOnline,
            ExternalEntityId = qbId,
```

```
                ExternalEntityName = snapshot.DisplayName,
                LastSyncedAtUtc = DateTime.UtcNow,
                SyncStatus = "Synced"
            });
            await _financeDb.SaveChangesAsync();

            return new AdapterResult { Success = true, ExternalId = qbId };
        }
    }

    // ── Product/Item Sync ──
    private async Task<AdapterResult> SyncProductAsync(
        EntitySyncOutbox entry, EntitySnapshot snapshot, SyncAction action)
    {
        var mapping = await _financeDb.ExternalEntityMappings
            .FirstOrDefaultAsync(m => m.OrgId == entry.OrgId
                                   && m.EntityType == "Product"
                                   && m.InternalEntityId == entry.EntityId
                                   && m.Provider ==
ExternalSystemType.QuickBooksOnline);

        // Resolve QB account references from mappings
        string? incomeAccountRef = null;
        string? expenseAccountRef = null;
        string? assetAccountRef = null;

        if (snapshot.IncomeAccountCode != null)
        {
            var acctMap = await _financeDb.ExternalAccountMappings
                .FirstOrDefaultAsync(m => m.OrgId == entry.OrgId
                    && m.SmsAccountCode == snapshot.IncomeAccountCode
                    && m.Provider == ExternalSystemType.QuickBooksOnline);
            incomeAccountRef = acctMap?.ExternalAccountId;
        }

        if (snapshot.ExpenseAccountCode != null)
        {
            var acctMap = await _financeDb.ExternalAccountMappings
                .FirstOrDefaultAsync(m => m.OrgId == entry.OrgId
                    && m.SmsAccountCode == snapshot.ExpenseAccountCode
```

```
            && m.Provider == ExternalSystemType.QuickBooksOnline);
    expenseAccountRef = acctMap?.ExternalAccountId;
}

if (snapshot.AssetAccountCode != null)
{
    var acctMap = await _financeDb.ExternalAccountMappings
        .FirstOrDefaultAsync(m => m.OrgId == entry.OrgId
            && m.SmsAccountCode == snapshot.AssetAccountCode
            && m.Provider == ExternalSystemType.QuickBooksOnline);
    assetAccountRef = acctMap?.ExternalAccountId;
}

// Determine QB Item type
string itemType = snapshot.TrackQuantityOnHand == true
    ? "Inventory" : "NonInventory";

var qbItem = new
{
    Name = snapshot.DisplayName,
    Sku = snapshot.Sku,
    Description = snapshot.Description,
    Type = itemType,
    Active = snapshot.IsActive,
    UnitPrice = snapshot.UnitPrice,
    PurchaseCost = snapshot.PurchaseCost,
    TrackQtyOnHand = snapshot.TrackQuantityOnHand ?? false,
    QtyOnHand = snapshot.QuantityOnHand ?? 0,
    IncomeAccountRef = incomeAccountRef != null
        ? new { value = incomeAccountRef } : null,
    ExpenseAccountRef = expenseAccountRef != null
        ? new { value = expenseAccountRef } : null,
    AssetAccountRef = assetAccountRef != null
        ? new { value = assetAccountRef } : null
};

if (mapping != null && action == SyncAction.Update)
{
    var current = await _qbClient.GetAsync<dynamic>(
        entry.OrgId, $"item/{mapping.ExternalEntityId}");
```

```
    await _qbClient.PostAsync(entry.OrgId, "item", new
    {
        Id = mapping.ExternalEntityId,
        SyncToken = current?.SyncToken,
        sparse = true,
        qbItem.Name,
        qbItem.Sku,
        qbItem.Description,
        qbItem.Active,
        qbItem.UnitPrice,
        qbItem.PurchaseCost,
        qbItem.IncomeAccountRef,
        qbItem.ExpenseAccountRef
    });

    return new AdapterResult
    {
        Success = true,
        ExternalId = mapping.ExternalEntityId
    };
}
else
{
    var result = await _qbClient.PostAsync<dynamic>(
        entry.OrgId, "item", qbItem);

    string qbId = result.Item.Id.ToString();

    _financeDb.ExternalEntityMappings.Add(new ExternalEntityMapping
    {
        OrgId = entry.OrgId,
        EntityType = "Product",
        InternalEntityId = entry.EntityId,
        Provider = ExternalSystemType.QuickBooksOnline,
        ExternalEntityId = qbId,
        ExternalEntityName = snapshot.DisplayName,
        LastSyncedAtUtc = DateTime.UtcNow,
        SyncStatus = "Synced"
    });
```

```
            await _financeDb.SaveChangesAsync();

            return new AdapterResult { Success = true, ExternalId = qbId };
        }
    }

    // ── Account Sync ──
    private async Task<AdapterResult> SyncAccountAsync(
        EntitySyncOutbox entry, EntitySnapshot snapshot, SyncAction action)
    {
        var mapping = await _financeDb.ExternalAccountMappings
            .FirstOrDefaultAsync(m => m.OrgId == entry.OrgId
                                   && m.SmsAccountCode == snapshot.AccountCode
                                   && m.Provider ==
ExternalSystemType.QuickBooksOnline);

        var qbAccount = new
        {
            Name = snapshot.DisplayName,
            AccountType = snapshot.AccountType,
            AccountSubType = snapshot.AccountSubType,
            Active = snapshot.IsActive,
            AcctNum = snapshot.AccountCode
        };

        if (mapping != null && action == SyncAction.Update)
        {
            var current = await _qbClient.GetAsync<dynamic>(
                entry.OrgId, $"account/{mapping.ExternalAccountId}");

            await _qbClient.PostAsync(entry.OrgId, "account", new
            {
                Id = mapping.ExternalAccountId,
                SyncToken = current?.SyncToken,
                sparse = true,
                qbAccount.Name,
                qbAccount.AccountType,
                qbAccount.AccountSubType,
                qbAccount.Active,
                qbAccount.AcctNum
```

```
            });

            return new AdapterResult
            {
                Success = true,
                ExternalId = mapping.ExternalAccountId
            };
        }
        else
        {
            var result = await _qbClient.PostAsync<dynamic>(
                entry.OrgId, "account", qbAccount);

            string qbId = result.Account.Id.ToString();

            // Account mapping goes to ExternalAccountMappings (not
ExternalEntityMappings)
            _financeDb.ExternalAccountMappings.Add(new ExternalAccountMapping
            {
                OrgId = entry.OrgId,
                SmsAccountCode = snapshot.AccountCode!,
                Provider = ExternalSystemType.QuickBooksOnline,
                ExternalAccountId = qbId,
                ExternalAccountName = snapshot.DisplayName,
                IsAutoMapped = false
            });
            await _financeDb.SaveChangesAsync();

            return new AdapterResult { Success = true, ExternalId = qbId };
        }
    }

    // ── Initial Import ──
    public async Task<ImportResult> ImportEntitiesAsync(int orgId,
SyncEntityType entityType)
    {
        string qbEntityName = entityType switch
        {
            SyncEntityType.Customer => "customer",
            SyncEntityType.Vendor   => "vendor",
```

```
            SyncEntityType.Product  => "item",
            SyncEntityType.Account  => "account",
            _ => throw new ArgumentException($"Unsupported entity type:
{entityType}")
        };

        // QB query — get all entities (paginated)
        var allEntities = new List<dynamic>();
        int startPosition = 1;
        const int pageSize = 100;
        bool hasMore = true;

        while (hasMore)
        {
            var query = $"SELECT * FROM {qbEntityName} STARTPOSITION
{startPosition} MAXRESULTS {pageSize}";
            var response = await _qbClient.QueryAsync<dynamic>(orgId, query);

            var items = response?.QueryResponse?[char.ToUpper(qbEntityName[0])
+ qbEntityName[1..]]
                        as IEnumerable<dynamic>;

            if (items == null || !items.Any())
            {
                hasMore = false;
            }
            else
            {
                allEntities.AddRange(items);
                startPosition += pageSize;
                hasMore = items.Count() == pageSize;
            }
        }

        int created = 0;
        int alreadyExist = 0;

        foreach (var entity in allEntities)
        {
            string externalId = entity.Id.ToString();
```

```
            string displayName = entity.DisplayName?.ToString()
                              ?? entity.Name?.ToString()
                              ?? "Unknown";

            if (entityType == SyncEntityType.Account)
            {
                // Account imports go to ExternalAccountMappings
                bool exists = await _financeDb.ExternalAccountMappings
                    .AnyAsync(m => m.OrgId == orgId
                                && m.ExternalAccountId == externalId
                                && m.Provider ==
ExternalSystemType.QuickBooksOnline);
                if (exists) { alreadyExist++; continue; }

                _financeDb.ExternalAccountMappings.Add(new
ExternalAccountMapping
                {
                    OrgId = orgId,
                    SmsAccountCode = entity.AcctNum?.ToString() ?? "",
                    Provider = ExternalSystemType.QuickBooksOnline,
                    ExternalAccountId = externalId,
                    ExternalAccountName = displayName,
                    IsAutoMapped = true
                });
                created++;
            }
            else
            {
                // Entity imports go to ExternalEntityMappings
                bool exists = await _financeDb.ExternalEntityMappings
                    .AnyAsync(m => m.OrgId == orgId
                                && m.EntityType == entityType.ToString()
                                && m.ExternalEntityId == externalId
                                && m.Provider ==
ExternalSystemType.QuickBooksOnline);
                if (exists) { alreadyExist++; continue; }

                _financeDb.ExternalEntityMappings.Add(new ExternalEntityMapping
                {
                    OrgId = orgId,
```

```
                    EntityType = entityType.ToString(),
                    InternalEntityId = 0,  // Unlinked until user maps
                    Provider = ExternalSystemType.QuickBooksOnline,
                    ExternalEntityId = externalId,
                    ExternalEntityName = displayName,
                    LastSyncedAtUtc = DateTime.UtcNow,
                    SyncStatus = "ImportedUnlinked"
                });
                created++;
            }
        }

        await _financeDb.SaveChangesAsync();

        _logger.LogInformation(
            "Import from QB: {Type} — {Found} found, {Created} mappings
created, {Exist} already exist",
            entityType, allEntities.Count, created, alreadyExist);

        return new ImportResult
        {
            Success = true,
            EntitiesFound = allEntities.Count,
            MappingsCreated = created,
            MappingsAlreadyExist = alreadyExist
        };
    }
}
```

17.3 QB Entity Mapping Reference

SMS Entity	QB API Endpoint	QB Entity	Key Fields Mapped
`Customer`	`POST /v3/company/{realmId}/customer`	Customer	DisplayName,
CompanyName, GivenName, FamilyName, PrimaryEmailAddr, PrimaryPhone, BillAddr,
ShipAddr, CurrencyRef
`Vendor`	`POST /v3/company/{realmId}/vendor`	Vendor	DisplayName,
CompanyName, GivenName, FamilyName, PrimaryEmailAddr, PrimaryPhone, BillAddr,
TaxIdentifier, CurrencyRef

| `Product` | `POST /v3/company/{realmId}/item` | Item | Name, Sku, Description, Type (Inventory/NonInventory), UnitPrice, PurchaseCost, TrackQtyOnHand, IncomeAccountRef, ExpenseAccountRef, AssetAccountRef |
| `Account` | `POST /v3/company/{realmId}/account` | Account | Name, AccountType, AccountSubType, AcctNum, Active |

17.4 Import Flow

```
INITIAL IMPORT — QB entities into mapping table
──────────────────────────────────────────────

User clicks [Import from QuickBooks] on Entity Mapping Panel

    1. API call: GET /api/finance/entity-sync/{provider}/import?entityType=Customer
    2. Gateway calls adapter.ImportEntitiesAsync()
    3. Adapter queries QB: SELECT * FROM Customer (paginated, 100 per page)
    4. For each QB Customer:
       a. Check if ExternalEntityMappings already has this externalId
       b. If not → create mapping with InternalEntityId = 0 (unlinked)
    5. Return count: { found: 250, created: 230, alreadyExist: 20 }
    6. UI shows mapping panel — user links unlinked QB entities to SMS entities

ONGOING SYNC — SMS entity changes pushed to QB
──────────────────────────────────────────────

CustomerService.CreateAsync() saves customer

    1. Build EntitySnapshot from the new customer
    2. Call _entitySync.PushAsync(new EntitySyncRequest { ... })
    3. Gateway writes EntitySyncOutbox entry (PayloadJson = serialized snapshot)
    4. SaveChangesAsync() commits customer + outbox row atomically
    5. Hangfire picks up outbox entry
    6. Adapter reads snapshot, builds QB Customer payload, calls QB API
    7. On success: creates ExternalEntityMappings entry (SMS Id ↔ QB Id)
    8. Logs to SyncLogs
```

17.5 Design Rules for QB Master Data Adapter

Rule	Detail
Reuses C8 API client	Same `IQuickBooksApiClient` with token refresh, rate limiting, and error handling.
Sparse update	QB updates use `sparse = true` so unchanged fields aren't overwritten. Requires fetching current `SyncToken` first.
Account mapping for Products	When syncing a Product with `IncomeAccountCode`, the adapter resolves the QB Account ID from `ExternalAccountMappings`. If unmapped, the outbox entry fails (retryable after admin maps the account).
Import creates unlinked mappings	`InternalEntityId = 0` means the QB entity exists in the mapping table but hasn't been linked to an SMS entity yet. User completes linking from the UI.
No SMS entity creation on import	Import NEVER creates Customers, Vendors, or Products in SMS. It only creates mapping entries. Entity creation stays in the core modules.

---

18. C14 — Entity Sync Outbox & Background Jobs

Location: `SMS.Finance/Entities/EntitySyncOutbox.cs`, `SMS.Finance/Jobs/EntitySyncOutboxProcessor.cs`

18.1 Entity: `EntitySyncOutbox`

```csharp
namespace SMS.Finance.Entities;

/// <summary>
/// Outbox table for entity sync operations. Same pattern as FinancialPostingOutbox
/// from C7, but for master data (Customers, Vendors, Products, Accounts).
/// Hangfire processes pending entries asynchronously.
/// </summary>
[Table("EntitySyncOutbox", Schema = "finance")]
public class EntitySyncOutbox
{
```

```
    [Key]
    public long Id { get; set; }

    public int OrgId { get; set; }

    /// <summary>SyncEntityType as string: Customer, Vendor, Product, Account.
</summary>
    [MaxLength(50)]
    public string EntityType { get; set; } = null!;

    /// <summary>SMS primary key of the entity.</summary>
    public int EntityId { get; set; }

    /// <summary>SyncAction as string: Create, Update, Deactivate.</summary>
    [MaxLength(20)]
    public string Action { get; set; } = null!;

    /// <summary>ExternalSystemType as string: QuickBooksOnline, Xero, etc.
</summary>
    [MaxLength(50)]
    public string Provider { get; set; } = null!;

    /// <summary>FK to ExternalSystemConnections (which connection to use).
</summary>
    public int ConnectionId { get; set; }

    /// <summary>
    /// Serialized EntitySnapshot — the entity data at the time of the request.
    /// The processor reads this, never queries ApplicationDbContext.
    /// </summary>
    public string PayloadJson { get; set; } = null!;

    /// <summary>
    /// Unique key: "{entityType}:{action}:{entityId}:{timestamp}".
    /// Prevents duplicate outbox entries.
    /// </summary>
    [MaxLength(200)]
    public string IdempotencyKey { get; set; } = null!;

    /// <summary>Pending → Processing → Completed | Failed → DeadLetter.
```

```
</summary>
    public OutboxStatus Status { get; set; }

    public DateTime CreatedAtUtc { get; set; }
    public DateTime? ProcessedAtUtc { get; set; }
    public DateTime? NextRetryAtUtc { get; set; }

    public int RetryCount { get; set; }
    public int MaxRetries { get; set; } = 5;

    /// <summary>External system's ID for the entity after successful sync.
</summary>
    [MaxLength(100)]
    public string? ExternalId { get; set; }

    /// <summary>Last error message if status is Failed or DeadLetter.
</summary>
    [MaxLength(2000)]
    public string? ErrorMessage { get; set; }

    /// <summary>Who triggered: "system", user email, or "import".</summary>
    [MaxLength(100)]
    public string InitiatedBy { get; set; } = "system";
}
```

18.2 EF Core Configuration

```csharp
// In FinanceDbContext.OnModelCreating()
modelBuilder.Entity<EntitySyncOutbox>(entity =>
{
    entity.ToTable("EntitySyncOutbox", "finance");

    entity.HasIndex(e => new { e.OrgId, e.Status, e.NextRetryAtUtc })
          .HasDatabaseName("IX_EntitySyncOutbox_OrgId_Status_NextRetry");

    entity.HasIndex(e => new { e.OrgId, e.IdempotencyKey })
          .IsUnique()
          .HasDatabaseName("IX_EntitySyncOutbox_OrgId_IdempotencyKey");
```

```
    entity.HasIndex(e => new { e.OrgId, e.EntityType, e.EntityId })
          .HasDatabaseName("IX_EntitySyncOutbox_OrgId_EntityType_EntityId");

    entity.Property(e => e.PayloadJson).HasColumnType("nvarchar(max)");
});
```

18.3 Hangfire Background Processor

```csharp
namespace SMS.Finance.Jobs;

/// <summary>
/// Processes pending EntitySyncOutbox entries. Mirrors OutboxProcessor from
C7.
/// Runs on a recurring schedule (every 30 seconds).
///
/// Processing loop:
///   1. Read batch of Pending entries where NextRetryAtUtc <= now
///   2. For each entry: resolve adapter → call PushEntityAsync()
///   3. On success: mark Completed, record ExternalId, log to SyncLogs
///   4. On failure: increment RetryCount, schedule next retry (exponential
backoff)
///   5. After max retries: move to DeadLetter
/// </summary>
public class EntitySyncOutboxProcessor
{
    private readonly FinanceDbContext _financeDb;
    private readonly IEnumerable<IEntitySyncAdapter> _adapters;
    private readonly ILogger<EntitySyncOutboxProcessor> _logger;
    private const int BatchSize = 20;

    public EntitySyncOutboxProcessor(
        FinanceDbContext financeDb,
        IEnumerable<IEntitySyncAdapter> adapters,
        ILogger<EntitySyncOutboxProcessor> logger)
    {
        _financeDb = financeDb;
        _adapters = adapters;
```

```
        _logger = logger;
    }

    [AutomaticRetry(Attempts = 0)]  // We handle retries ourselves
    public async Task ProcessPendingAsync()
    {
        var entries = await _financeDb.EntitySyncOutbox
            .Where(e => e.Status == OutboxStatus.Pending
                    && (e.NextRetryAtUtc == null || e.NextRetryAtUtc <=
DateTime.UtcNow))
            .OrderBy(e => e.CreatedAtUtc)
            .Take(BatchSize)
            .ToListAsync();

        foreach (var entry in entries)
        {
            await ProcessEntryAsync(entry);
        }
    }

    private async Task ProcessEntryAsync(EntitySyncOutbox entry)
    {
        var provider = Enum.Parse<ExternalSystemType>(entry.Provider);
        var adapter = _adapters.FirstOrDefault(a => a.Provider == provider);

        if (adapter == null)
        {
            _logger.LogError("No adapter for provider {Provider}",
entry.Provider);
            entry.Status = OutboxStatus.DeadLetter;
            entry.ErrorMessage = $"No adapter registered for {entry.Provider}";
            await _financeDb.SaveChangesAsync();
            return;
        }

        entry.Status = OutboxStatus.Processing;
        await _financeDb.SaveChangesAsync();

        try
        {
```

```
            var result = await adapter.PushEntityAsync(entry);

            if (result.Success)
            {
                entry.Status = OutboxStatus.Completed;
                entry.ExternalId = result.ExternalId;
                entry.ProcessedAtUtc = DateTime.UtcNow;
                entry.ErrorMessage = null;

                // Log to SyncLogs (reuse existing table from C9)
                _financeDb.SyncLogs.Add(new SyncLog
                {
                    OrgId = entry.OrgId,
                    Provider = entry.Provider,
                    DocumentType = entry.EntityType,
                    SmsDocumentId = entry.EntityId,
                    ExternalId = result.ExternalId,
                    Direction = "Outbound",
                    Status = "Synced",
                    SyncedAtUtc = DateTime.UtcNow,
                    Details = $"{entry.Action} {entry.EntityType} #
{entry.EntityId}"
                });
            }
            else
            {
                HandleFailure(entry, result.ErrorMessage, result.Retryable);
            }
        }
        catch (QuickBooksApiException ex) when (ex.StatusCode == 401)
        {
            // Token expired — will be refreshed on next attempt
            HandleFailure(entry, "OAuth token expired — will retry after
refresh", true);
        }
        catch (Exception ex)
        {
            HandleFailure(entry, ex.Message, true);
        }
```

```
        await _financeDb.SaveChangesAsync();
    }

    private void HandleFailure(EntitySyncOutbox entry, string? errorMessage,
bool retryable)
    {
        entry.RetryCount++;
        entry.ErrorMessage = errorMessage;

        if (!retryable || entry.RetryCount >= entry.MaxRetries)
        {
            entry.Status = OutboxStatus.DeadLetter;
            _logger.LogWarning(
                "Entity sync dead-lettered: {EntityType} #{EntityId} to
{Provider} — {Error}",
                entry.EntityType, entry.EntityId, entry.Provider,
errorMessage);

            _financeDb.SyncLogs.Add(new SyncLog
            {
                OrgId = entry.OrgId,
                Provider = entry.Provider,
                DocumentType = entry.EntityType,
                SmsDocumentId = entry.EntityId,
                Direction = "Outbound",
                Status = "DeadLetter",
                SyncedAtUtc = DateTime.UtcNow,
                Details = $"Dead letter after {entry.RetryCount} attempts:
{errorMessage}"
            });
        }
        else
        {
            entry.Status = OutboxStatus.Pending;
            // Exponential backoff: 30s, 2m, 8m, 32m, 2h
            var delaySeconds = 30 * Math.Pow(4, entry.RetryCount - 1);
            entry.NextRetryAtUtc = DateTime.UtcNow.AddSeconds(delaySeconds);

            _logger.LogWarning(
                "Entity sync retry {Count}/{Max}: {EntityType} #{EntityId} —
```

```
next at {Next}",
                entry.RetryCount, entry.MaxRetries,
                entry.EntityType, entry.EntityId, entry.NextRetryAtUtc);
        }
    }
}
```

18.4 Hangfire Registration

```csharp
// In SMS.Finance/DependencyInjection.cs or Startup
RecurringJob.AddOrUpdate<EntitySyncOutboxProcessor>(
    "entity-sync-outbox-processor",
    processor => processor.ProcessPendingAsync(),
    "*/30 * * * * *");  // Every 30 seconds
```

18.5 Core Module Caller Pattern

This shows how existing services call the entity sync gateway. Same pattern across all modules:

```csharp
// In SMS.Sales/Services/CustomerService.cs
public class CustomerService
{
    private readonly ApplicationDbContext _db;
    private readonly IEntitySyncGateway _entitySync;   // ← injected

    public async Task<Customer> CreateAsync(CreateCustomerDto dto)
    {
        var customer = new Customer
        {
            OrgId = _currentOrg.Id,
            DisplayName = dto.DisplayName,
            CompanyName = dto.CompanyName,
            FirstName = dto.FirstName,
            LastName = dto.LastName,
            Email = dto.Email,
```

```
            Phone = dto.Phone,
            // ... other fields
        };

        _db.Customers.Add(customer);

        // ── Entity Sync — one call, no QB knowledge ──
        await _entitySync.PushAsync(new EntitySyncRequest
        {
            EntityType = SyncEntityType.Customer,
            Action = SyncAction.Create,
            EntityId = customer.Id,
            OrgId = customer.OrgId,
            IdempotencyKey = $"Customer:Create:{customer.Id}:
{DateTime.UtcNow.Ticks}",
            Snapshot = new EntitySnapshot
            {
                DisplayName = customer.DisplayName,
                CompanyName = customer.CompanyName,
                FirstName = customer.FirstName,
                LastName = customer.LastName,
                Email = customer.Email,
                Phone = customer.Phone,
                BillingAddressLine1 = customer.BillingAddress?.Line1,
                BillingAddressCity = customer.BillingAddress?.City,
                BillingAddressState = customer.BillingAddress?.State,
                BillingAddressPostalCode = customer.BillingAddress?.PostalCode,
                BillingAddressCountry = customer.BillingAddress?.Country,
                IsActive = true,
                CurrencyCode = customer.CurrencyCode
            }
        });

        await _db.SaveChangesAsync();  // Commits customer + outbox row
atomically
        return customer;
    }

    public async Task<Customer> UpdateAsync(int id, UpdateCustomerDto dto)
    {
```

```
        var customer = await _db.Customers.FindAsync(id);
        // ... apply updates ...

        await _entitySync.PushAsync(new EntitySyncRequest
        {
            EntityType = SyncEntityType.Customer,
            Action = SyncAction.Update,
            EntityId = customer.Id,
            OrgId = customer.OrgId,
            IdempotencyKey = $"Customer:Update:{customer.Id}:
{DateTime.UtcNow.Ticks}",
            Snapshot = new EntitySnapshot { /* ... current customer fields ...
*/ }
        });

        await _db.SaveChangesAsync();
        return customer;
    }
}
```

```csharp
// In SMS.Purchase/Services/VendorService.cs — same pattern
public class VendorService
{
    private readonly IEntitySyncGateway _entitySync;

    public async Task<Vendor> CreateAsync(CreateVendorDto dto)
    {
        var vendor = new Vendor { /* ... */ };
        _db.Vendors.Add(vendor);

        await _entitySync.PushAsync(new EntitySyncRequest
        {
            EntityType = SyncEntityType.Vendor,
            Action = SyncAction.Create,
            EntityId = vendor.Id,
            OrgId = vendor.OrgId,
            IdempotencyKey = $"Vendor:Create:{vendor.Id}:
{DateTime.UtcNow.Ticks}",
```

```
            Snapshot = new EntitySnapshot
            {
                DisplayName = vendor.DisplayName,
                CompanyName = vendor.CompanyName,
                // ... vendor fields
            }
        });

        await _db.SaveChangesAsync();
        return vendor;
    }
}
```

```csharp
// In SMS.Inventory/Services/ProductService.cs — same pattern
public class ProductService
{
    private readonly IEntitySyncGateway _entitySync;

    public async Task<Product> CreateAsync(CreateProductDto dto)
    {
        var product = new Product { /* ... */ };
        _db.Products.Add(product);

        await _entitySync.PushAsync(new EntitySyncRequest
        {
            EntityType = SyncEntityType.Product,
            Action = SyncAction.Create,
            EntityId = product.Id,
            OrgId = product.OrgId,
            IdempotencyKey = $"Product:Create:{product.Id}:
{DateTime.UtcNow.Ticks}",
            Snapshot = new EntitySnapshot
            {
                DisplayName = product.Name,
                Sku = product.Sku,
                Description = product.Description,
                UnitPrice = product.SellingPrice,
                PurchaseCost = product.CostPrice,
```

```
                IncomeAccountCode = product.IncomeAccountCode,
                ExpenseAccountCode = product.ExpenseAccountCode,
                AssetAccountCode = product.AssetAccountCode,
                TrackQuantityOnHand = product.IsTracked,
                QuantityOnHand = product.CurrentStock,
                UnitOfMeasure = product.UomCode,
                IsActive = product.IsActive
            }
        });

        await _db.SaveChangesAsync();
        return product;
    }
}
```

18.6 Design Rules for Entity Sync Outbox

Rule	Detail
Same table structure as FinancialPostingOutbox	Status enum, retry count, exponential backoff, dead-letter — identical pattern from C7.
Separate table, not shared	Entity sync and financial posting have different payloads, different processing cadence, and different adapters. Separate outbox tables keep them independent.
Snapshot in PayloadJson	The processor reads entity data from the outbox entry's JSON, never from `ApplicationDbContext`. This makes entity sync separation-ready.
SyncLogs reused	Entity sync writes to the same `finance.SyncLogs` table from C9. The `DocumentType` column distinguishes "Customer" from "SalesInvoice".
Exponential backoff	Same schedule as financial outbox: 30s → 2m → 8m → 32m → 2h. After 5 attempts → DeadLetter.
Hangfire every 30s	Separate recurring job from the financial outbox processor — they run independently.

---

19. C15 — Tax Code Mapping

Location: `SMS.Finance/Entities/ExternalTaxMapping.cs`,
`SMS.Finance/Services/TaxMappingService.cs`

19.1 Entity: `ExternalTaxMapping`

```csharp
namespace SMS.Finance.Entities;

/// <summary>
/// Maps SMS tax codes to external system tax identifiers.
/// QuickBooks requires TaxCode references on invoice lines.
/// Without this mapping, invoices with tax will fail to sync.
///
/// Reuses the same per-tenant, per-provider pattern as
ExternalAccountMappings.
/// </summary>
[Table("ExternalTaxMappings", Schema = "finance")]
public class ExternalTaxMapping
{
    [Key]
    public int Id { get; set; }

    public int OrgId { get; set; }

    /// <summary>SMS tax code identifier (e.g., "VAT-17", "GST-5", "TAX-
EXEMPT").</summary>
    [MaxLength(50)]
    public string SmsTaxCode { get; set; } = null!;

    /// <summary>SMS tax rate percentage for display (e.g., 17.00, 5.00, 0.00).
</summary>
    public decimal SmsTaxRate { get; set; }

    /// <summary>Target external system.</summary>
    public ExternalSystemType Provider { get; set; }

    /// <summary>External system's tax code ID (e.g., QB TaxCode.Id = "2").
</summary>
    [MaxLength(100)]
    public string ExternalTaxCodeId { get; set; } = null!;
```

```
    /// <summary>External system's tax code name for display (e.g., "GST on
Sales").</summary>
    [MaxLength(200)]
    public string ExternalTaxCodeName { get; set; } = null!;

    /// <summary>Whether this mapping was auto-created during import.</summary>
    public bool IsAutoMapped { get; set; }

    /// <summary>Soft-delete: inactive mappings are skipped during enrichment.
</summary>
    public bool IsActive { get; set; } = true;
}
```

19.2 EF Core Configuration

```csharp
// In FinanceDbContext.OnModelCreating()
modelBuilder.Entity<ExternalTaxMapping>(entity =>
{
    entity.ToTable("ExternalTaxMappings", "finance");

    entity.HasIndex(e => new { e.OrgId, e.SmsTaxCode, e.Provider })
          .IsUnique()
          .HasDatabaseName("IX_ExternalTaxMappings_OrgId_SmsTaxCode_Provider");

    entity.HasIndex(e => new { e.OrgId, e.Provider })
          .HasDatabaseName("IX_ExternalTaxMappings_OrgId_Provider");
});
```

19.3 Tax Mapping Service

```csharp
namespace SMS.Finance.Services;

public interface ITaxMappingService
{
    /// <summary>Resolve SMS tax code to external provider tax ID.</summary>
```

```
    Task<string?> ResolveExternalTaxCodeAsync(
        int orgId, string smsTaxCode, ExternalSystemType provider);

    /// <summary>Import tax codes from provider and auto-map by rate matching.
</summary>
    Task<TaxImportResult> ImportTaxCodesFromProviderAsync(
        int orgId, ExternalSystemType provider);
}

public class TaxMappingService : ITaxMappingService
{
    private readonly FinanceDbContext _financeDb;
    private readonly IQuickBooksApiClient _qbClient;

    public TaxMappingService(
        FinanceDbContext financeDb,
        IQuickBooksApiClient qbClient)
    {
        _financeDb = financeDb;
        _qbClient = qbClient;
    }

    public async Task<string?> ResolveExternalTaxCodeAsync(
        int orgId, string smsTaxCode, ExternalSystemType provider)
    {
        var mapping = await _financeDb.ExternalTaxMappings
            .FirstOrDefaultAsync(m => m.OrgId == orgId
                                   && m.SmsTaxCode == smsTaxCode
                                   && m.Provider == provider
                                   && m.IsActive);
        return mapping?.ExternalTaxCodeId;
    }

    public async Task<TaxImportResult> ImportTaxCodesFromProviderAsync(
        int orgId, ExternalSystemType provider)
    {
        // QB: query TaxCode entities
        var qbTaxCodes = await _qbClient.QueryAsync<dynamic>(
            orgId, "SELECT * FROM TaxCode");
```

```
        int created = 0, alreadyExist = 0;

        foreach (var taxCode in qbTaxCodes?.QueryResponse?.TaxCode ??
Array.Empty<dynamic>())
        {
            string externalId = taxCode.Id.ToString();
            string name = taxCode.Name?.ToString() ?? "Unknown";

            bool exists = await _financeDb.ExternalTaxMappings
                .AnyAsync(m => m.OrgId == orgId
                            && m.ExternalTaxCodeId == externalId
                            && m.Provider == provider);

            if (exists) { alreadyExist++; continue; }

            _financeDb.ExternalTaxMappings.Add(new ExternalTaxMapping
            {
                OrgId = orgId,
                SmsTaxCode = "",   // Unlinked — user maps in UI
                SmsTaxRate = 0,
                Provider = provider,
                ExternalTaxCodeId = externalId,
                ExternalTaxCodeName = name,
                IsAutoMapped = true
            });
            created++;
        }

        await _financeDb.SaveChangesAsync();
        return new TaxImportResult
        {
            TaxCodesFound = created + alreadyExist,
            MappingsCreated = created,
            AlreadyExist = alreadyExist
        };
    }
}

public class TaxImportResult
{
```

```
    public int TaxCodesFound { get; init; }
    public int MappingsCreated { get; init; }
    public int AlreadyExist { get; init; }
}
```

19.4 Integration with Provider Enrichment (C6)

The `ProviderEnrichmentService` from C6 is extended to resolve tax codes during financial transaction enrichment:

```csharp
// Addition to existing ProviderEnrichmentService from C6
public class ProviderEnrichmentService
{
    private readonly ITaxMappingService _taxMapping;
    // ... existing dependencies ...

    public async Task<EnrichedFinancialPayload> EnrichAsync(
        FinancialTransaction transaction,
        ExternalSystemType provider)
    {
        var payload = new EnrichedFinancialPayload { /* ... existing enrichment ... */ };

        // ── Tax Code Resolution (NEW) ──
        foreach (var line in payload.Lines)
        {
            if (!string.IsNullOrEmpty(line.TaxCode))
            {
                line.ExternalTaxCodeId = await _taxMapping
                    .ResolveExternalTaxCodeAsync(
                        transaction.OrgId, line.TaxCode, provider);
            }
        }

        return payload;
    }
}
```

19.5 Design Rules for Tax Mapping

Rule	Detail
Same table pattern as account/entity mappings	Per tenant, per provider, with import + manual link.
Enrichment integration	Tax codes are resolved during the enrichment step (C6), before the QB adapter builds the API payload.
Tax-exempt handling	SMS tax code "TAX-EXEMPT" or rate 0% maps to QB "NON" (non-taxable). This is seeded as a default mapping.
QB TaxCode vs TaxRate	QB uses TaxCode (the classification) on line items, not the rate directly. The mapping resolves SMS tax code → QB TaxCode.Id.
Missing mapping = outbox failure	If a tax code is unmapped, the outbox entry fails with a clear error. The admin maps it, then retries.

---

20. C16 — Pre-Sync Validation & Reference Checks

Location: `SMS.Finance/Services/PreSyncValidator.cs`

20.1 Validation Service

```csharp
namespace SMS.Finance.Services;

/// <summary>
/// Validates that all external references required for a financial posting
/// or entity sync exist before calling the external API.
///
/// Runs inside the outbox processor AFTER reading the entry,
/// BEFORE calling the provider adapter. If validation fails,
/// the entry is marked Failed with a clear error listing every
/// missing reference — so the admin can fix all mappings at once
/// rather than discovering them one at a time.
/// </summary>
public interface IPreSyncValidator
{
```

```
    Task<ValidationResult> ValidateFinancialPostingAsync(
        FinancialPostingOutbox entry, ExternalSystemType provider);

    Task<ValidationResult> ValidateEntitySyncAsync(
        EntitySyncOutbox entry, ExternalSystemType provider);
}

public class PreSyncValidator : IPreSyncValidator
{
    private readonly FinanceDbContext _financeDb;

    public PreSyncValidator(FinanceDbContext financeDb)
    {
        _financeDb = financeDb;
    }

    public async Task<ValidationResult> ValidateFinancialPostingAsync(
        FinancialPostingOutbox entry, ExternalSystemType provider)
    {
        var errors = new List<string>();
        var transaction = JsonSerializer
            .Deserialize<FinancialTransaction>(entry.PayloadJson)!;

        // ── 1. Account Mapping Checks ──
        var accountCodes = transaction.Lines
            .Select(l => l.AccountCode)
            .Distinct()
            .ToList();

        foreach (var code in accountCodes)
        {
            var mapped = await _financeDb.ExternalAccountMappings
                .AnyAsync(m => m.OrgId == entry.OrgId
                            && m.SmsAccountCode == code
                            && m.Provider == provider);
            if (!mapped)
                errors.Add($"Account '{code}' not mapped to {provider}");
        }

        // ── 2. Entity Mapping Check (Customer/Vendor) ──
```

```
        if (transaction.PartnerId > 0)
        {
            string entityType = transaction.PartnerType switch
            {
                "Customer" => "Customer",
                "Vendor"   => "Vendor",
                _          => transaction.PartnerType ?? "Customer"
            };

            var entityMapped = await _financeDb.ExternalEntityMappings
                .AnyAsync(m => m.OrgId == entry.OrgId
                            && m.EntityType == entityType
                            && m.InternalEntityId == transaction.PartnerId
                            && m.Provider == provider);
            if (!entityMapped)
                errors.Add($"{entityType} #{transaction.PartnerId} not mapped
to {provider}");
        }

        // ── 3. Tax Code Mapping Check ──
        var taxCodes = transaction.Lines
            .Where(l => !string.IsNullOrEmpty(l.TaxCode))
            .Select(l => l.TaxCode!)
            .Distinct()
            .ToList();

        foreach (var taxCode in taxCodes)
        {
            var taxMapped = await _financeDb.ExternalTaxMappings
                .AnyAsync(m => m.OrgId == entry.OrgId
                            && m.SmsTaxCode == taxCode
                            && m.Provider == provider
                            && m.IsActive);
            if (!taxMapped)
                errors.Add($"Tax code '{taxCode}' not mapped to {provider}");
        }

        // ── 4. Currency Validation ──
        if (!string.IsNullOrEmpty(transaction.CurrencyCode))
        {
```

```
            // QB validates currency at API level — log warning if unusual
            var commonCurrencies = new[] { "USD", "GBP", "EUR", "CAD", "AUD",
"PKR", "AED", "SAR" };
            if (!commonCurrencies.Contains(transaction.CurrencyCode))
            {
                // Warning only, not blocking
            }
        }

        return new ValidationResult
        {
            IsValid = errors.Count == 0,
            Errors = errors,
            CheckedAt = DateTime.UtcNow
        };
    }

    public async Task<ValidationResult> ValidateEntitySyncAsync(
        EntitySyncOutbox entry, ExternalSystemType provider)
    {
        var errors = new List<string>();
        var snapshot = JsonSerializer.Deserialize<EntitySnapshot>
(entry.PayloadJson)!;
        var entityType = Enum.Parse<SyncEntityType>(entry.EntityType);

        // ── Product: validate account references ──
        if (entityType == SyncEntityType.Product)
        {
            if (!string.IsNullOrEmpty(snapshot.IncomeAccountCode))
            {
                var mapped = await _financeDb.ExternalAccountMappings
                    .AnyAsync(m => m.OrgId == entry.OrgId
                                && m.SmsAccountCode ==
snapshot.IncomeAccountCode
                                && m.Provider == provider);
                if (!mapped)
                    errors.Add($"Income account '{snapshot.IncomeAccountCode}'
not mapped to {provider}");
            }
```

```
            if (!string.IsNullOrEmpty(snapshot.ExpenseAccountCode))
            {
                var mapped = await _financeDb.ExternalAccountMappings
                    .AnyAsync(m => m.OrgId == entry.OrgId
                                && m.SmsAccountCode ==
snapshot.ExpenseAccountCode
                                && m.Provider == provider);
                if (!mapped)
                    errors.Add($"Expense account
'{snapshot.ExpenseAccountCode}' not mapped to {provider}");
            }
        }

        return new ValidationResult
        {
            IsValid = errors.Count == 0,
            Errors = errors,
            CheckedAt = DateTime.UtcNow
        };
    }
}

public class ValidationResult
{
    public bool IsValid { get; init; }
    public List<string> Errors { get; init; } = new();
    public DateTime CheckedAt { get; init; }
}
```

20.2 Integration with Outbox Processors

Both `OutboxProcessor` (C7) and `EntitySyncOutboxProcessor` (C14) call the validator before the adapter:

```csharp
// Inside OutboxProcessor.ProcessEntryAsync() — added validation step
private async Task ProcessEntryAsync(FinancialPostingOutbox entry)
{
    var provider = Enum.Parse<ExternalSystemType>(entry.Provider);
```

```
    // ── NEW: Pre-sync validation ──
    var validation = await _validator.ValidateFinancialPostingAsync(entry,
provider);
    if (!validation.IsValid)
    {
        // All missing references reported at once — admin fixes in one pass
        entry.Status = OutboxStatus.Failed;
        entry.ErrorMessage = "Validation failed: " + string.Join("; ",
validation.Errors);
        entry.RetryCount++;
        // Schedule retry — admin may map the missing references
        var delaySeconds = 30 * Math.Pow(4, entry.RetryCount - 1);
        entry.NextRetryAtUtc = DateTime.UtcNow.AddSeconds(delaySeconds);
        await _financeDb.SaveChangesAsync();
        return;
    }

    // ... existing adapter call ...
}
```

20.3 Mapping Coverage API

```csharp
// GET /api/finance/mappings/coverage/{provider}
// Returns comprehensive coverage report across ALL mapping types

public class MappingCoverageReport
{
    public string Provider { get; init; } = null!;
    public CoverageDetail Accounts { get; init; } = null!;
    public CoverageDetail Customers { get; init; } = null!;
    public CoverageDetail Vendors { get; init; } = null!;
    public CoverageDetail Products { get; init; } = null!;
    public CoverageDetail TaxCodes { get; init; } = null!;
    public bool ReadyForSync { get; init; }   // true only when all > 0%
coverage
    public List<string> BlockingIssues { get; init; } = new();
}
```

```
public class CoverageDetail
{
    public int TotalInSms { get; init; }
    public int Mapped { get; init; }
    public int Unmapped { get; init; }
    public decimal CoveragePercent { get; init; }
}
```

20.4 Design Rules for Pre-Sync Validation

Rule	Detail
All-at-once error reporting	Validator reports ALL missing references in one pass. The admin sees "Account 4100, Customer #42, Tax code VAT-17 not mapped" — not three separate failures on three retries.
Runs in outbox processor, not gateway	PushAsync/PostAsync still writes the outbox entry. Validation runs when the background job picks it up. This keeps the gateway fast and allows time for mappings to be created between entity save and outbox processing.
Retryable failure	Validation failures schedule a retry. The admin creates the missing mapping, and the next retry passes validation.
Coverage API	The coverage endpoint shows readiness across all mapping types. The UI shows a "ready for sync" badge when all entity types have > 0% coverage.

---

21. C17 — Document Lifecycle Gateway (Void / Cancel / Reversal)

Location: `SMS.Core/Contracts/Finance/IFinancialPostingGateway.cs` (extension), `SMS.Finance/Services/`

21.1 Extended Gateway Interface

```csharp
namespace SMS.Core.Contracts.Finance;

// Extension to existing IFinancialPostingGateway from C1
```

```
public interface IFinancialPostingGateway
{
    // ... existing PostAsync() and ReverseAsync() from C1 ...

    /// <summary>
    /// Voids a previously synced document in external systems.
    /// For QuickBooks: calls the void endpoint for the specific document type.
    /// Internal GL: creates reversing journal entry.
    ///
    /// Used for: Invoice cancellation, payment void, bill void.
    /// </summary>
    Task<PostingResult> VoidAsync(VoidRequest request);

    /// <summary>
    /// Issues a credit memo / debit note against a previously synced document.
    /// For QuickBooks: creates CreditMemo (sales) or VendorCredit (purchase).
    /// Internal GL: creates credit/debit journal entries.
    ///
    /// Used for: Sales returns, purchase returns, partial refunds.
    /// </summary>
    Task<PostingResult> IssueCreditAsync(CreditRequest request);
}
```

21.2 Request DTOs

```csharp
namespace SMS.Core.Contracts.Finance;

public class VoidRequest
{
    public int OrgId { get; init; }
    public string SourceModule { get; init; } = null!;       // "Sales",
"Purchase"
    public FinDocType DocumentType { get; init; }
    public int SourceDocumentId { get; init; }
    public string Reason { get; init; } = null!;
    public string VoidedBy { get; init; } = "system";

    /// <summary>
```

```
    /// Idempotency key for the void operation.
    /// Format: "{module}:{docType}:{docId}:void"
    /// </summary>
    public string IdempotencyKey => $"{SourceModule}:{DocumentType}:
{SourceDocumentId}:void";
}

public class CreditRequest
{
    public int OrgId { get; init; }
    public string SourceModule { get; init; } = null!;
    public FinDocType OriginalDocumentType { get; init; }     // What's being
credited
    public int OriginalDocumentId { get; init; }
    public FinDocType CreditDocumentType { get; init; }       // CreditNote,
VendorCredit
    public int CreditDocumentId { get; init; }
    public decimal CreditAmount { get; init; }
    public List<FinancialTransactionLine> Lines { get; init; } = new();
    public string IssuedBy { get; init; } = "system";

    public string IdempotencyKey =>
        $"{SourceModule}:{CreditDocumentType}:{CreditDocumentId}";
}
```

21.3 QB Document Lifecycle Mapping

SMS Operation	Gateway Method	QB API Call	QB Entity
Cancel Sale Invoice	`VoidAsync()`	`POST /invoice?operation=void`	Invoice (voided)
Cancel Purchase Bill	`VoidAsync()`	`POST /bill?operation=void`	Bill (voided)
Void Customer Payment	`VoidAsync()`	`POST /payment?operation=void`	Payment (voided)
Void Supplier Payment	`VoidAsync()`	`POST /billpayment?operation=void`	BillPayment (voided)
Sales Return / Credit Note	`IssueCreditAsync()`	`POST /creditmemo`	CreditMemo (new)

Purchase Return	`IssueCreditAsync()`	`POST /vendorcredit`	VendorCredit (new)
Customer Refund	`IssueCreditAsync()`	`POST /refundreceipt`	RefundReceipt (new)
Supplier Invoice Reversal	`VoidAsync()` with flag	No QB call — flagged for accountant review	N/A

21.4 Supplier Invoice Reversal — Flag-Only Behavior

```csharp
// In gateway implementation
public async Task<PostingResult> VoidAsync(VoidRequest request)
{
    // ... license check, idempotency ...

    var targets = await _targetResolver.GetActiveTargetsAsync(request.OrgId);

    foreach (var target in targets)
    {
        // ── Supplier Invoice Reversal: flag-only, no QB transaction ──
        if (request.DocumentType == FinDocType.PurchaseBill
            && request.SourceModule == "Purchase")
        {
            // Write to SyncLogs as "FlaggedForReview" — no outbox entry
            _financeDb.SyncLogs.Add(new SyncLog
            {
                OrgId = request.OrgId,
                Provider = target.Provider.ToString(),
                DocumentType = "PurchaseBillReversal",
                SmsDocumentId = request.SourceDocumentId,
                Direction = "Outbound",
                Status = "FlaggedForReview",
                SyncedAtUtc = DateTime.UtcNow,
                Details = $"Supplier invoice reversal flagged for accountant: {request.Reason}"
            });
            continue;  // Do NOT create outbox entry for QB
        }

        // ── All other voids: write to outbox for async processing ──
```

```
        _financeDb.FinancialPostingOutbox.Add(new FinancialPostingOutbox
        {
            OrgId = request.OrgId,
            SourceModule = request.SourceModule,
            DocumentType = $"{request.DocumentType}:Void",
            SourceDocumentId = request.SourceDocumentId,
            Provider = target.Provider.ToString(),
            ConnectionId = target.ConnectionId,
            PayloadJson = JsonSerializer.Serialize(request),
            IdempotencyKey = request.IdempotencyKey,
            Status = OutboxStatus.Pending,
            CreatedAtUtc = DateTime.UtcNow
        });
    }

    // Internal GL: create reversing journal entry
    if (targets.Any(t => t.IsInternalGL))
    {
        await CreateReversingJournalEntryAsync(request);
    }

    return new PostingResult { Success = true, Outcome = PostingOutcome.Queued
};
}
```

21.5 QB Void Adapter Extension

```csharp
// Extension to existing QuickBooksTarget from C8
public async Task<bool> VoidDocumentAsync(
    int orgId, string documentType, string externalId)
{
    // QB void requires fetching current SyncToken first
    string qbEntity = documentType switch
    {
        "SalesInvoice:Void"  => "invoice",
        "PurchaseBill:Void"  => "bill",
        "Payment:Void"       => "payment",
        "BillPayment:Void"   => "billpayment",
```

```
        _ => throw new NotSupportedException($"Void not supported for
{documentType}")
    };

    var current = await _qbClient.GetAsync<dynamic>(orgId, $"
{qbEntity}/{externalId}");
    string syncToken = current?.SyncToken?.ToString()
        ?? throw new InvalidOperationException("Cannot void: SyncToken not
found");

    // QB void endpoint
    await _qbClient.PostAsync(orgId, $"{qbEntity}?operation=void", new
    {
        Id = externalId,
        SyncToken = syncToken
    });

    return true;
}
```

21.6 FinDocType Enum — Extended

```csharp
// Add to existing FinDocType enum from C1
public enum FinDocType
{
    // ... existing types ...
    SalesInvoice,
    PurchaseBill,
    Payment,
    BillPayment,
    CreditNote,
    SalesReturn,
    POSSale,
    POSReturn,
    InventoryAdjustment,
    ProductionCost,
    ProductionCompletion,
    ManualJournal,
```

```
    PurchaseReturn,
    BankFee,

    // ── NEW: Added for complete integration coverage ──
    Expense,            // Business expense → QB Purchase
    VendorCredit,       // Purchase return credit → QB VendorCredit
    RefundReceipt,      // Customer refund → QB RefundReceipt
    SalesReceipt,       // POS cash sale (no AR) → QB SalesReceipt
    DepositSlip         // Bank deposit → QB Deposit
}
```

21.7 Design Rules for Document Lifecycle

Rule	Detail
Void goes through gateway	Core modules call `_gateway.VoidAsync()` — never QB void API directly.
Void requires external ID	The gateway looks up the external document ID from `SyncLogs` before writing the outbox entry. If the original document was never synced, void is a no-op for external targets.
Supplier reversal = flag only	No QB transaction sent. Entry written to SyncLogs as `FlaggedForReview`. Accountant handles the QB side manually. Automation deferred per feature table.
Credit creates new QB document	QB CreditMemo / VendorCredit / RefundReceipt are separate documents, not modifications to the original. The gateway writes a NEW outbox entry with the credit document type.
Internal GL always reversed	Void and credit operations always create reversing journal entries in the internal GL, regardless of external provider behavior.

---

22. C18 — Sync Control Configuration & POS Hold/Release

Location: `SMS.Finance/Entities/SyncControlConfig.cs`, `SMS.Finance/Services/SyncControlService.cs`

22.1 Entity: `SyncControlConfig`

```csharp
namespace SMS.Finance.Entities;

/// <summary>
/// Per-tenant, per-provider, per-document-type toggle that controls
/// whether a specific transaction type is synced to the external system.
///
/// Enables phased rollout:
///   Phase 1: Enable SalesInvoice + Customer Payment only
///   Phase 2: Enable POS Sales, Supplier Bills, etc.
///
/// When a document type is disabled, the outbox processor either:
///   - Skips the entry (default)
///   - Holds it for later release (POS mode)
/// </summary>
[Table("SyncControlConfigs", Schema = "finance")]
public class SyncControlConfig
{
    [Key]
    public int Id { get; set; }

    public int OrgId { get; set; }
    public ExternalSystemType Provider { get; set; }

    /// <summary>FinDocType or SyncEntityType as string.</summary>
    [MaxLength(50)]
    public string DocumentType { get; set; } = null!;

    /// <summary>When false, postings of this type are not sent to this
provider.</summary>
    public bool IsEnabled { get; set; }

    /// <summary>
    /// When true AND IsEnabled=false, outbox entries are written with
status=Held
    /// instead of being skipped. When IsEnabled flips to true, held entries
    /// are released to Pending.
    ///
    /// Used for POS: transactions accumulate while sync is disabled,
    /// then batch-release when enabled.
```

```
    /// </summary>
    public bool HoldWhenDisabled { get; set; }

    /// <summary>Display label for UI.</summary>
    [MaxLength(100)]
    public string? DisplayLabel { get; set; }

    public DateTime UpdatedAtUtc { get; set; }
    public string UpdatedBy { get; set; } = "system";
}
```

22.2 EF Core Configuration

```csharp
modelBuilder.Entity<SyncControlConfig>(entity =>
{
    entity.ToTable("SyncControlConfigs", "finance");

    entity.HasIndex(e => new { e.OrgId, e.Provider, e.DocumentType })
          .IsUnique()
          .HasDatabaseName("IX_SyncControlConfigs_OrgId_Provider_DocType");
});
```

22.3 Sync Control Service

```csharp
namespace SMS.Finance.Services;

public interface ISyncControlService
{
    /// <summary>Check if a specific document type is enabled for sync.
</summary>
    Task<SyncControlDecision> CheckAsync(
        int orgId, ExternalSystemType provider, string documentType);

    /// <summary>Release all held entries for a document type.</summary>
    Task<int> ReleaseHeldEntriesAsync(
        int orgId, ExternalSystemType provider, string documentType);
```

```
    /// <summary>Seed default configs when a provider is first connected.
</summary>
    Task SeedDefaultsAsync(int orgId, ExternalSystemType provider);
}

public class SyncControlDecision
{
    public bool IsEnabled { get; init; }
    public bool HoldWhenDisabled { get; init; }
    public SyncAction Action => IsEnabled ? SyncAction.Process
                              : HoldWhenDisabled ? SyncAction.Hold
                              : SyncAction.Skip;

    public enum SyncAction { Process, Hold, Skip }
}

public class SyncControlService : ISyncControlService
{
    private readonly FinanceDbContext _financeDb;

    public SyncControlService(FinanceDbContext financeDb)
    {
        _financeDb = financeDb;
    }

    public async Task<SyncControlDecision> CheckAsync(
        int orgId, ExternalSystemType provider, string documentType)
    {
        var config = await _financeDb.SyncControlConfigs
            .FirstOrDefaultAsync(c => c.OrgId == orgId
                                   && c.Provider == provider
                                   && c.DocumentType == documentType);

        if (config == null)
        {
            // No config = enabled by default (backward compatible)
            return new SyncControlDecision
            {
                IsEnabled = true,
```

```
            HoldWhenDisabled = false
        };
    }

    return new SyncControlDecision
    {
        IsEnabled = config.IsEnabled,
        HoldWhenDisabled = config.HoldWhenDisabled
    };
}

public async Task<int> ReleaseHeldEntriesAsync(
    int orgId, ExternalSystemType provider, string documentType)
{
    // Release held financial posting outbox entries
    var heldFinancial = await _financeDb.FinancialPostingOutbox
        .Where(e => e.OrgId == orgId
                 && e.Provider == provider.ToString()
                 && e.DocumentType == documentType
                 && e.Status == OutboxStatus.Held)
        .ToListAsync();

    foreach (var entry in heldFinancial)
    {
        entry.Status = OutboxStatus.Pending;
        entry.NextRetryAtUtc = null;
    }

    // Release held entity sync outbox entries
    var heldEntity = await _financeDb.EntitySyncOutbox
        .Where(e => e.OrgId == orgId
                 && e.Provider == provider.ToString()
                 && e.EntityType == documentType
                 && e.Status == OutboxStatus.Held)
        .ToListAsync();

    foreach (var entry in heldEntity)
    {
        entry.Status = OutboxStatus.Pending;
        entry.NextRetryAtUtc = null;
```

```
        }

        await _financeDb.SaveChangesAsync();
        return heldFinancial.Count + heldEntity.Count;
    }

    public async Task SeedDefaultsAsync(int orgId, ExternalSystemType provider)
    {
        // Default sync control: enable safe types, hold POS, disable risky
types
        var defaults = new List<SyncControlConfig>
        {
            new() { OrgId = orgId, Provider = provider, DocumentType =
"SalesInvoice",
                    IsEnabled = true, HoldWhenDisabled = false, DisplayLabel =
"Sales Invoices" },
            new() { OrgId = orgId, Provider = provider, DocumentType =
"Payment",
                    IsEnabled = true, HoldWhenDisabled = false, DisplayLabel =
"Customer Payments" },
            new() { OrgId = orgId, Provider = provider, DocumentType =
"CreditNote",
                    IsEnabled = true, HoldWhenDisabled = false, DisplayLabel =
"Credit Notes & Refunds" },
            new() { OrgId = orgId, Provider = provider, DocumentType =
"POSSale",
                    IsEnabled = false, HoldWhenDisabled = true, DisplayLabel =
"POS Sales" },
            new() { OrgId = orgId, Provider = provider, DocumentType =
"POSReturn",
                    IsEnabled = false, HoldWhenDisabled = true, DisplayLabel =
"POS Returns" },
            new() { OrgId = orgId, Provider = provider, DocumentType =
"PurchaseBill",
                    IsEnabled = false, HoldWhenDisabled = false, DisplayLabel
= "Supplier Bills" },
            new() { OrgId = orgId, Provider = provider, DocumentType =
"BillPayment",
                    IsEnabled = false, HoldWhenDisabled = false, DisplayLabel
= "Supplier Payments" },
```

```
            new() { OrgId = orgId, Provider = provider, DocumentType =
"Expense",
                    IsEnabled = false, HoldWhenDisabled = false, DisplayLabel
= "Business Expenses" },
            new() { OrgId = orgId, Provider = provider, DocumentType =
"ManualJournal",
                    IsEnabled = false, HoldWhenDisabled = false, DisplayLabel
= "Journal Entries" },
            new() { OrgId = orgId, Provider = provider, DocumentType =
"InventoryAdjustment",
                    IsEnabled = false, HoldWhenDisabled = false, DisplayLabel
= "Inventory Postings" },
        };

        foreach (var config in defaults)
        {
            config.UpdatedAtUtc = DateTime.UtcNow;
            bool exists = await _financeDb.SyncControlConfigs
                .AnyAsync(c => c.OrgId == orgId
                            && c.Provider == provider
                            && c.DocumentType == config.DocumentType);
            if (!exists)
                _financeDb.SyncControlConfigs.Add(config);
        }

        await _financeDb.SaveChangesAsync();
    }
}
```

22.4 OutboxStatus Enum — Extended

```csharp
public enum OutboxStatus
{
    Pending,       // Ready for processing
    Processing,    // Currently being processed
    Completed,     // Successfully synced
    Failed,        // Failed, will retry
    DeadLetter,    // Max retries exceeded
```

```
    Held           // ← NEW: Disabled document type with hold mode
}
```

22.5 Integration with Gateway and Outbox Processor

```csharp
// In FinancialPostingGateway.PostAsync() — sync control check
public async Task<PostingResult> PostAsync(FinancialTransaction transaction)
{
    // ... existing license, idempotency, target resolution ...

    foreach (var target in externalTargets)
    {
        // ── NEW: Check sync control ──
        var control = await _syncControl.CheckAsync(
            transaction.OrgId, target.Provider,
transaction.DocumentType.ToString());

        if (control.Action == SyncControlDecision.SyncAction.Skip)
        {
            _logger.LogDebug("Sync disabled for {DocType} to {Provider},
skipping",
                transaction.DocumentType, target.Provider);
            continue;
        }

        var outboxEntry = new FinancialPostingOutbox { /* ... */ };

        if (control.Action == SyncControlDecision.SyncAction.Hold)
        {
            outboxEntry.Status = OutboxStatus.Held;
            _logger.LogDebug("Sync disabled with hold for {DocType} to
{Provider}, holding",
                transaction.DocumentType, target.Provider);
        }

        _financeDb.FinancialPostingOutbox.Add(outboxEntry);
    }
```

```
    // ... rest of method ...
}
```

22.6 POS Hold/Release Flow

```
POS SYNC — HOLD AND RELEASE
──────────────────────────────

Phase 1 (POS Sync Disabled, HoldWhenDisabled = true):
  POS Sale #1 → Gateway → Sync Control → Status: Held
  POS Sale #2 → Gateway → Sync Control → Status: Held
  POS Sale #3 → Gateway → Sync Control → Status: Held

  OutboxProcessor ignores Held entries.
  POS transactions accumulate safely in the outbox.

Phase 2 (Admin enables POS Sync):
  PUT /api/finance/sync-control/QuickBooksOnline/POSSale { isEnabled: true }
    → SyncControlService sets IsEnabled = true
    → SyncControlService.ReleaseHeldEntriesAsync() runs automatically
    → POS Sales #1, #2, #3 move from Held → Pending
    → OutboxProcessor picks them up on next cycle
    → All POS transactions sync to QuickBooks
```

22.7 Sync Control UI Panel

Route: `/settings/finance/integrations/quickbooks/sync-control`

```
┌──────────────────────────────────────────────────────────────────┐
│  QUICKBOOKS — SYNC CONTROL                                       │
│                                                                  │
│  Enable or disable synchronization per document type.            │
│  Held transactions will sync when enabled.                       │
│                                                                  │
│  Document Type          Sync     Hold     Status                 │
│  ──────────────────────────────────────────────────────────      │
```

```
│  Sales Invoices         [ON ]    ──       ● Active          │
│  Customer Payments      [ON ]    ──       ● Active          │
│  Credit Notes & Refunds [ON ]    ──       ● Active          │
│  POS Sales              [OFF]    [ON ]    ○ Held (47)       │
│  POS Returns            [OFF]    [ON ]    ○ Held (3)        │
│  Supplier Bills         [OFF]    [OFF]    ○ Disabled        │
│  Supplier Payments      [OFF]    [OFF]    ○ Disabled        │
│  Business Expenses      [OFF]    [OFF]    ○ Disabled        │
│  Journal Entries        [OFF]    [OFF]    ○ Disabled        │
│  Inventory Postings     [OFF]    [OFF]    ○ Disabled        │
│                                                             │
│  ⓘ Enabling a held type will release 50 queued transactions │
│                                                             │
│  [Save Configuration]                                       │
└─────────────────────────────────────────────────────────────┘
```

22.8 Design Rules for Sync Control

Rule	Detail
Per-document-type granularity	Each FinDocType / SyncEntityType can be independently enabled or disabled per provider.
Hold mode for POS	POS defaults to disabled with `HoldWhenDisabled = true`. Transactions accumulate safely.
Auto-release on enable	When a held document type is enabled, all held outbox entries are released to Pending automatically.
No data loss	Disabling a type never deletes outbox entries. Skip mode = entry not created. Hold mode = entry created with Held status.
Seeded on connect	When a provider is first connected (C3/C8 OAuth flow), default sync control configs are seeded with safe defaults.
Backward compatible	No config row = enabled. Existing tenants with no SyncControlConfigs continue syncing everything.

---

23. Database Migrations

#	Migration Name	Schema	Description

M1	`AddTenantFinanceConfigs`	`finance`	Create `finance.TenantFinanceConfigs` table
M2	`AddExternalSystemConnections`	`finance`	Create `finance.ExternalSystemConnections` table
M3	`AddExternalAccountMappings`	`finance`	Create `finance.ExternalAccountMappings` table
M4	`AddExternalEntityMappings`	`finance`	Create `finance.ExternalEntityMappings` table
M5	`AddExternalProviderDefaults`	`finance`	Create `finance.ExternalProviderDefaults` table
M6	`AddExternalDocumentOverrides`	`finance`	Create `finance.ExternalDocumentOverrides` table
M7	`AddFinancialPostingOutbox`	`finance`	Create `finance.FinancialPostingOutbox` table
M8	`AddSyncLogs`	`finance`	Create `finance.SyncLogs` table
M9	`AddJournalEntries`	`finance`	Create `finance.JournalEntries` + `finance.JournalEntryLines` tables (basic structure for internal GL target — full GL engine in Addendum 46)
M10	`SeedQuickBooksProviderDefaults`	`finance`	Seed default QB setting keys for reference
M11	`AddEntitySyncOutbox`	`finance`	Create `finance.EntitySyncOutbox` table with indexes on (OrgId, Status, NextRetryAtUtc), unique (OrgId, IdempotencyKey), and (OrgId, EntityType, EntityId)
M12	`AddEntitySyncEnabledToConnections`	`finance`	Add `EntitySyncEnabled` (bit, default 1) column to `finance.ExternalSystemConnections`
M13	`AddSyncLogEntityTypeSupport`	`finance`	Add index on `finance.SyncLogs` for (OrgId, DocumentType, SmsDocumentId) to support entity sync log queries
M14	`AddExternalTaxMappings`	`finance`	Create `finance.ExternalTaxMappings` table with unique index on (OrgId, SmsTaxCode, Provider)
M15	`AddSyncControlConfigs`	`finance`	Create `finance.SyncControlConfigs` table with unique index on (OrgId, Provider, DocumentType)
M16	`AddOutboxHeldStatus`	`finance`	Add `Held` value to OutboxStatus enum check constraint on `finance.FinancialPostingOutbox` and `finance.EntitySyncOutbox`

---

24. API Endpoints

24.1 Finance Configuration APIs

Method	Route	Description	Auth
`GET`	`/api/finance/config`	Get tenant's finance configuration	Tenant Admin
`PUT`	`/api/finance/config`	Update finance config (enable/disable internal GL)	Tenant Admin
`GET`	`/api/finance/connections`	List external system connections	Tenant Admin
`POST`	`/api/finance/connections/{provider}/connect`	Initiate OAuth flow for provider	Tenant Admin
`POST`	`/api/finance/connections/{provider}/callback`	OAuth callback (receive tokens)	System
`DELETE`	`/api/finance/connections/{id}`	Disconnect external system	Tenant Admin
`POST`	`/api/finance/connections/{id}/test`	Test connection (ping external API)	Tenant Admin

24.2 Mapping APIs

Method	Route	Description	Auth
`GET`	`/api/finance/mappings/accounts/{provider}`	List account mappings for provider	Tenant Admin
`POST`	`/api/finance/mappings/accounts/{provider}`	Create/update account mapping	Tenant Admin
`DELETE`	`/api/finance/mappings/accounts/{id}`	Delete account mapping	Tenant Admin
`GET`	`/api/finance/mappings/entities/{provider}`	List entity mappings	Tenant Admin
`POST`	`/api/finance/mappings/entities/{provider}`	Create/update entity mapping	Tenant Admin
`POST`	`/api/finance/mappings/accounts/{provider}/auto-map`	Auto-map by matching account names	Tenant Admin
`GET`	`/api/finance/mappings/coverage/{provider}`	Get mapping coverage report	Tenant Admin

24.3 Provider Defaults APIs

Method	Route	Description	Auth
`GET`	`/api/finance/provider-defaults/{provider}`	List provider-specific defaults	Tenant Admin
`PUT`	`/api/finance/provider-defaults/{provider}`	Update defaults (batch)	Tenant Admin
`GET`	`/api/finance/document-overrides/{provider}`	List document-type overrides	Tenant Admin
`PUT`	`/api/finance/document-overrides/{provider}`	Update overrides (batch)	Tenant Admin

24.4 Sync & Monitoring APIs

Method	Route	Description	Auth
`GET`	`/api/finance/sync/logs`	List sync logs (filterable, paginated)	Tenant Admin
`GET`	`/api/finance/sync/summary`	Sync summary (counts by status)	Tenant Admin
`GET`	`/api/finance/sync/outbox`	View outbox entries (pending, failed, dead letter)	Tenant Admin
`POST`	`/api/finance/sync/outbox/{id}/retry`	Retry a failed/dead-letter entry	Tenant Admin
`POST`	`/api/finance/sync/outbox/{id}/skip`	Skip (dismiss) a dead-letter entry	Tenant Admin

24.5 Entity Sync APIs

Method	Route	Description	Auth
`POST`	`/api/finance/entity-sync/push`	Manually trigger entity sync for a specific entity	Tenant Admin
`POST`	`/api/finance/entity-sync/{provider}/import`	Import entities from provider to create mapping entries	Tenant Admin
`GET`	`/api/finance/entity-sync/{provider}/coverage`	Get entity mapping coverage report (% mapped per entity type)	Tenant Admin
`GET`	`/api/finance/entity-sync/outbox`	View entity sync outbox entries

(pending, failed, dead letter) | Tenant Admin |
`POST`	`/api/finance/entity-sync/outbox/{id}/retry`	Retry a failed/dead-letter entity sync entry	Tenant Admin
`POST`	`/api/finance/entity-sync/outbox/{id}/skip`	Skip (dismiss) a dead-letter entity sync entry	Tenant Admin
`GET`	`/api/finance/entity-sync/logs`	List entity sync logs (filterable by entity type, status)	Tenant Admin
`POST`	`/api/finance/entity-sync/{provider}/link`	Link an imported (unlinked) external entity to an SMS entity	Tenant Admin
`DELETE`	`/api/finance/entity-sync/{provider}/link/{mappingId}`	Unlink an entity mapping	Tenant Admin

24.6 Tax Mapping APIs

Method	Route	Description	Auth
`GET`	`/api/finance/mappings/tax/{provider}`	List tax code mappings for provider	Tenant Admin
`POST`	`/api/finance/mappings/tax/{provider}`	Create/update tax code mapping	Tenant Admin
`DELETE`	`/api/finance/mappings/tax/{id}`	Delete tax code mapping	Tenant Admin
`POST`	`/api/finance/mappings/tax/{provider}/import`	Import tax codes from provider (auto-map by rate)	Tenant Admin

24.7 Sync Control APIs

Method	Route	Description	Auth
`GET`	`/api/finance/sync-control/{provider}`	List sync control configs for provider	Tenant Admin
`PUT`	`/api/finance/sync-control/{provider}/{documentType}`	Update sync control (enable/disable, hold mode)	Tenant Admin
`POST`	`/api/finance/sync-control/{provider}/{documentType}/release`	Release held entries for a document type	Tenant Admin

24.8 Document Lifecycle APIs

Method	Route	Description	Auth

`POST`	`/api/finance/gateway/void`	Void a previously synced document in external systems	Tenant Admin
`POST`	`/api/finance/gateway/credit`	Issue credit memo / vendor credit against a synced document	Tenant Admin
`GET`	`/api/finance/sync/flagged-reviews`	List supplier reversals flagged for accountant review	Tenant Admin
`POST`	`/api/finance/sync/flagged-reviews/{id}/resolve`	Mark a flagged reversal as resolved by accountant	Tenant Admin

24.9 Validation & Coverage APIs

Method	Route	Description	Auth
`GET`	`/api/finance/mappings/coverage/{provider}`	Full mapping coverage report (accounts, entities, tax codes)	Tenant Admin
`POST`	`/api/finance/validation/check`	Run pre-sync validation on a pending document without actually syncing	Tenant Admin

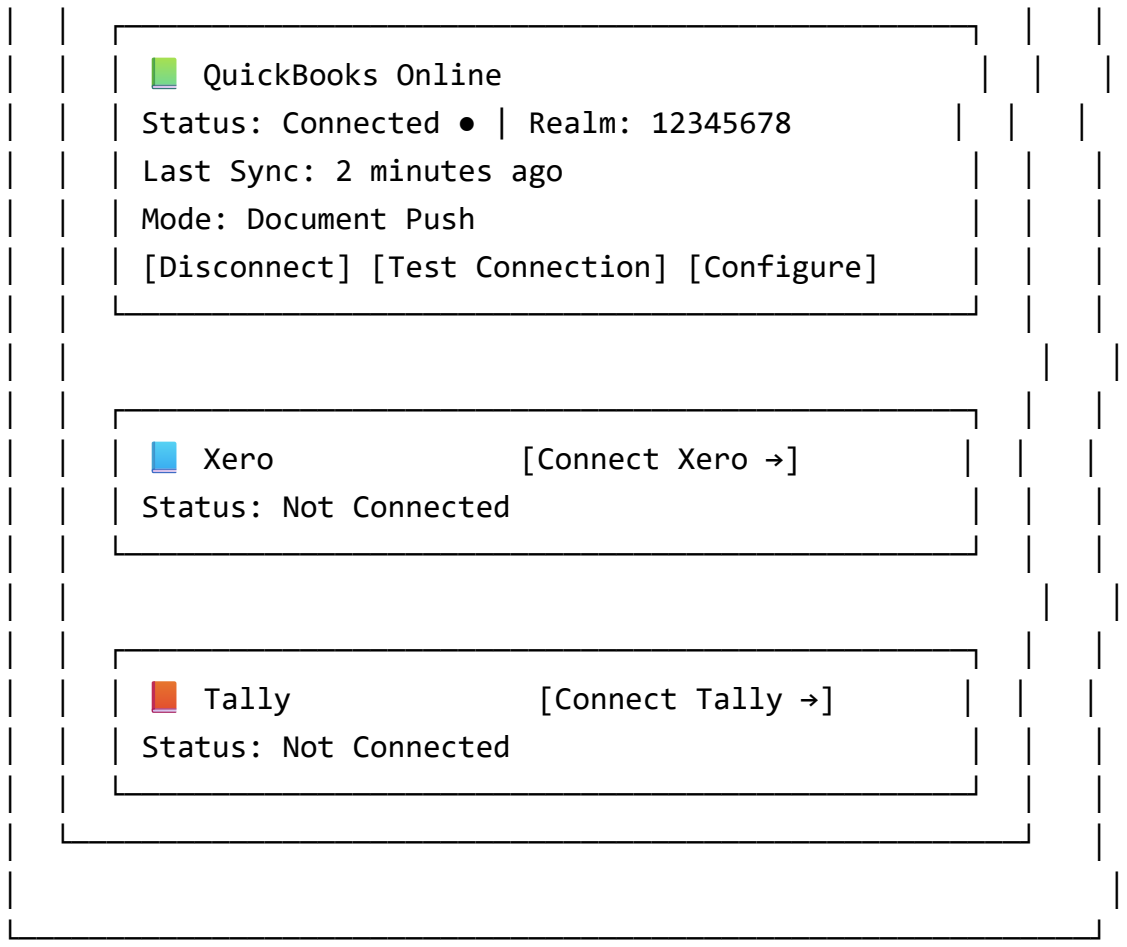
---

25. UI Wireframes & Specifications

25.1 Finance Settings — External Integrations Panel

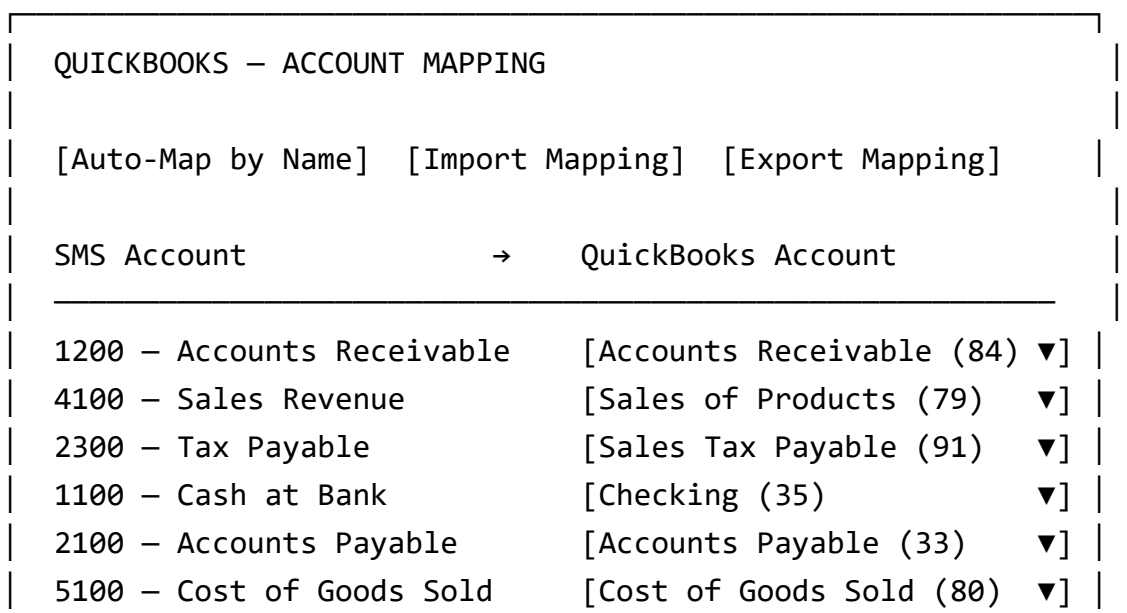
Route: `/settings/finance/integrations`

```
┌──────────────────────────────────────────────────────────────┐
│  FINANCE SETTINGS                                            │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  Internal Accounting                                         │
│  ┌──────────────────────────────────────────────────────┐    │
│  │  Enable Internal GL    [Toggle: ON/OFF]              │    │
│  │  Auto-Post to GL       [Toggle: ON/OFF]              │    │
│  │  Default Currency       [USD ▼]                      │    │
│  └──────────────────────────────────────────────────────┘    │
│                                                              │
│  External Connections                                        │
│  ┌──────────────────────────────────────────────────────┐    │
```



25.2 Account Mapping Panel

Route: `/settings/finance/integrations/quickbooks/mappings`



```
│  ...                              ...                                  │
│                                                                        │
│  ⚠ 4 accounts unmapped — these will fail on sync                       │
│                                                                        │
│  [Save Mappings]                                                       │
└────────────────────────────────────────────────────────────────────────┘
```

25.3 Sync Status Dashboard

Route: `/settings/finance/integrations/sync-status`

```
┌────────────────────────────────────────────────────────────────────────┐
│  SYNC STATUS — QUICKBOOKS                                              │
│                                                                        │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────┐    │
│  │   1,247      │  │      3       │  │      1       │  │    0     │    │
│  │   Synced     │  │   Pending    │  │   Failed     │  │   Dead   │    │
│  │  (24 hrs)    │  │              │  │              │  │  Letter  │    │
│  └──────────────┘  └──────────────┘  └──────────────┘  └──────────┘    │
│                                                                        │
│  Recent Activity                                                       │
│  ┌──────────────────────────────────────────────────────────────┐      │
│  │ Doc #       | Type    | QB ID  | Status  | Time            │      │
│  │ INV-0042    | Invoice | 156    | Synced  | 2m ago          │      │
│  │ PAY-0019    | Payment | 89     | Synced  | 5m ago          │      │
│  │ BILL-0033   | Bill    | —      | Failed  | 8m ago          │      │
│  │             |         |        | ↳ "Customer not          │      │
│  │             |         |        |   mapped in QB"          │      │
│  └──────────────────────────────────────────────────────────────┘      │
│                                                                        │
│  [View All Logs]  [Export Report]                                      │
└────────────────────────────────────────────────────────────────────────┘
```

25.4 Entity Mapping Panel

Route: `/settings/finance/integrations/quickbooks/entity-mappings`

```
┌──────────────────────────────────────────────────────────────┐
│  QUICKBOOKS — ENTITY MAPPING                                 │
│                                                              │
│  [Customers]  [Vendors]  [Products]  [Accounts]   ← tabs     │
│                                                              │
│   ┌──────────────────────────────────────────────────────┐   │
│   │  Coverage: 87% (213 of 245 customers mapped)         │   │
│   │  ████████████████████▒▒                              │   │
│   └──────────────────────────────────────────────────────┘   │
│                                                              │
│  [Import from QuickBooks]  [Auto-Link by Name]               │
│                                                              │
│  SMS Customer              →    QB Customer       Status     │
│  ──────────────────────────────────────────────────────────  │
│  Acme Corp (#42)                Acme Corp (QB#156)  Synced   │
│  Global Trade Ltd (#78)         Global Trade (QB#89) Synced  │
│  New Client Inc (#201)          [Select QB Customer ▼] ──    │
│  Fresh Import (#—)              Mario's Pizza (QB#234)  ⚠    │
│    └─ Imported from QB, not linked to SMS entity             │
│  ...                           ...                           │
│                                                              │
│  ⚠ 32 SMS customers unmapped — new entities won't sync       │
│  ⚠ 5 QB entities imported but unlinked                       │
│                                                              │
│  [Save Mappings]                                             │
└──────────────────────────────────────────────────────────────┘
```

25.5 Entity Sync Status Panel

Route: `/settings/finance/integrations/entity-sync-status`

```
┌──────────────────────────────────────────────────────────────┐
│  ENTITY SYNC STATUS — QUICKBOOKS                             │
│                                                              │
│  Entity Sync  [Toggle: ON/OFF]                               │
│                                                              │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐      │
```

```
|  |   1,892    |  |     5      |  |     2      |  |   0    ||
|  |   Synced   |  |  Pending   |  |  Failed    |  |  Dead  ||
|  |  (all time)|  |            |  |            |  | Letter ||
|  └────────────┘  └────────────┘  └────────────┘  └────────┘|
|                                                            |
|  Recent Entity Sync Activity                               |
|  ┌──────────────────────────────────────────────────────┐   |
|  | Entity    | Type     | QB ID  | Action  | Time      |   |
|  | Cust #42  | Customer | 156    | Update  | 1m ago    |   |
|  | Prod #88  | Product  | 312    | Create  | 3m ago    |   |
|  | Vend #15  | Vendor   | —      | Create  | 5m ago    |   |
|  |           |          |        | ↳ "DisplayName      |   |
|  |           |          |        |   already in use"   |   |
|  └──────────────────────────────────────────────────────┘   |
|                                                            |
|  [View Entity Sync Logs]  [View Outbox]                    |
└────────────────────────────────────────────────────────────┘
```

25.6 Tax Code Mapping Panel

Route: `/settings/finance/integrations/quickbooks/tax-mappings`

```
┌────────────────────────────────────────────────────────────┐
|  QUICKBOOKS — TAX CODE MAPPING                             |
|                                                            |
|  [Import Tax Codes from QuickBooks]                        |
|                                                            |
|  SMS Tax Code     Rate    →   QB Tax Code        Status    |
|  ──────────────────────────────────────────────────────    |
|  VAT-17           17%         [GST on Sales ▼]    ● Mapped |
|  VAT-5            5%          [Reduced Rate ▼]    ● Mapped |
|  TAX-EXEMPT       0%          [NON (Auto)    ▼]   ● Auto   |
|  CESS-1           1%          [Select Tax Code ▼] ○ Unmapped|
|  ...                          ...                          |
|                                                            |
|  QB Tax Codes (Imported, Not Linked)                       |
|  ──────────────────────────────────────────────────────    |
|  Out of Scope (QB#5)          [Link to SMS Tax Code ▼]     |
```

```
│  Zero Rate (QB#7)              [Link to SMS Tax Code ▼]       │
│                                                               │
│  ⚠ 1 SMS tax code unmapped — invoices with this tax will     │
│    fail to sync                                               │
│                                                               │
│  [Save Mappings]                                              │
└──────────────────────────────────────────────────────────────┘
```

25.7 Pre-Sync Validation Report Panel

Route: `/settings/finance/integrations/quickbooks/validation`

```
┌──────────────────────────────────────────────────────────────┐
│  QUICKBOOKS — SYNC READINESS                                  │
│                                                               │
│  ┌──────────────────────────────────────────────────┐        │
│  │  Overall: ⚠ NOT READY (2 blocking issues)        │        │
│  └──────────────────────────────────────────────────┘        │
│                                                               │
│  Mapping Coverage                                             │
│  ──────────────────────────────────────────────────────       │
│  Accounts      ████████████████████  96% (48/50)  ● Good     │
│  Customers     ██████████████████    87% (213/245) ● Good    │
│  Vendors       ██████████            52% (34/65)  ⚠ Low      │
│  Products      ██████████████████    89% (312/350) ● Good    │
│  Tax Codes     ████████████          75% (6/8)    ⚠ Low       │
│                                                               │
│  Blocking Issues                                              │
│  ──────────────────────────────────────────────────────       │
│  ⚠ Account '5200 — Admin Expenses' not mapped                │
│  ⚠ Account '6100 — Depreciation' not mapped                  │
│  ⚠ Tax code 'CESS-1' not mapped                              │
│                                                               │
│  [Refresh Coverage]  [Go to Account Mapping]                  │
└──────────────────────────────────────────────────────────────┘
```

25.8 Sync Control Panel

Route: `/settings/finance/integrations/quickbooks/sync-control`

See wireframe in §22.7 (C18 — Sync Control Configuration).

---

26. Business Rules

#	Rule	Enforcement
BR-01	Every financial posting MUST have balanced debits and credits (total debits = total credits within 0.001 tolerance)	Gateway validates before any target dispatch
BR-02	Duplicate postings with the same idempotency key are silently skipped	Gateway checks `finance.JournalEntries.IdempotencyKey`
BR-03	If Finance module is not licensed, gateway returns `ModuleNotLicensed` — source document still posts successfully	Gateway checks `ModuleActivations` table
BR-04	If no targets are configured (no internal GL, no external connections), gateway returns `NoTargetConfigured` — source document still posts	Target resolver returns empty list
BR-05	Internal GL entries are added to the same `DbContext` as the source document — committed in one transaction	`TransactionScope` wraps both contexts
BR-06	External API calls always go through the outbox — never called synchronously from the gateway	Gateway writes to `FinancialPostingOutbox`; Hangfire processes
BR-07	Outbox entries retry with exponential backoff: 30s, 2m, 8m, 32m, 2h	`OutboxProcessor.HandleFailure()`
BR-08	After max retries (default 5), outbox entry moves to `DeadLetter` status	Admin must manually retry or skip
BR-09	OAuth tokens are encrypted at rest using Data Protection API	`ExternalSystemConnections.AccessTokenEncrypted`
BR-10	OAuth access tokens are refreshed automatically on 401 response	`QuickBooksApiClient` retries once after refresh
BR-11	Unmapped accounts or entities cause the specific outbox entry to fail, not the entire posting	Error recorded per-entry in outbox
BR-12	Provider-specific fields (QB Class, Dept, Terms) come from `ExternalProviderDefaults` and `ExternalDocumentOverrides` — never from core

module code | Enrichment service loads and attaches |
BR-13	`FinanceDbContext` has no foreign keys to sales, purchase, pos, or inventory schemas	Code review enforcement
BR-14	Source document identity stored as `SourceDocumentId` (int) + `SourceDocumentType` (string) + `SourceModule` (string) — plain values, not FKs	Entity configuration in `FinanceDbContext.OnModelCreating()`
BR-15	Entity sync always goes through `IEntitySyncGateway.PushAsync()` — core modules NEVER call QuickBooks Customer/Vendor/Item APIs directly	Code review enforcement + architecture tests
BR-16	Entity sync is always async via outbox — no synchronous API calls to external systems during entity create/update	`EntitySyncGateway.PushAsync()` only writes outbox rows
BR-17	Entity sync outbox entries retry with same exponential backoff as financial outbox: 30s, 2m, 8m, 32m, 2h	`EntitySyncOutboxProcessor.HandleFailure()`
BR-18	Initial import from QB creates mapping entries with `InternalEntityId = 0` (unlinked) — it NEVER creates SMS entities (Customers, Vendors, Products)	`QuickBooksMasterDataAdapter.ImportEntitiesAsync()`
BR-19	Entity sync respects `EntitySyncEnabled` flag on `ExternalSystemConnections` — if disabled for a provider, no entity sync outbox entries are created for that provider even if the connection is active	`EntitySyncTargetResolver.GetActiveProvidersAsync()`
BR-20	QB sparse update requires fetching current `SyncToken` before update — adapter calls GET first, then POST with SyncToken	`QuickBooksMasterDataAdapter.SyncCustomerAsync()` et al.
BR-21	Product sync to QB resolves `IncomeAccountRef`, `ExpenseAccountRef`, `AssetAccountRef` from `ExternalAccountMappings` — if accounts are unmapped, the outbox entry fails with a retryable error	`QuickBooksMasterDataAdapter.SyncProductAsync()`
BR-22	Entity snapshot is serialized into `EntitySyncOutbox.PayloadJson` — the background processor reads entity data from the outbox entry, NEVER from `ApplicationDbContext`	Separation-ready design
BR-23	Tax code mapping is required for invoice lines with tax — unmapped tax codes cause outbox entry failure with a retryable error	`PreSyncValidator.ValidateFinancialPostingAsync()`
BR-24	Pre-sync validation runs inside the outbox processor BEFORE calling the provider adapter — all missing references are reported at once, not one per retry	`OutboxProcessor.ProcessEntryAsync()`
BR-25	Void operations require the external document ID from SyncLogs — if the original document was never synced to external, void is a no-op for

external targets (internal GL reversal still occurs) |
`FinancialPostingGateway.VoidAsync()` |
BR-26	Supplier invoice reversal is flag-only — no QB transaction is sent; entry is written to SyncLogs as `FlaggedForReview` for accountant handling	`FinancialPostingGateway.VoidAsync()` with `FinDocType.PurchaseBill`
BR-27	QB void requires fetching the current `SyncToken` before calling `POST /{entity}?operation=void`	`QuickBooksTarget.VoidDocumentAsync()`
BR-28	Credit memo / vendor credit creates a NEW QB document, not a modification to the original — the gateway writes a new outbox entry with the credit document type	`FinancialPostingGateway.IssueCreditAsync()`
BR-29	Sync control configs default to all enabled when no config row exists — backward compatible with existing tenants	`SyncControlService.CheckAsync()`
BR-30	Disabled document types with `HoldWhenDisabled = true` cause outbox entries to be written with `Status = Held` — entries are not skipped but not processed	`FinancialPostingGateway.PostAsync()`
BR-31	Enabling a previously held document type automatically releases all held outbox entries to `Pending` status	`SyncControlService.ReleaseHeldEntriesAsync()` via API
BR-32	Default sync control seed enables safe types (SalesInvoice, Payment, CreditNote) and disables with hold for POS types	`SyncControlService.SeedDefaultsAsync()`
BR-33	Tax-exempt mapping (SMS `TAX-EXEMPT` or 0% rate) auto-maps to QB `NON` (non-taxable) during initial import	`TaxMappingService.ImportTaxCodesFromProviderAsync()`

---

27. Test Scenarios

27.1 Gateway Routing Tests

#	Scenario	Input	Expected
T-01	No finance license	PostAsync() for tenant with no FINANCE module	Returns `ModuleNotLicensed`, invoice still posts
T-02	No targets configured	Finance licensed but no internal GL, no external	Returns `NoTargetConfigured`
T-03	Internal GL only	`internal_accounting_enabled = true`, no connections	JournalEntry created, no outbox entry

T-04	QuickBooks only	`internal_accounting_enabled = false`, QB connected	No JournalEntry, one outbox entry for QB
T-05	Both internal + QB	Both enabled	JournalEntry created AND outbox entry for QB
T-06	Unbalanced entry	Debits != Credits	Returns `ValidationFailed` with message
T-07	Duplicate posting	Same idempotency key posted twice	Second call returns `DuplicateSkipped`
T-08	Multiple external	Internal + QB + Xero connected	JournalEntry + two outbox entries

27.2 Outbox Processing Tests

#	Scenario	Expected
T-09	Successful QB push	Outbox entry moves to `Completed`, ExternalId populated
T-10	QB API returns 401	Token refreshed, retry succeeds, entry completed
T-11	QB API returns 500	Entry marked `Failed`, retry_count++, scheduled for retry
T-12	Max retries exceeded	Entry moves to `DeadLetter`
T-13	Dead letter manual retry	Admin clicks retry, entry reset to `Pending`
T-14	Unmapped account	Entry fails with "Account code 4200 not mapped to QuickBooks"
T-15	Unmapped customer	Entry fails with "Customer 42 not mapped to QuickBooks"

27.3 Enrichment Tests

#	Scenario	Expected
T-16	Provider defaults exist	QB Class, Dept, Terms attached to payload
T-17	Document-type override	POS sale uses override ClassRef, not default
T-18	No defaults configured	Payload has empty ProviderFields — QB document created without optional fields
T-19	Account mappings complete	All SMS account codes resolved to QB account IDs

| T-20 | Partial mappings | Unmapped accounts cause clear error in sync log |

27.4 Transaction Atomicity Tests

#	Scenario	Expected
T-21	GL insert fails	TransactionScope rolls back — invoice stays unposted, no outbox entry
T-22	Outbox insert fails	TransactionScope rolls back — invoice stays unposted, no GL entry
T-23	SaveChanges succeeds	Invoice posted + GL entry created + outbox entry created — all in one commit

27.5 Entity Sync Gateway Tests

#	Scenario	Input	Expected
T-24	No finance license	PushAsync() for tenant without FINANCE module	Returns `ModuleNotLicensed`, customer still saves
T-25	No providers configured	Finance licensed but no connections with EntitySyncEnabled	Returns `NoTargetConfigured`
T-26	QB connected, entity sync enabled	Create customer, QB connected	One EntitySyncOutbox entry created
T-27	QB connected, entity sync disabled	Create customer, QB connected but EntitySyncEnabled=false	Returns `NoTargetConfigured`, no outbox entry
T-28	Duplicate sync request	Same idempotency key pushed twice	Second call returns `DuplicateSkipped`
T-29	Multiple providers	QB + Xero both connected with entity sync	Two outbox entries created (one per provider)
T-30	Batch push	PushBatchAsync() with 50 entities	50 outbox entries, duplicates skipped
T-31	Entity + financial in same transaction	Create invoice (PostAsync) + Create customer (PushAsync) in same SaveChanges	Both FinancialPostingOutbox and EntitySyncOutbox entries committed atomically

27.6 Entity Sync Outbox Processing Tests

#	Scenario	Expected
T-32	Successful customer push to QB	Outbox entry Completed,

ExternalEntityMappings entry created, SyncLog entry written |
T-33	Successful vendor push to QB	Same as T-32 for vendor entity type
T-34	Successful product push to QB	Product synced, IncomeAccountRef resolved from ExternalAccountMappings
T-35	Product with unmapped income account	Outbox entry fails with "Account code 4100 not mapped to QuickBooks" — retryable
T-36	QB 401 on entity push	Token refreshed, next retry succeeds
T-37	QB DisplayName conflict	QB rejects duplicate DisplayName — entry fails, retryable
T-38	Max retries exceeded	Entity sync outbox entry moves to DeadLetter, SyncLog records dead letter
T-39	Customer update (mapping exists)	Adapter fetches SyncToken, sends sparse update, no new mapping created

27.7 Initial Import Tests

#	Scenario	Expected
T-40	Import QB customers	All QB customers fetched (paginated), mappings created with InternalEntityId=0
T-41	Import with existing mappings	Already-mapped entities skipped, count reported in result
T-42	Import QB items (products)	QB Items fetched, mappings created — no SMS Products created
T-43	Import QB accounts	Account mappings created in ExternalAccountMappings (not ExternalEntityMappings)
T-44	User links imported entity	Admin links QB entity to SMS entity via UI — InternalEntityId updated from 0 to real ID

27.8 Tax Code Mapping Tests

#	Scenario	Expected
T-45	Import QB tax codes	TaxCode entities fetched from QB, ExternalTaxMapping entries created with empty SmsTaxCode
T-46	Map SMS tax code to QB	Admin links SMS `VAT-17` to QB `GST on Sales` — enrichment resolves correctly
T-47	Tax-exempt auto-map	SMS `TAX-EXEMPT` (0%) auto-maps to QB `NON` during import
T-48	Unmapped tax code in invoice	Invoice line with tax code `CESS-1`

(unmapped) causes outbox entry failure with retryable error |
| T-49 | Tax code enrichment during posting | ProviderEnrichmentService resolves `TaxCode = "VAT-17"` → `ExternalTaxCodeId = "2"` for QB payload |

27.9 Pre-Sync Validation Tests

#	Scenario	Expected
T-50	All references mapped	ValidateFinancialPostingAsync returns IsValid=true, 0 errors
T-51	Multiple missing references	Account 4200, Customer #42, and Tax VAT-5 all unmapped — validator returns all 3 errors at once
T-52	Validation failure in outbox	Outbox processor marks entry Failed with all missing-reference errors, schedules retry
T-53	Product entity with unmapped accounts	ValidateEntitySyncAsync reports unmapped IncomeAccountCode and ExpenseAccountCode
T-54	Coverage API	GET /api/finance/mappings/coverage/QuickBooksOnline returns counts for accounts, customers, vendors, products, tax codes

27.10 Document Lifecycle Tests

#	Scenario	Expected
T-55	Void sales invoice	VoidAsync → outbox entry with `SalesInvoice:Void` type, QB `POST /invoice?operation=void` called
T-56	Void purchase bill (supplier reversal)	VoidAsync → no outbox entry, SyncLog entry with `FlaggedForReview` status
T-57	Void customer payment	VoidAsync → outbox entry, QB `POST /payment?operation=void` called with correct SyncToken
T-58	Issue sales credit note	IssueCreditAsync → outbox entry with `CreditNote` type, QB `POST /creditmemo` called
T-59	Issue vendor credit	IssueCreditAsync → outbox entry with `VendorCredit` type, QB `POST /vendorcredit` called
T-60	Void document never synced to external	VoidAsync for a document with no SyncLog entry for QB → void is no-op for external, internal GL reversal still created
T-61	Void with duplicate idempotency key	Same void request twice → second call returns `DuplicateSkipped`

27.11 Sync Control Tests

#	Scenario	Expected
T-62	Document type disabled (skip mode)	PostAsync for `PurchaseBill` with sync disabled → no outbox entry created, no error
T-63	Document type disabled (hold mode)	PostAsync for `POSSale` with HoldWhenDisabled=true → outbox entry created with `Status=Held`
T-64	Enable held document type	Admin enables POSSale → all Held POSSale outbox entries released to Pending
T-65	No config row (backward compatible)	PostAsync for `SalesInvoice` with no SyncControlConfig row → processed normally (enabled by default)
T-66	Seed defaults on connect	New QB connection established → SyncControlConfigs seeded with default settings
T-67	POS hold accumulation	100 POS sales with sync disabled+hold → 100 Held entries, 0 processed
T-68	POS release batch	Enable POS sync → all 100 held entries move to Pending, outbox processor picks them up

---

28. Development Phases

Phase	Items	Estimated Duration	Dependencies
Phase 1 — Gateway Foundation	C1 (Contract), C2 (FinanceDbContext), C4 (Gateway Implementation)	2 weeks	Addendum 42 (CoA), Addendum 43 (Multi-Tenant)
Phase 2 — Configuration & Mapping	C3 (Tenant Config + Connections), C5 (Account/Entity Mappings), C6 (Provider Defaults + Enrichment)	2 weeks	Phase 1
Phase 3 — Outbox & QuickBooks	C7 (Outbox + Hangfire), C8 (QuickBooks Target + OAuth)	3 weeks	Phase 2
Phase 4 — Monitoring & Integration	C9 (Sync Log + Dashboard UI), C10 (Core Module Integration)	2 weeks	Phase 3
Phase 5 — Entity Sync	C11 (Entity Sync Contract), C12 (Gateway Implementation), C13 (QB Master Data Adapter), C14 (Entity Sync Outbox)	2 weeks	Phase 2 (needs C3 connections, C5 mappings)
Phase 6 — Tax, Validation & Sync Control	C15 (Tax Code Mapping), C16 (Pre-Sync Validation), C18 (Sync Control + POS Hold/Release)	2 weeks	Phase 2 (needs C3 connections, C5/C6 enrichment)

| **Phase 7 — Document Lifecycle** | C17 (Void/Cancel/Reversal, Credit Memo, Expense type, FinDocType extensions) | 1 week | Phase 3 (needs outbox + QB adapter) |
| **Phase 8 — Testing & Stabilization** | End-to-end testing, QB sandbox validation, entity sync E2E, tax mapping E2E, void/credit E2E, sync control E2E, error scenario coverage | 2 weeks | Phase 4 + Phase 5 + Phase 6 + Phase 7 |

Total Estimated Duration: 16 weeks

---

29. Deferred Items

Item	Reason	Target
General Ledger Journal Engine (trial balance, period closing, ledger reports)	Requires this gateway foundation; scope is a separate addendum	Addendum 46
AR/AP Sub-Ledgers & Payment Matching	Depends on GL Journal Engine	Addendum 47
Bank Reconciliation	Depends on GL + inbound bank sync	Addendum 48
Multi-Currency Engine	Cross-cutting across all financial entities	Addendum 49
Tax Configuration & Reporting	Requires GL + AR/AP	Addendum 50
Financial Statements Generator	Requires all sub-ledgers	Addendum 51
Xero Integration Adapter	Gateway supports it; adapter built when customer demand exists	Future
Tally Integration Adapter	Same as Xero	Future
ERPNext Integration Adapter	Same as Xero	Future
Inbound Bank Transaction Sync	Outbound first, inbound in Bank Reconciliation addendum	Addendum 48
Scheduled Full Entity Import from QB	Initial import is covered; scheduled recurring pull (e.g., nightly full sync from QB to SMS mappings) is future scope	Future
Inbound Entity Creation from QB	Current import only creates mapping entries. Auto-creating SMS Customers/Vendors/Products from QB entities is future scope	Future
QuickBooks Desktop Integration	QBD uses a different SDK (Web Connector); separate from QBO	Future

Supplier Reversal Automation (Feature #19)	Currently flag-only (C17). Full automation using void/credit/adjustment workflow when QB accounting rules are clear	Future (Phase 2)
Expense Sync (Feature #24)	`FinDocType.Expense` added to enum (C17). Expense adapter and caller pattern — awaits SMS Expense module	Future (Phase 2)
Inventory Financial Postings (Feature #25)	`FinDocType.InventoryAdjustment` exists in enum. Inventory value + COGS accounting entries — awaits SMS Inventory module integration	Future (Phase 2)
General Journal Entries (Feature #26)	`FinDocType.ManualJournal` exists in enum. Journal entry sync — awaits GL Journal Engine (Addendum 46)	Addendum 46
Reconciliation Engine (Feature #27)	Compare SMS and QB totals and identify mismatches. Requires SyncLogs + QB API query	Future (Phase 2)
Financial Reporting Dashboard (Feature #28)	Accounting sync and reconciliation reports within SMS	Future (Phase 2)
Financial Posting Gateway Extraction	The gateway can be extracted to a separate microservice. All architectural rules in this addendum (separate DbContext, no cross-schema FKs, idempotency keys, serializable DTOs) prepare for this. Extraction happens when business scale justifies it.	Operational decision

---

30. Feature Coverage Matrix

This matrix maps each of the 28 consolidated QuickBooks integration features to the specific change items in this addendum that provide the middleware coverage.

#	Feature	Change Items	Middleware Components	Phase
1	Customer Synchronization	C11, C12, C13, C14	`IEntitySyncGateway.PushAsync()` → `EntitySyncOutbox` → `QuickBooksMasterDataAdapter` (Customer)	Existing → Phase 5
2	Supplier Synchronization	C11, C12, C13, C14, C18	`IEntitySyncGateway.PushAsync()` → `EntitySyncOutbox` → `QuickBooksMasterDataAdapter` (Vendor). SyncControl: disabled by default (C18)	Existing → Phase 5
3	Product / Item Synchronization	C11, C12, C13, C14	

`IEntitySyncGateway.PushAsync()` → `EntitySyncOutbox` → `QuickBooksMasterDataAdapter` (Product). Account refs resolved via C5 mappings | Existing → Phase 5 |
4	Sales Invoice Synchronization	C1, C4, C6, C7, C8, C10	`IFinancialPostingGateway.PostAsync()` → enrichment (C6) → `FinancialPostingOutbox` → `QuickBooksTarget` (Invoice)	Existing → Phase 1-4
5	Sales Invoice Cancellation	C17	`IFinancialPostingGateway.VoidAsync()` → outbox → QB `POST /invoice?operation=void`	Existing → Phase 7
6	POS Sales Synchronization	C1, C4, C7, C8, C10, C18	`IFinancialPostingGateway.PostAsync()` with `FinDocType.POSSale` → SyncControl hold/release (C18) → outbox → QB (SalesReceipt)	Existing → Phase 6
7	Supplier Bill Synchronization	C1, C4, C7, C8, C10, C18	`IFinancialPostingGateway.PostAsync()` with `FinDocType.PurchaseBill` → SyncControl (C18, disabled default) → outbox → QB (Bill)	Existing → Phase 6
8	Supplier Invoice Reversal	C17	`IFinancialPostingGateway.VoidAsync()` with `PurchaseBill` → flag-only (`FlaggedForReview` in SyncLogs). No QB transaction	Existing → Phase 7
9	QuickBooks OAuth 2.0	C3, C8	`ExternalSystemConnections` (encrypted tokens), `QuickBooksApiClient` (token refresh on 401)	Phase 1 → Phase 2-3
10	Chart of Accounts Mapping	C5, C13	`ExternalAccountMappings` table, auto-map by name, import from QB. Used by enrichment (C6)	Phase 1 → Phase 2
11	Tax Mapping	C15	`ExternalTaxMappings` table, `TaxMappingService`, import from QB, auto-map by rate. Integrated into enrichment (C6)	Phase 1 → Phase 6
12	Invoice and Item Validation	C16	`IPreSyncValidator` — checks account, entity, tax mappings before QB push. All-at-once error reporting	Phase 1 → Phase 6
13	Idempotency and Duplicate Prevention	C1, C4, C7, C11, C14	`IdempotencyKey` on `FinancialPostingOutbox` + `EntitySyncOutbox`. Gateway checks unique constraint before writing	Phase 1 → Phase 1-5
14	Retry and Error Handling	C7, C14	Exponential backoff (30s, 2m, 8m, 32m, 2h), max 5 retries, dead-letter queue. OAuth 401 auto-refresh	Phase 1 → Phase 3-5
15	Sync Status and Logs	C9	`SyncLogs` entity, filterable/paginated API, Sync Status Dashboard UI	Phase 1 → Phase 4
16	End-to-End Sandbox Testing	§27 (Test Scenarios)	68 test scenarios (T-01 to T-68) across 11 subsections	Phase 1 → Phase 8
17	Supplier Synchronization Rollout	C18	SyncControl enables supplier entity sync when ready — `SyncControlConfig` toggle for Vendor entity type

Phase 2 → Phase 6 |
18	Supplier Bill Synchronization Rollout	C18	SyncControl enables `PurchaseBill` document type when ready	Phase 2 → Phase 6
19	Supplier Reversal Automation	C17 (partial), Deferred	Flag-only in this addendum. Full automation deferred to Phase 2	Phase 2 → Deferred
20	POS Sales Rollout	C18	SyncControl releases held POS entries when enabled. `HoldWhenDisabled = true` by default	Phase 2 → Phase 6
21	Customer Payment Synchronization	C1, C4, C7, C8	`IFinancialPostingGateway.PostAsync()` with `FinDocType.Payment` → outbox → QB (Payment). SyncControl ready	Phase 2 → Phase 3-4
22	Supplier Payment Synchronization	C1, C4, C7, C8, C18	`IFinancialPostingGateway.PostAsync()` with `FinDocType.BillPayment` → outbox → QB (BillPayment). SyncControl disabled by default	Phase 2 → Phase 3-4
23	Credit Notes and Refunds	C17	`IFinancialPostingGateway.IssueCreditAsync()` → outbox → QB (CreditMemo / RefundReceipt / VendorCredit)	Phase 2 → Phase 7
24	Expense Synchronization	C17 (FinDocType.Expense added), Deferred	Enum added. Full expense adapter and caller pattern deferred — awaits SMS Expense module	Phase 2 → Deferred
25	Inventory Financial Postings	C1 (FinDocType.InventoryAdjustment exists), Deferred	Enum exists. COGS and inventory value accounting entries — awaits SMS Inventory module integration	Phase 2 → Deferred
26	General Journal Entries	C1 (FinDocType.ManualJournal exists), Deferred	Enum exists. Journal entry sync to QB — awaits GL Journal Engine (Addendum 46)	Phase 2 → Addendum 46
27	Reconciliation	Deferred	Compare SMS SyncLogs totals with QB API query results. Build on SyncLogs (C9)	Phase 2 → Deferred
28	Financial Reporting	Deferred	Accounting sync reports and reconciliation dashboards within SMS	Phase 2 → Deferred

Coverage Summary:
- **Fully covered (middleware path defined):** Features 1–18, 20–23 (22 of 28)
- **Partially covered (enum/foundation added, adapter deferred):** Features 19, 24–26 (4 of 28)
- **Deferred (no middleware component yet):** Features 27–28 (2 of 28)

End of FSD Addendum 44